中文自学资料 · 案例与习题

数据结构学习手册

从基本原理到算法设计与工程应用

包含十章正文、习题索引与公式术语附录

目录

点击目录条目可跳转；阅读顺序建议从算法基础开始。

1. 第 01 章　学习指南与算法基础 3
2. 第 02 章　线性表 8
3. 第 03 章　栈、队列与数组 21
4. 第 04 章　串与模式匹配 32
5. 第 05 章　树与二叉树基础 38
6. 第 06 章　查找树、哈夫曼与并查集 47
7. 第 07 章　图 56
8. 第 08 章　查找 72
9. 第 09 章　内部排序 82
10. 第 10 章　外部排序与综合应用 91
11. 习题索引 119
12. 公式与术语附录 133

第 01 章　学习指南与算法基础

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

学习路径与约定

建议先学习算法基础和线性表，再学习栈、队列、数组与串，随后进入树、图、查找、排序和综合应用。

符号、下标与复杂度约定

时间与空间复杂度的基本定义由算法基础与复杂度统一讲解。每个操作须明确下标、空值、输入规模与空间统计口径。

习题来源约定

区分已核实真题、平台题、自编题和来源待核实题；详见题目总索引。

算法基础与复杂度

> 学习目标：能区分逻辑结构、存储结构和抽象数据类型；能说明复杂度的统计口径，并比较解决同一问题的不同算法。本文中的运算计数默认单个固定宽度整数运算为 O(1)；使用任意精度大整数时需另计位运算成本。

从一个实际问题开始

某商店记录每天相对计划的利润变化，例如 `[-2，3，-1，4，-5]`。希望找到一段**连续、非空**的日期，使这段时间的利润变化之和最大。列举所有区间可以解决，但数据量扩大后，我们需要比较方法的增长速度；而记录如何存放、允许哪些操作，也影响算法选择。

数据结构：关系、表示与操作

- **数据元素**是考虑问题时作为一个整体处理的单位，如一条商品记录；**数据项**是该记录中进一步划分的字段，如商品编号。二者是组成关系，不应机械地写成集合包含关系。
- **数据对象**是一类性质相同的数据元素的集合。**数据结构**关注元素之间的关系、这种关系在计算机中的表示，以及可施加的操作。
- **逻辑结构**描述元素之间的关系：线性结构表示前后次序；树形结构表现层级；图描述一般的多对多关系；集合强调成员身份。它不决定元素在内存中的具体位置。
- **存储结构**是表示方式：顺序存储将元素按规则安排在连续空间；链式存储用链接表示关系；索引存储通过附加索引定位；散列存储根据键计算位置并处理冲突。这些类别不是互斥的完整实现清单，实际系统常组合使用。

例如，同一个有序序列既可用数组也可用链表表示。数组能按下标直接定位；链表要沿链接访问。它们的逻辑次序可以相同，操作成本却不同。索引查找并不天然是 O(1)：它取决于索引组织方式，树形索引和散列索引的成本就不同。

抽象数据类型（ADT）规定数据对象、关系和操作的语义，而不绑定具体存储。例如“列表”可以规定取第 i 个元素、插入和删除；数组与链表是不同实现。接口说明“做什么”，实现说明“怎样做”。

```
问题需求 → 逻辑关系与操作（ADT） → 选择存储方式 → 设计算法
                                                ↘ 评估时间和空间
```

算法与评价

算法是解决问题的一组明确步骤。传统教材常用五项特征描述确定性算法：**输入**可以有零个或多个，**输出**至少一个，步骤有确定含义，基本步骤可执行，且对规定的合法输入在有限步内结束。随机算法可以把随机选择及其分布明确写入规则，其执行路径或结果可以受随机数影响；不要把“步骤明确”等同于所有算法都不能使用随机数。

评价还要看正确性、可读性、健壮性、时间和空间消耗。正确性需要根据前提和不变量论证，测试能发现错误，但有限测试不能证明对所有输入正确。

时间与空间复杂度

设 n 为输入规模，T(n) 为某种固定操作模型下的操作次数。若存在常数 c、n_0，使所有 $n \ge n_0$ 都满足 $T(n) \le c \cdot f(n)$，则 T(n) 是 **O(f(n))**：这是渐近上界，不自动表示紧确数量级。Θ(f(n)) 同时给出渐近上界和下界；例如 $3n+2$ 是 Θ(n)，也属于 $O(n^2)$，但后者不够紧。

顺序执行的成本相加；嵌套循环要统计内层总执行次数，不是看到两层就必然是 n^2。二分规模的循环通常为 Θ(log n)。常见增长顺序：常数、对数、线性、线性对数、平方、指数。具体运行时间还取决于实现、硬件和数据。

口径	含义	示例
最好	同规模合法输入中成本最小	无序数组顺序查找，首元素命中只需 1 次比较
最坏	同规模合法输入中成本最大	查找失败需检查 n 个元素
平均	按明确的输入或访问概率求期望	每个位置等概率命中，比较次数为 (n+1)/2
均摊	对一串操作的总成本求每次平均上界，不依赖随机输入	动态数组偶尔扩容，但连续尾插的总复制次数为 O(n)

空间复杂度必须说清是否包含输入和输出。本文通常单独报告**辅助空间**：额外变量、辅助数组和递归调用栈。递归即使不显式分配数组，也可能使用与递归深度成正比的栈空间。原地算法常指辅助空间为 O(1)，但有些语境允许对数级栈，应以题目定义为准。

案例一：最大子列和

要求从非空数组中选择连续非空区间。对于示例 `[-2，3，-1，4，-5]`，最佳区间是 `[3，-1，4]`，和为 6。

方法	核心思路	时间	辅助空间
三重枚举	枚举左右端点，再逐项求和	$\Theta(n^3)$	O(1)
双重枚举	固定左端点，右端点右移时累加	$\Theta(n^2)$	O(1)
线性递推	保留“以当前位置结尾的最大和”	Θ(n)	O(1)

线性方法之所以可行：处理第 i 个数时，最优的以 i 结尾的区间，要么只包含 `A[i]`，要么接在以 $i-1$ 结尾的最优区间后。无需保存其他已经结束的区间。

```
最大子列和(A):                    // 前提：A 非空
    end ← A[0]                    // 以当前位置结尾的最大和
    best ← A[0]                   // 已扫描范围内的最大和
    对 i 从 1 到 n-1:
        end ← max(A[i], end + A[i])
        best ← max(best, end)
    返回 best
```

i	当前值	end：以此处结尾	best：迄今最优
0	-2	-2	-2
1	3	3	3
2	-1	2	3
3	4	6	6
4	-5	1	6

算法只返回最大和；若还需区间端点，须同步记录候选起点和最优起止位置，并规定同和时如何选择。全负数组如 `[-4,-2,-7]` 的结果是 -2，不能把初值设为 0；若允许空区间，则问题和答案约定不同。时间线性是因为每个位置只处理一次；除输入外仅保留常数个变量。

案例二：斐波那契与重复子问题

定义 F(0)=0、F(1)=1、F(n)=F(n−1)+F(n−2)，n 为非负整数。求 F(5) 时，朴素递归分别为 F(4) 和 F(3) 再次求解 F(3)，子问题会重复。记忆化保存已算结果；迭代则从 0、1 向上更新相邻两项。

```
求斐波那契(n):                 // n ≥ 0
    a ← 0; b ← 1
    重复 n 次:
        (a, b) ← (b, a + b)    // 同时更新
    返回 a
```

已完成轮数	a	b
0	0	1
1	1	1
2	1	2
3	2	3
4	3	5
5	5	8

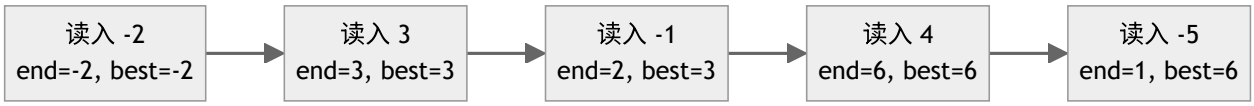
在固定宽度、每次加法按常数成本计且不考虑溢出的模型下：朴素递归时间 $\Theta(\varphi^n)$、栈空间 $O(n)$；记忆化时间 $O(n)$、额外空间 $O(n)$；迭代时间 $\Theta(n)$、辅助空间 $O(1)$。若精确表示任意大的 $F(n)$，整数的位数随 n 增长，加法和存储就不能视为常数；实际程序还需处理整数溢出。

工程应用与取舍

处理不断增长的日志记录时，动态数组适合顺序扫描和按下标读取；扩容时可能复制已有元素，单次追加存在延迟尖峰。若必须稳定控制单次延迟，可考虑分块存储，代价是定位和内存管理更复杂。分析时应同时说明吞吐、最坏延迟与内存，不能仅凭一个均摊 $O(1)$ 就判断系统一定满足实时要求。

图解：最大子列和的状态转移

以下图中的每个节点对应上方表格的一行；箭头表示读取下一个元素后的状态，不是不同候选区间的连接。`end` 必须以当前元素结尾，`best` 允许在更早位置结束。



工程案例：选择日报趋势分析算法

背景与输入：运营系统要在一年的 365 条每日利润变化中，找到连续非空区间的最大和；示例仍用 `[-2,3,-1,4,-5]`。数据已经按日期放在顺序表里；问题只需一次扫描，不需要频繁中间插入。

分析与选择：三重枚举直接重算每段，二重枚举复用当前区间和，二者适合小规模核对；线性递推只保留以当前日结尾的最优值和全局最优值，适合持续接收数据。顺序读取数组有较好的局部性，若数据以流形式到达，也可以不保存完整数组。

执行：从第一个值初始化，依次得到 `end=-2,3,2,6,1`，全局 `best=-2,3,3,6,6`。第 4 天处理后，候选区间为第 2 至第 4 天，和为 6；第 5 天虽使 end 降为 1，best 保持 6。最终结果是 6。若要返回日期区间，还要保存候选起点和最优端点。

成本与启示：对 n 天数据只扫描一次，时间 $\Theta(n)$，维护常数个状态，辅助空间 $O(1)$；若把数据输入数组也算在内，另需 $O(n)$ 输入空间。算法减少计算不意味着系统一定快：读库、网络、对象分配、缓存命中及日志也可能占主要时间。应在真实数据量和工作负载下测量吞吐、内存与高分位延迟，并分别分析扩容或垃圾回收带来的延迟尖峰；不要用大 O 推断具体毫秒数。

分级练习

本教材分级与平台难度不是同一标准。标注“自编”的题目不是 408 历年真题；平台题题面以链接为准。

基础

1. **自编**：学生记录中的学号是数据项还是数据元素？一条学生记录是什么？提示：按字段与整体的组成关系判断。
2. **自编**：代码顺序执行一个 n 次循环与一个 $2n$ 次循环，求时间和常数变量的辅助空间。提示：相加得 $\Theta(n)$，辅助空间 $O(1)$。
3. **自编**：无序表 n 个元素等概率成功命中，顺序扫描平均比较多少次？提示：$(n+1)/2$；写明成功和等概率条件。

进阶

1. **平台题**：LeetCode 509 斐波那契数。比较递归、记忆化和迭代；注意平台 n 的取值范围及正文的整数成本模型。

2. **平台题**：LeetCode 53 最大子数组和。要求连续非空；先用正文状态表手算，再分析 O(n) 时间、O(1) 辅助空间。
3. **自编**：对 `[-3,-1,-4]` 分别按“非空”和“允许空区间”计算最大和。提示：分别为 -1 和 0；两者是不同问题。

综合

1. **自编**：某日报系统输入一次 100 万条记录，也可能逐条接收数据。比较二重枚举与线性递推，讨论输入存储、内存、吞吐和高分位延迟；给出至少两种实测指标。提示：线性算法的辅助空间 O(1)，不等于整个系统总空间 O(1)。

第 02 章　线性表

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

顺序表

> 前置：算法基础与复杂度。本文约定数组下标从 0 开始，长度为 n，容量为 cap；辅助空间不计已有输入和输出。

为什么需要顺序表

学生成绩按报名顺序排列，常常需要快速读取第 i 条记录。线性表是有限个元素组成的序列：除首元素外每个元素有唯一直接前驱，除尾元素外每个元素有唯一直接后继；空表允许存在。逻辑上的先后由位置决定，不要求数值递增。

顺序表用连续存储单元保存元素。若每个槽宽度为 w，基地址为 base，则下标 i 的地址为 `base + i×w`。已知合法下标即可直接定位，时间 O(1)；这要求元素槽位宽度固定，且不代表任意抽象数据都必须物理等长。

```
下标       0       1       2       3       4

数组     [10]    [20]    [30]    [  ]    [  ]

           └──── 有效元素 ────┘    可用容量

长度 n = 3，容量 cap = 5
```

数组的连续空间与逻辑顺序一致。固定容量的顺序表满时要拒绝插入；动态顺序表可申请更大空间，但仍可能分配失败。动态分配不等于可无限增长。

查找、插入和删除

按下标查找合法范围为 `0 ≤ i < n`，直接读取，O(1)。按值查找通常从头扫描无序表，成功最多比较 n 次，失败需检查 n 个元素，最坏 O(n)；有序表若支持随机访问，可用折半查找，需维护有序性。

在位置 i **之前**插入 x，合法范围为 `0 ≤ i ≤ n`。先检查容量；从后往前搬移，避免覆盖尚未移动的数据。

```
插入(A, n, cap, i, x):
    若 i < 0 或 i > n: 返回“位置无效”
    若 n = cap: 返回“容量不足”        // 也可先调用扩容
    对 j 从 n-1 递减到 i:
        A[j+1] ← A[j]
    A[i] ← x
    返回 n+1
```

删除位置 i 的元素时，范围为 $0 \le i < n$；保存被删值后，将后缀从前往后搬移，长度减一。最后一个槽位是否清空是实现与资源管理问题，不改变逻辑长度。

```
删除(A, n, i):
    若 i < 0 或 i ≥ n: 返回“位置无效”
    x ← A[i]
    对 j 从 i 到 n-2:
        A[j] ← A[j+1]
    返回 (n-1, x)
```

空表不能删除，尾部插入和尾部删除不需要搬移已有元素；这些边界中循环执行零次。插入前应确认内存足够或扩容成功。

案例：插入、删除与移动次数

已有 [10,20,30,40]，在下标 1 插入 15：

```
初始        [10, 20, 30, 40,  _]
40 后移     [10, 20, 30, 40, 40]
30 后移     [10, 20, 30, 30, 40]
20 后移     [10, 20, 20, 30, 40]
写入 15     [10, 15, 20, 30, 40]
```

此处移动 3 个已有元素，即 n−i=4−1。若随后删除下标 2 的 20，30、40 向前各移动一次，得到 [10,15,30,40]。

对于长度 n 的表，插入位置 i 对应移动 n−i 个元素；若 n+1 个位置等概率，则平均移动次数为

(n + (n−1) + … + 0)/(n+1) = n/2。

删除位置 i 对应移动 n−i−1 个元素；若 n 个已有位置等概率，则平均为

((n−1) + … + 0)/n = (n−1)/2。

这只统计已有元素的**移动次数**，不包括定位、写入新元素、清理和扩容，也依赖等概率假设。

操作	时间	额外空间	条件
按下标读取	O(1)	O(1)	下标合法
无序表按值查找	最坏 O(n)	O(1)	每次比较视为 O(1)
插入或删除	最坏 O(n)	O(1)	不计扩容；尾部操作可 O(1)
一次扩容	最坏 O(n)	O(n) 新空间	新旧缓冲可能同时存在；具体分配器可能原地扩展

动态扩容与均摊

采用容量按固定倍数增长（例如满时翻倍）的策略，从空表连续尾插 n 次：扩容时复制的元素数量近似为 1+2+4+…，小于常数倍的 n。加上 n 次写入，总成本 O(n)，因此每次尾插**均摊 O(1)**；触发扩容的单次操作最坏仍为 O(n)。这不适用于任意位置插入，因为中间插入本身仍需搬移后缀；也不适用于每次只增加一个槽位的扩容策略。

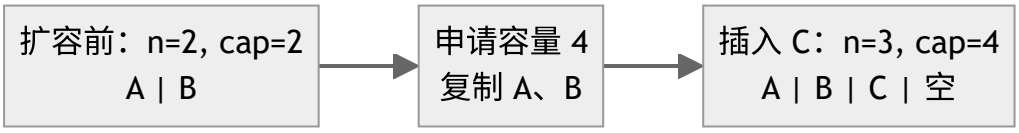
与链表如何选

需求	顺序表	单链表
按位置读取	O(1)	从头访问最坏 O(n)
已定位插入处	仍可能搬移 O(n)	已知前驱时改链接 O(1)
位置未定位	可能先搜索 O(n)	也需搜索 O(n)
内存	连续、可能预留容量	结点按需分配，有链接开销

工程中，动态数组适合读取频繁、顺序扫描较多的商品列表；局部移动虽然有渐进代价，却常有较好的缓存局部性。若经常持有要操作位置的结点引用、需要频繁局部插删，可考虑链表，但逐项查找、额外指针和内存分配可能抵消收益。选择应看真实访问模式及测量结果，不能只比较一个 O 记号。链表详见单链表。

图解：容量变化与扩容

下面展示一种翻倍策略：扩容前长度为 2、容量为 2，插入 C 时申请容量 4 的新数组并复制；实际实现也可能原地扩展，结果取决于分配器。



复制已有元素是 O(n) 的一次性成本。若新旧数组暂时同时存在，峰值内存可能超过最终容量；扩容失败时应保留原数据并报告失败。

工程案例：业务列表追加与中间插入

背景与输入：系统维护当天订单编号，初始有效数据 `[A,B]`、长度 2、容量 2；先在尾部追加 C，再将紧急订单 X 插到下标 1。操作顺序是题设，不假设特定编程语言容器的内部策略。

选择理由：页面经常按下标读取、从头遍历，动态数组能提供 O(1) 按位访问和连续扫描；代价是扩容及中间插入的移动。若频繁在已知结点前后插入且很少随机读取，可比较链表，但定位位置和指针开销不能忽略。

步骤	长度 / 容量	有效数据	发生的工作
初始	2 / 2	A, B	—
容量翻倍后追加 C	3 / 4	A, B, C	在本例复制 A、B，再写 C
下标 1 插入 X	4 / 4	A, X, B, C	先将 C 后移，再将 B 后移

结果与成本：订单次序是 `[A,X,B,C]`。本次追加触发扩容，最坏 O(n)；第二次中间插入最坏 O(n)。只有在固定倍数扩容且连续尾插的操作序列中，尾插才有均摊 O(1)；记录对象大小、并发修改、扩容失败及峰值内存都影响实际工程选择。

分级练习

自编题不是历年真题；平台题须以官方题设为准。

基础

1. **自编**：长度 6 的表在下标 2 插入、删除该位置原元素，分别移动几个已有元素？提示：4 个、3 个。
2. **自编**：容量 5、长度 0 的空表，插入位置 0 是否有效？删除位置 0 呢？提示：插入有效，删除无效。
3. **自编**：等概率删除长度为 7 的任意元素，平均移动次数是多少？提示：`(7-1)/2=3`，只统计已有元素移动。

进阶

1. **平台题**：LeetCode 27 移除元素。原地处理所有目标值；注意平台允许改变顺序。稳定覆盖法 O(n) 时间、O(1) 辅助空间。
2. **平台题**：LeetCode 88 合并两个有序数组。利用第一数组末尾的额外容量从后向前归并；独立题解。
3. **自编**：长度 4、容量 4，按翻倍扩容后在头部插入 9；分别统计本例复制和移动已有元素的次数。提示：若扩容分配新数组，复制 4 次、头插移动 4 次；原地扩容时复制次数可不同。

综合

1. **自编**：订单系统需要高频按序扫描、少量中间插入和偶发大批追加。比较动态数组与链表，并说明为何仅凭尾插均摊 O(1) 不能保证单次延迟。提示：考虑定位、缓存局部性、扩容和分配失败。

单链表

> 前置：顺序表。本文主体采用带头结点的单链表：`head` 是哨兵，不保存有效元素；`head.next` 指向首个数据结点，空值记为 `null`。位序从 1 开始。

为什么用链接组织元素

歌曲播放列表经常在已知位置插入一首歌。若元素按顺序存放在数组里，插入中间可能移动后缀；链表将每个数据元素放在结点里，通过 `next` 指向后继。结点不要求物理相邻，但次序只能沿链接确定。

```
head（头结点）       第 1 个          第 2 个

   [ next ] ─────→ [A | next] ──→ [B | null]

空表：head.next → null
```

头结点和头指针不同：头指针是保存起始位置的引用；头结点是可选的哨兵结点。不带头结点时，头指针直接指向首元结点，空表时为 null；在表头插入或删除后要把新头指针返回给调用方。带头结点时这些操作可统一为修改头结点的 next。

查找与建表

按位查找第 i 个元素（i ≥ 1）从 `head.next` 起走 i−1 条链接，若提前遇到 null 则位置无效；按值查找逐个比较。最坏都要检查 n 个结点，时间 O(n)，辅助空间 O(1)。若头结点另维护长度，取长度可为 O(1)，但每次插删都必须同步更新。

头插法依次读取元素，在头结点后插入：新结点先指向原首元结点，再让 `head.next` 指向新结点。输入 A、B、C 后得到 C→B→A，顺序反转。**尾插法**保留指向最后结点的 `tail`，每次将新结点接到其后并更新 tail；输入 A、B、C 得到 A→B→C。空表时 tail 初始指向 head。每次连接 O(1)，建 n 个结点 O(n) 时间、O(n) 结点空间；若每次从头找尾，建表为 $O(n^2)$。

已知前驱时插入与删除

下图在前驱 p 后插入 node。插入前 node 已分配，且不属于该链表：

```
初始：p ──→ q ──→ …        node

先接：p ──→ q ──→ …        node ──→ q

再改：p ──→ node ──→ q ──→ …
```

```
在 p 后插入(node):

    node.next ← p.next        // 必须先保存原后继关系

    p.next ← node
```

删除 p 的后继时，若 `p.next = null` 应返回失败；否则先保存待删结点和数据，再令 `p.next ← victim.next`，最后按运行环境释放或回收 victim。不能释放后再读取它的 next。已知 p 时插入和删除其后继为 O(1)；若只给位序或值，先找到 p 最坏要 O(n)，总时间仍为 O(n)。仅有待删结点引用且它是单链表尾结点时，通常无法在 O(1) 时间内定位前驱并删除该结点。

删除所有值为 x 的结点时，p 始终指向最后一个保留结点（初始为 head）：

```
删除所有(head, x):
    p ← head
    当 p.next ≠ null:
        若 p.next.data = x:
            victim ← p.next
            p.next ← victim.next
            释放 victim                  // 需手动管理内存时
        否则:
            p ← p.next
```

删除后 p 不前进，才能检查连续的 x。每个结点至多检查和删除一次，时间 O(n)、辅助空间 O(1)。空表、全部匹配、首尾结点匹配均适用。

案例：原地逆置单链表

问题与选型：把 A→B→C 改成 C→B→A。顺序表可逆序复制，但这里要求复用结点、辅助空间 O(1)。选择三个指针：prev 保存已逆置部分，p 指向未处理部分，next 暂存 p 的原后继，避免断链。

```
逆置首元结点(first):
    prev ← null
    p ← first
    当 p ≠ null:
        next ← p.next
        p.next ← prev
        prev ← p
        p ← next
    返回 prev

head.next ← 逆置首元结点(head.next)
```

时刻	prev 指向的已逆置部分	p 指向的未处理部分	暂存后继 next
初始	null	A→B→C→null	—
处理 A	A→null	B→C→null	B
处理 B	B→A→null	C→null	C
处理 C	C→B→A→null	null	null

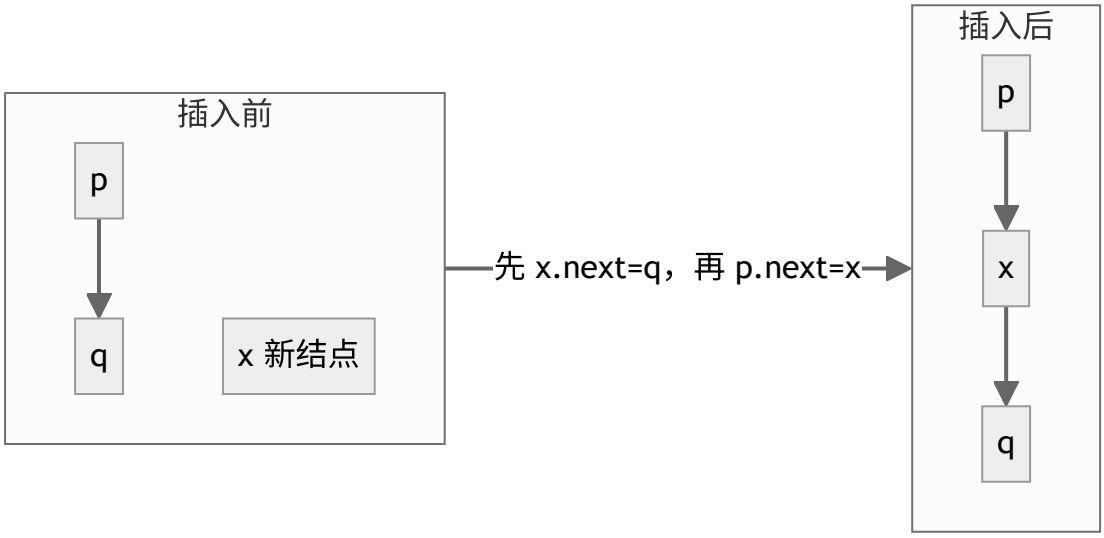
不变量：已逆置部分与未处理部分不重叠，合起来恰好包含原链表结点；每次处理一个结点，因此循环终止后 prev 即新首元结点。空表直接返回 null；单结点仍指向 null。n 个结点各处理一次，时间 O(n)、辅助空间 O(1)。这里的伪代码不适用于含环链表；若不保证无环，先检测或明确输入约束。

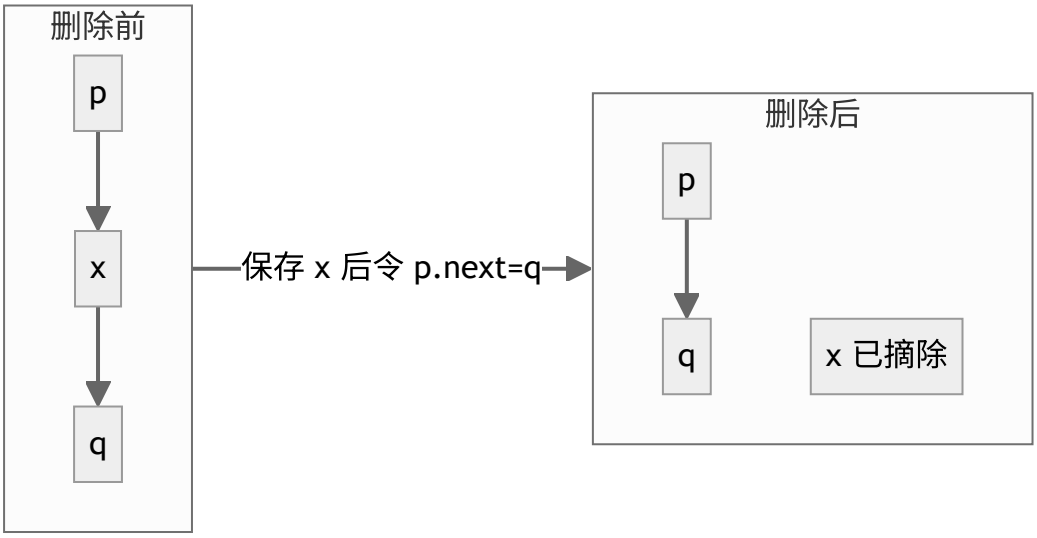
工程应用与边界

内存管理中的空闲块链、邻接表中的邻接结点，都可以用链接维护动态集合。已知前驱时插删只改少量链接，但查找位置仍要遍历；结点分散可能降低缓存局部性，额外链接和分配也有成本。程序中若外部仍持有被删除结点引用，还需定义引用失效与内存回收规则；并发操作需要额外同步，以上伪代码只描述单线程逻辑。

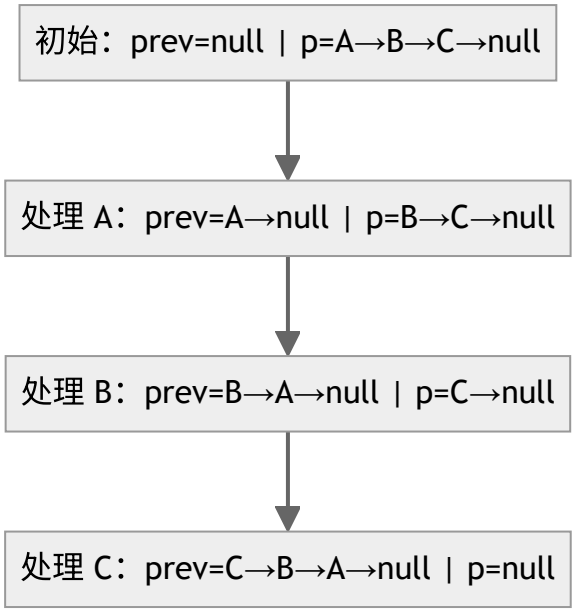
图解：插入、删除与逆置

每幅图中的箭头代表操作完成后的 `next`；`head` 是不存数据的哨兵，删除图假设 x 的后继为 q，且 x 不是 null。





逆置图每行分别是一个时刻，| 左侧为已逆置部分，右侧为未处理部分，左右两部分不存在连接：



工程案例：空闲块链的分配与回收

背景与输入：某教学版内存池使用单向空闲链记录三个同尺寸块，初始为 `head→A→B→C→null`；依次分配一块，再回收 A。这里是简化的定长块池，并非对某个生产分配器的实现描述。

分析与选择：分配只需要任意一个空闲块，不需要按位置随机访问。用头结点链接空闲块可在已知头结点时 O(1) 摘首和头插；若需要快速按地址合并相邻空闲块，还需额外的地址信息或其他索引，单链表本身不保证快速合并。

步骤	空闲链	解释
初始	head→A→B→C	三块均空闲
分配 A	head→B→C	保存 A 后令 head.next 指向 B
回收 A	head→A→B→C	A.next 先指向 B，再更新 head.next

结果与成本：空闲数从 3 变为 2 再恢复为 3；上述两次局部修改各 O(1) 时间和 O(1) 辅助空间。实际分配器还要保证块大小、对齐、所有权、重复释放检测及并发安全，不能把简单链表视作完整内存管理方案。

分级练习

自编题不是历年真题；n 为数据结点数。

基础

1. **自编**：对空表依次头插 1、2、3，删除首元后是什么？提示：`2→1`。
2. **自编**：带头结点的空表满足什么条件？不带头结点的空表呢？提示：分别为 `head.next=null` 和 `head=null`。
3. **自编**：已知前驱删除其后继时，若后继为 null，应该如何处理？提示：报告不存在，不解引用空值。

进阶

1. **平台题**：LeetCode 206 反转链表，题设为首元引用；独立题解使用三指针，时间 O(n)、辅助空间 O(1)。
2. **平台题**：LeetCode 141 环形链表。快慢指针相遇表示有环；注意输入可为空和单结点，时间 O(n)、辅助空间 O(1)。

3. **自编**：删除 1→2→2→3→2 中所有 2，并说明连续匹配时前驱为何不前移。提示：结果 1→3，时间 O(n)、辅助空间 O(1)。

综合

1. **平台题**：LeetCode 707 设计链表。按题设的 0 起始下标处理头尾、越界、空表；讨论只存头结点与额外存尾结点时的追加成本。不要把本页的 1 起始位序直接套入平台接口。

双向、循环与静态链表

> 前置：单链表。本文将 head 用作不保存有效数据的头结点，除特别说明外，空引用记为 null；操作均为单线程，复杂度不计结点的已有存储。

从需求出发

单链表向后走很方便，但要找到某个结点的前驱，通常必须从头扫描。如果系统需要在**已知结点**的位置经常向前、向后移动，可以给每个结点增加前驱链接。代价是更多链接、更新步骤和存储空间。

双向链表

双链表数据结点保存 prior、data、next，分别指向前驱和后继。非循环版本中，头结点的 prior 为 null，尾结点的 next 为 null；空表为 head.next = null。遍历可向前或向后，但从头按位查找仍为 O(n)。

```
null ← [head] ⇄ [A] ⇄ [B] → null

                prior / next 分别指向前驱 / 后继
```

在已知数据结点 p 和其非空后继 q 之间插入新结点 x：

```
x.prior ← p

x.next ← q

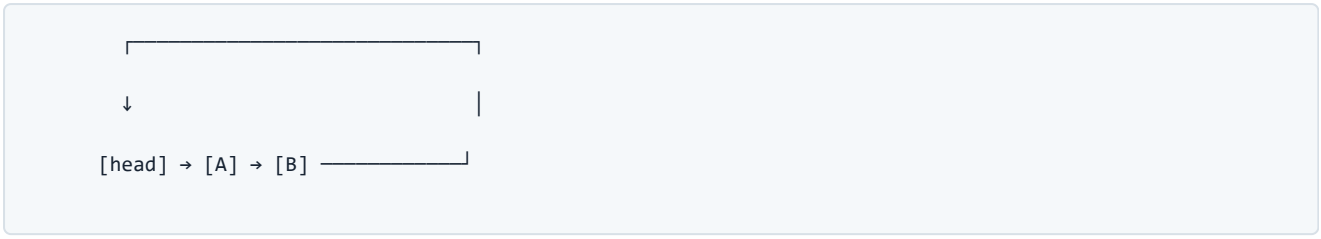
p.next ← x

q.prior ← x
```

若在非循环表尾插入，q 为 null，此时不能写 q.prior；若另维护尾指针，也要更新它。删除已知数据结点 x 时，先令前驱的 next 越过 x，再在后继非空时令其 prior 指回前驱；若 x 为尾结点，还需更新尾指针。带头结点时首元结点的前驱为 head，哨兵不能当普通数据删除。这些局部操作都在已知位置时为 O(1)；寻找位置仍可能 O(n)。

循环单链表

将尾结点的 next 指回头结点，形成环。以下采用**带头结点**约定：空表 head.next = head，非空时尾结点 tail.next = head。遍历从首元结点开始，到再次遇到 head 为止。



只保存 head 时，头插 O(1)，尾插需先找尾，最坏 O(n)；若额外保存 tail，头插与尾插均可 O(1)。带尾指针并非所有操作都变成 O(1)：单链表删尾仍需找到尾的前驱，最坏 O(n)。由于没有以 null 终止的尾链接，不应使用“走到 null”为停止条件；损坏的链接也可能导致遍历无法终止。

循环双链表：一次插入的完整过程

循环双链表同时维护两个方向。空表 `head.next = head` 且 `head.prior = head`；非空时 head 的 next 指向首元、prior 指向尾元。要在 p 后插入 x，先保存原后继 q：

```
q ← p.next

x.prior ← p

x.next ← q

p.next ← x

q.prior ← x
```

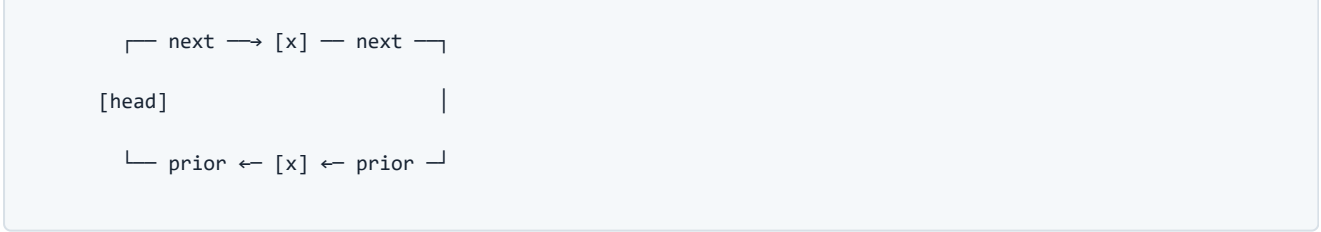
初始为空表时 p=q=head，操作依次得到：

```
初始：head.next = head, head.prior = head

保存：q = head

连接：x.prior = head, x.next = head

完成：head.next = x, head.prior = x
```



图中的两个 `[x]` 表示同一结点：head 与 x 的 next、prior 都指向对方。对于已有多个结点的情况，q 是 p 原来的后继，插入后必须同时满足 `p.next = x`、`x.prior = p`、`x.next = q`、`q.prior = x`。

删除已知数据结点 x 的局部操作为 `x.prior.next ← x.next` 和 `x.next.prior ← x.prior`；之后回收 x。只有一个数据结点时，两个赋值会使 head 的两个链接都指回自身。head 为哨兵，不能作为待删数据结点。借助 `head.prior` 可 O(1) 找到尾结点，因此头尾插入、删除在已知头结点和结点有效的条件下均为 O(1)。

静态链表：用数组下标当链接

有些环境希望使用预先分配的结点池。静态链表把结点放在数组槽位中，用整数游标代替指针；数组在物理上连续，但逻辑次序由游标决定。下例使用 -1 表示无后继，并将“数据链”和“空闲链”分开：

```
槽位    0        1        2        3

数据   [A| 2]  [空| 3]  [B|-1]  [空|-1]

数据链：head = 0   → 2 → -1        即 A → B

空闲链：free = 1   → 3 → -1
```

要在槽位 0 后插入 C：从空闲链取下 1，将 free 更新为 3；写入 `data[1]=C`；令 `next[1]=next[0]`，再令 `next[0]=1`。结果数据链为 `0→1→2→-1`，空闲链为 `3→-1`。删除槽位时先从数据链摘除，再把槽位接回空闲链；否则容量会逐渐泄漏。若 free 为 -1，固定容量池已满，插入须报告失败。数据链 head 和空闲链 free 是不同的游标；二者都为 -1 含义也不同。

操作或成本	循环单链表	循环双链表	静态链表
已知前驱后插入	O(1)	O(1)	O(1)，且需有空闲槽
从头按位查找	O(n)	O(n)	O(n)
已知数据结点删除	一般需找前驱，O(n)	O(1)	仅知游标一般需找前驱，O(n)
额外结构	一个链接/结点	两个链接/结点	数组、游标及空闲链管理

表中假设不额外维护前驱索引等辅助结构；静态链表也可以设计成可扩容的数组池，但本例固定容量。

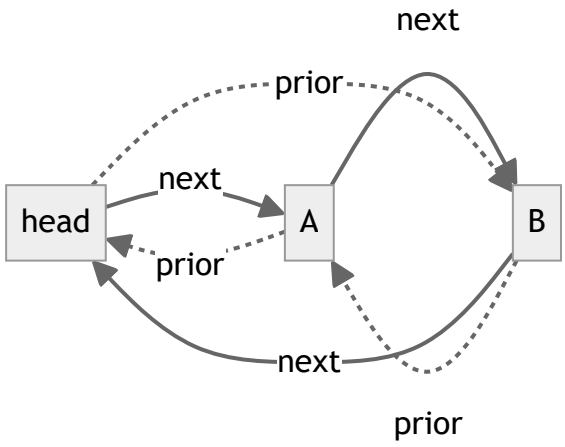
工程应用与取舍

循环双链表适合需要从已知结点频繁摘除、移动到另一端的场景。例如 LRU 缓存可用哈希表定位缓存项，再用双链表调整访问次序；只用单链表时，除非额外维护前驱信息，摘除已知结点可能仍需扫描。双链表需要更多链接，哈希表与链表更新还须保持一致；并发、容量和淘汰策略另需设计。

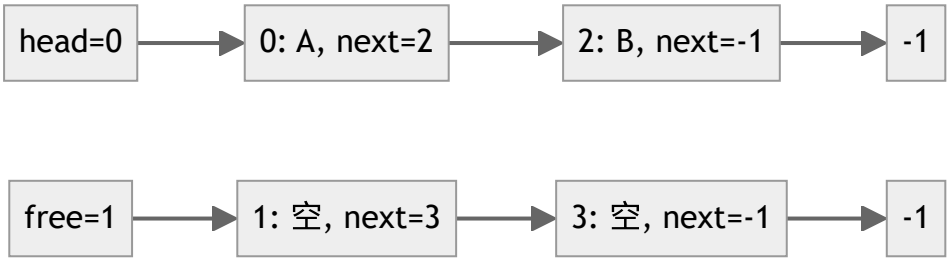
静态链表适合预先分配内存、希望限制分配次数或使用整数句柄的结点池；优点是容量可控，局限是必须维护空闲链、处理满池和避免使用已释放游标。循环链表可表达轮转次序，但不自动解决调度公平性或并发安全。

图解：双向关系与静态链表

双向箭头在下图中画成两条有方向的边：实线方向为 next，虚线方向为 prior。图表示带哨兵的**循环双链表**，不是非循环双链表。



静态链表使用上面的四个槽位；图中的边标注的是数组下标或 -1，不表示物理地址。方框 `free` 与 `head` 是独立游标。



工程案例：LRU 缓存的访问与淘汰

背景与输入：缓存最多保存两个键，操作依次为 `put(A)`、`put(B)`、`get(A)`、`put(C)`。左端表示最近使用，右端表示最久未使用；每个键对应的数据值省略。

分析与选择：需要按键找到缓存项，又需要把已找到的结点移到队首，并能快速淘汰队尾。哈希表提供平均 O(1) 键定位；双向链表提供已知结点的 O(1) 摘除与头插。仅用单链表时，找到结点后通常还要寻找前驱；仅用双链表查找键则可能 O(n)。

操作	最近 → 最久	哈希表包含的键	说明
初始	空	∅	—
put(A)	A	A	头插 A
put(B)	B, A	A, B	头插 B
get(A)	A, B	A, B	A 被访问，移到头部
put(C)	C, A	A, C	淘汰尾部 B，并删除其哈希项

结果与复杂度：缓存保留 A、C。按容量为 c、哈希操作平均 O(1) 的模型，每次 get/put 平均 O(1)，存储 O(c)；哈希退化时最坏不保证 O(1)。这是单线程模型；实际系统还需处理并发同步、异常时两种结构的一致性、内存占用与淘汰开销。适合近期访问具有参考价值的负载；扫描型访问可能污染 LRU，需根据工作负载测量并考虑替代策略。

分级练习

自编题不是历年真题；head 均为哨兵，除非题目另有说明。

基础

1. **自编**：循环单链表只有数据结点 A 时，head.next 与 A.next 分别是什么？提示：A 和 head。
2. **自编**：循环双链表只含 x，删除后 head 的两个链接指向哪里？提示：都指向 head。
3. **自编**：静态链表 `next[0]=2，next[2]=-1、head=0` 的逻辑槽位顺序是什么？提示：0、2；下标不要求连续。

进阶

1. **平台题**：LeetCode 707 设计链表。可以使用双链表；注意平台下标从 0 开始，边界条件以原题为准。
2. **自编**：循环单链表已知 tail，为何尾插 O(1) 但删除尾结点一般仍为 O(n)？提示：删除还要定位前驱。
3. **自编**：原静态链案例中插入 C 后删除槽位 1 并回收；写出两条链。提示：数据链 `0→2→-1`，空闲链 `1→3→-1`。

综合

1. **平台题**：LeetCode 146 LRU 缓存。用哈希表加双向链表；解释容量、更新已有键、淘汰尾部与两种结构一致性。平均操作 O(1)，空间 O(c)。

第 03 章　栈、队列与数组

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

数组与矩阵

> 前置：顺序表。本文数组下标从 0 开始；矩阵压缩公式中另外注明使用的下标。元素占 w 字节，地址计算不考虑越界和整数溢出。

从存储传感器数据开始

某设备每小时记录多个传感器读数。数据形成“小时 × 传感器”的矩阵：程序要能快速定位某一行某一列，也希望在大量读数为零时节省空间。选择时需要分清逻辑上的二维结构和底层的一维存储布局。

二维数组 A 有 r 行、c 列，0 基下标 $0\le i<r$，$0\le j<c$。元素槽宽相同，存放连续，首元素地址为 base：

- 行优先：$LOC(A[i,j]) = base + (i\times c+j)\times w$。
- 列优先：$LOC(A[i,j]) = base + (j\times r+i)\times w$。

例如 r=2、c=3 时，行优先依次存 A00,A01,A02,A10,A11,A12；列优先存 A00,A10,A01,A11,A02,A12。这两式以整个矩阵为一块连续存储为前提；由多个独立行对象组成的结构不一定满足。已知下标计算地址 O(1)，保存整个矩阵占 O(rc) 空间。

特殊矩阵：利用重复或固定值

以下矩阵为 n×n、n≥1，矩阵下标 i、j 从 1 开始，压缩数组下标 k 从 0 开始。压缩只改变表示，读取时必须根据规则还原逻辑值。

类型	性质和保存方式	映射
对称矩阵	A[i,j]=A[j,i]，按行保存下三角含主对角线，共 n(n+1)/2 项	当 i≥j：k=i(i-1)/2+j-1；否则交换 i、j 再用同式
下三角矩阵	主对角线上方都是同一常数 c；按行存下三角，再存一个 c	i≥j 用上式，否则读末尾常数单元
上三角矩阵	主对角线下方均为 c；按行保存上三角，再存一个 c	i≤j 时 k=(i-1)(2n-i+2)/2+(j-i)，否则读末尾常数
三对角矩阵	仅主对角线及相邻两条对角线可能非零，按行存有效带内位置	1 基 i、j 时 k=2i+j-3，仅在 abs(i-j)≤1 适用；其他位置读 0

三对角矩阵的有效槽位是 3n−2（n=1 时为 1）；带内值也可以是 0。上述直接映射访问为 O(1)，对称和三角压缩空间仍为 $\Theta(n^2)$，三对角为 Θ(n)。下三角公式来自前 i−1 行分别保存 1、2、…、i−1 项，再加本行从 0 开始的偏移 j−1。

案例：对称距离表

背景与输入：三个机房之间的距离满足双向相同，矩阵如下；主对角线为 0：

```
      1   2   3
  1 [ 0   4   7 ]
  2 [ 4   0   5 ]
  3 [ 7   5   0 ]
```

选择理由：距离无方向，`A[i,j]=A[j,i]`。保存下三角比存两个方向的重复值省空间，并可以按公式随机读取；如果关系变成有向距离，这一假设不再成立。

步骤	压缩数组（下标从 0）	说明
存第 1 行	`[0]`	A11
存第 2 行	`[0,4,0]`	A21、A22
存第 3 行	`[0,4,0,7,5,0]`	A31、A32、A33

查询 A[2,3] 时先交换得到 A[3,2]；`k=3×2/2+2−1=4`，数组下标 4 的值为 5。构建时写 6 项，$\Theta(n^2)$；单次取值 O(1)，保存 n(n+1)/2 项。压缩矩阵不能按普通完整二维数组的布局计算地址。

```
读对称(A压缩, i, j):          // 1 ≤ i,j ≤ n
    若 i < j: 交换 i, j
    k ← i×(i-1)/2 + j-1
    返回 A压缩[k]
```

案例：三对角矩阵的压缩与还原

某线性方程组的系数矩阵只有主对角线和相邻两条对角线可能非零。给定 4×4 矩阵，矩阵行列下标从 **1** 开始：

```
       j=1  2  3  4
i=1     1  2  0  0
i=2     3  4  5  0
i=3     0  6  7  8
i=4     0  0  9 10
```

按行优先只存储满足 `|i-j|≤1` 的位置。带内位置即使值为 0 也要占用槽位，本例的带内元素恰好都非零。

矩阵行	按列递增的带内位置	保存的值	压缩数组下标
1	(1,1)、(1,2)	1、2	0、1

矩阵行	按列递增的带内位置	保存的值	压缩数组下标
2	(2,1)、(2,2)、(2,3)	3、4、5	2、3、4
3	(3,2)、(3,3)、(3,4)	6、7、8	5、6、7
4	(4,3)、(4,4)	9、10	8、9

压缩数组下标从 **0** 开始，因此得到 `B=[1,2,3,4,5,6,7,8,9,10]`。查询 A[3,4] 时满足 `|3-4|≤1`，代入 `k=2i+j-3` 得 `k=2×3+4-3=7`；读取 `B[7]=8`，与原矩阵一致。反向按给定位置还原时，先检查矩阵下标是否合法：如果在带内，按公式读取 B；如果在带外，按三对角矩阵的定义返回 0。例如 A[1,4] 不在带内，返回 0，不应对它使用带内映射。

4×4 稠密表示需要 16 个元素槽，压缩后需要 `3n-2=10` 个槽；一般 n×n 矩阵分别使用 Θ(n²) 和 Θ(n) 个元素槽。这里比较的是元素槽位，不包含记录矩阵维度等元数据，也不意味着所有带内元素必须非零。

稀疏矩阵

当非零项 t 远少于 rc，可保存 (行,列,值) 三元组；行列下标与排序规则必须声明。若以下标从 0 开始、按行列升序：`[(0,2,5),(2,0,7)]` 表示 3×3 矩阵仅 A[0,2]=5、A[2,0]=7。需要另记行数、列数和 t；存储 O(t) 条记录加 O(1) 元数据，顺序查找某位置最坏 O(t)，增加索引可改变查找成本。

十字链表中每个非零结点分别沿行链和列链链接，可按行或列访问，但需要额外指针和头部管理；若只做顺序遍历，三元组更简单。稀疏表示是否节省实际字节取决于数值、下标、指针和分配开销，不能只看非零比例。

工程应用与练习

科学计算和图数据可能存在稀疏矩阵，压缩能减少存储和对零项的遍历，但随机更新、索引结构和局部性都有成本；选择表示时先确认矩阵规模、零的比例和主要操作。

基础

1. **自编**：2×3 行优先数组，A[1,2] 的元素偏移是几？答：5。
2. **自编**：4×4 对称矩阵下三角含对角线共几项？答：10。
3. **自编**：n=3 的三对角矩阵最多保存几个带内位置？答：7。

进阶

1. **自编**：按上述对称例计算 A[1,3] 的压缩下标和值。答：交换为 (3,1)，k=3，值 7。
2. **平台题**：LeetCode 867 转置矩阵。m×n 的结果是 n×m；复制所有元素时间 O(mn)，输出占 O(mn)，不计输出的辅助空间 O(1)。
3. **自编**：两条三元组 `(0,2,5),(2,0,7)` 的 3×3 矩阵，A[1,1] 是多少？答：0。

综合

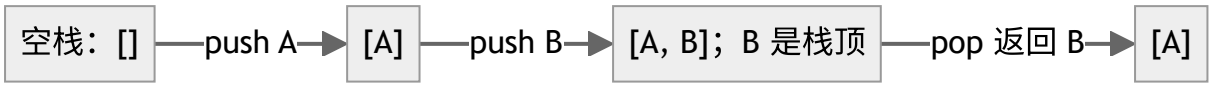
1. **自编**：给定 10,000×10,000 矩阵、仅 1,000 个非零项，分别估算稠密元素槽和三元组数，讨论哪种操作适合何种结构。提示：前者 10^8 槽，后者 1,000 条加元数据；不要忽略下标开销和随机查找。

栈与递归

> 前置：顺序表、单链表。图中栈底在左、栈顶在右；空栈的出栈和取顶均须报告失败。

栈：后进先出

浏览器记录的简化“后退”操作需要先取出最近一次访问的位置。栈是仅允许在同一端插入和删除的线性结构，遵循后进先出（LIFO）。ADT 包含 `push(x)`、`pop()`、`top()` 和 `empty()`；ADT 规定行为，不要求使用某一种存储方式。



顺序栈用容量 M 的数组，约定 top 指向栈顶元素：空时 top=-1，满时 top=M−1；先判满，再 `top←top+1`；`A[top]←x`。出栈先判空，返回 A[top] 后令 top 减一。两者均 O(1)，固定数组占 O(M)。链栈把栈顶设为单链表首元，入栈相当于头插、出栈相当于头删，O(1) 时间、O(n) 结点存储；结点分配仍可能失败。

共享栈：两栈共用 M 个槽，top0 从 -1 向右，top1 从 M 向左，二者都指向各自栈顶元素；空分别为 top0=-1、top1=M，满为 `top0+1=top1`。例如 top0=2、top1=7，空闲下标是 3、4、5、6，共 4 格，而不是 1 格。两栈共享的是总容量，不保证某一栈始终拥有固定配额。

固定输入顺序的 n 个互不相同元素允许入栈、出栈交错时，合法出栈序列数是卡特兰数 $C_n=\text{binom}(2n,n)/(n+1)$；该计数假设栈容量足够且每个元素恰好入栈、出栈一次。例如 n=3 时有 5 种。

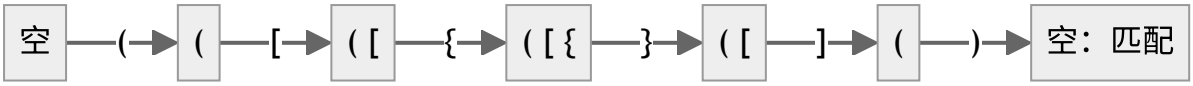
案例：括号匹配

背景：配置文件需要检查 `([{}])` 的括号是否按类型和嵌套次序闭合。最近遇到的尚未闭合左括号必须最先匹配，所以选栈而不是普通先进先出队列。

```
括号匹配(s):                  // 只接受 ()、[]、{} 六种字符
    创建空栈 S
    对 s 中每个字符 c:
        若 c 是左括号: S.push(c)
        否则若 S 为空或 S.top() 与 c 类型不匹配: 返回 false
        否则: S.pop()
    返回 S.empty()
```

栈底在左：

字符	动作	栈
(	入栈	(
[	入栈	([
{	入栈	([{
}	与 { 匹配并出栈	([
]	与 [匹配并出栈	(
)	与 (匹配并出栈	空



结果为 true。空串按本算法为 true；`([)]` 因栈顶类型不匹配而失败，右括号多于左括号时不能对空栈取顶。每个字符最多进出栈一次，时间 O(n)，最坏辅助空间 O(n)。若要检查实际代码，还需考虑字符串字面量、注释等语法，不能直接把这个简化算法当完整编译器。

案例：后缀表达式求值

`(2+3)×4` 的后缀形式为 `2 3 + 4 *`。遇数字入栈，遇二元运算符先弹出**右操作数**，再弹出左操作数，计算并压回。

输入	操作	栈（底→顶）
2	入栈	2
3	入栈	2,3
+	2+3	5
4	入栈	5,4
*	5×4	20

结果 20。token 数为 n 时，栈操作总数 O(n)，辅助空间最坏 O(n)；假定算术操作 O(1)。若输入不受信任，还需检查操作数数量、非法 token、除零和结束时栈中是否恰有一个值；整数除法的舍入和溢出按题设确定。

案例：中缀表达式转后缀表达式

中缀表达式 `(2+3)*4-6/2` 中，括号、优先级和结合性决定先算什么。转换时使用**运算符栈**暂存还不能输出的运算符，操作数直接输出。以下约定 `*`、`/` 的优先级高于 `+`、`-`，四者均为左结合；输出中的空格用来区分操作数和运算符。

处理规则如下：

1. 读到操作数，直接追加到后缀输出；多位数先作为一个完整 token 读取。
2. 读到左括号 `(`，直接入栈。它是边界，比较优先级时不能越过它。
3. 读到右括号 `)`，反复弹出并输出栈顶运算符，直到遇到 `(`；再弹出这个 `(`，括号本身不输出。若找不到匹配左括号，表达式无效。
4. 读到当前运算符时，反复查看栈顶：只要栈顶不是 `(`，且**栈顶优先级高于当前运算符**，就弹出并输出；若二者优先级相等、**当前运算符是左结合**，也弹出并输出。直到条件不再成立，再将当前运算符入栈。
5. 对右结合运算符，栈顶同级运算符**不因同级而弹出**；栈顶优先级更高时仍须弹出。例如若另行定义 `^` 为右结合的幂运算，`2^3^4` 转为 `2 3 4 ^ ^`，表示 `2^(3^4)`。具体优先级和结合性由所处理语言的规则决定。
6. 输入结束后依次弹出并输出剩余运算符；若栈中仍有括号，表达式无效。

下表从左到右扫描，运算符栈的**栈底在左、栈顶在右**；每行显示本次操作完成后的状态。

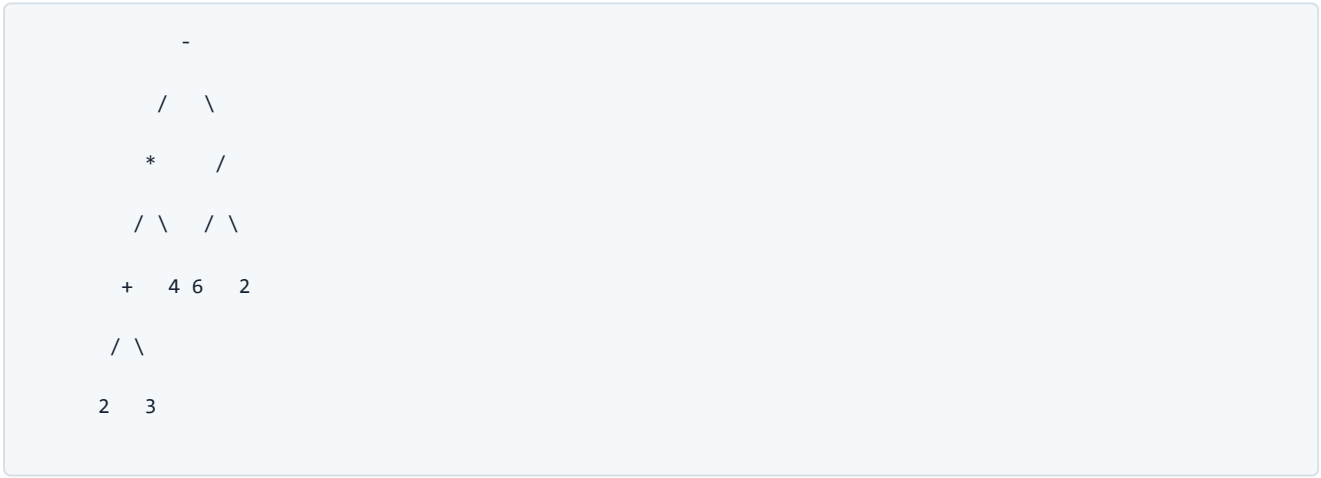
当前字符	运算符栈	后缀输出	本次操作
(	(	空	左括号入栈
2	(	2	输出操作数
+	(+	2	栈顶为左括号，+ 入栈

当前字符	运算符栈	后缀输出	本次操作
3	(+	2 3	输出操作数
)	空	2 3 +	输出 +，弹出但不输出 (
*	*	2 3 +	栈空，* 入栈
4	*	2 3 + 4	输出操作数
-	-	2 3 + 4 *	栈顶 * 优先级更高，先输出 *，再压入 -
6	-	2 3 + 4 * 6	输出操作数
/	- /	2 3 + 4 * 6	栈顶 - 优先级较低，/ 入栈
2	- /	2 3 + 4 * 6 2	输出操作数
输入结束	空	2 3 + 4 * 6 2 / -	依次输出 /、-

最终后缀表达式为 `2 3 + 4 * 6 2 / -`，按上节方法求值：`(2+3)×4−6/2=17`。例如 `8-3-2` 中两个减号同级且左结合，第二个 - 入栈前应先输出第一个，得到 `8 3 - 2 -`，结果为 3。

对已正确分词且语法合法的输入，每个 token 最多入栈和出栈一次，因此转换时间 O(n)，运算符栈最坏 O(n)；保存输出还需要 O(n) 空间。实际解析器还应检查缺少操作数、未知运算符和括号不匹配；一元负号、函数和其他运算符需要额外规则，不能直接套用本例的二元运算符处理方式。

前缀表达式把运算符放在操作数之前；后缀表达式把运算符放在对应的操作数之后。对完整中缀 `(2+3)*4-6/2`，表达式树为：

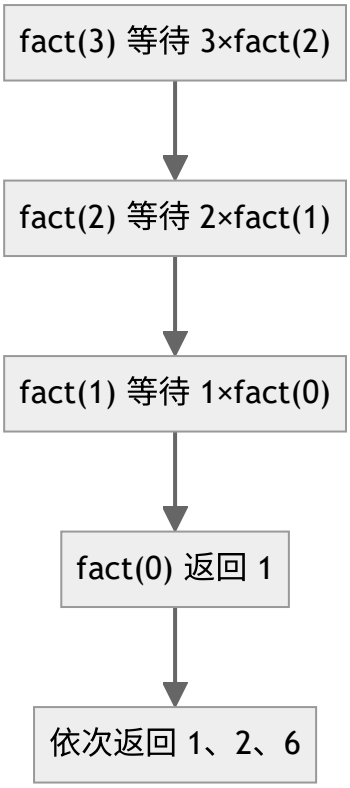


根结点 - 的左操作数是 `(2+3)*4`，右操作数是 `6/2`；+ 的左右操作数分别是 2、3，/ 的左右操作数分别是 6、2。按根→左→右读取得到前缀 `- * + 2 3 4 / 6 2`；按左→右→根读取得到后缀 `2 3 + 4 * 6 2 / -`，与上表一致。树的左右子树不能交换：先计算 `2+3=5`、`5*4=20` 和 `6/2=3`，最后为 `20-3=17`，前缀与后缀均表示同一个运算顺序。单独的 `* + 2 3 4` 只表示左子表达式 `(2+3)*4`，值为 20。

由中缀转前缀也可先构造表达式树再前序遍历；普通中序序列若省略必要括号，可能无法唯一表达运算次序。

递归与显式栈：同一任务

求 3!，递归式 `fact(0)=1`、`fact(n)=n×fact(n−1)`（n 为非负整数）。调用栈逐层保存尚未完成的乘法：



用显式栈记录等待乘的数，模拟相同后进先出次序：

```
阶乘_显式栈(n):                // 前提 n ≥ 0
    S ← 空栈
    当 n > 0:
        S.push(n); n ← n-1
    result ← 1
    当 S 非空: result ← result × S.pop()
    返回 result
```

n=3 时依次入栈 3、2、1，出栈 1、2、3，结果 1→1→2→6；n=0 时不入栈，返回 1。两种形式均做 O(n) 次固定成本乘法，栈占 O(n)；直接迭代累乘可降至 O(1) 辅助空间。精确大整数的乘法和存储成本需另计。

工程应用

运行时常用调用栈或等效机制维护返回位置、局部状态等；具体布局、尾调用优化及栈大小依语言与运行时而异。递归深度过大可能耗尽资源，显式栈能让程序控制存储方式，但并不自动降低最坏空间。编译器或表达式解释器可用栈保存嵌套状态与操作数，真实语言还需词法、语法、异常和类型处理。浏览器“后退/前进”可用两个栈建模；访问新页面通常清除前进记录，实际浏览器还涉及标签页和持久化。

分级练习

自编题不是历年真题；平台题按其原题约束处理。

基础

1. **自编**：依次 push A、B，pop 后栈顶是什么？答：A。
2. **自编**：容量 8 的顺序栈、top 指向栈顶且空时 -1；何时满？答：top=7。
3. **自编**：`([)]` 是否匹配？答：否，遇) 时栈顶为 [。

进阶

1. **平台题**：LeetCode 20 有效的括号，注意平台输入长度及字符范围。
2. **平台题**：LeetCode 150 逆波兰表达式求值，题目规定整数除法向零截断。
3. **自编**：后缀 `8 3 - 2 *` 求值。答：(8−3)×2=10；时间 O(n)、辅助空间 O(n)。

综合

1. **自编**：用两个栈建模浏览器后退/前进，说明新页面访问后为什么清空前进栈，并手动追踪 A→B→C、后退一次、访问 D。答：当前 D，后退历史 A、B，前进历史空；模型需为每个标签页维护独立状态。

队列与双端队列

> 前置：顺序表、单链表、栈。本文环形数组下标从 0 开始，M≥2；`front` 指向队头元素，`rear` 指向队尾元素的下一位置。

队列与假溢出

任务依次到达后台，单个工作线程先处理最早到达且尚未处理的任务。队列是先进先出（FIFO）的抽象：在队尾入队，在队头出队。空队列取头或出队必须返回失败。

非循环顺序队列用数组保存数据，front、rear 只向右移动。即使前面已有出队后空出的槽位，rear 到数组末尾也不能继续追加，这叫**假溢出**。循环队列让下标对 M 取模，将空出的槽位重新利用；这不消除真正的容量限制。

循环队列：三种判空、判满约定

方案	空	满	元素个数与容量
牺牲一个槽位	front=rear	(rear+1) mod M=front	`(rear-front+M) mod M`；容量 M−1
计数 size	size=0	size=M	size；容量 M
标志 tag	front=rear 且 tag=0	front=rear 且 tag=1	相等时由 tag 区分满/空；否则用模公式，容量 M

标志法初始化为 tag=0；**成功**入队后设 1，成功出队后设 0；失败操作不更改状态。计数和标志法在满时 `front=rear`，直接使用模公式会得到 0，因此必须单独处理。不能混合不同约定的公式。

以下伪代码使用牺牲一个槽位的方案：

```
入队(x):
    若 (rear+1) mod M = front: 返回“满”
    Q[rear] ← x
    rear ← (rear+1) mod M
    返回“成功”

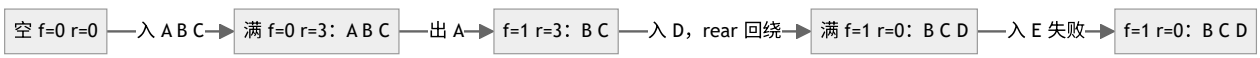
出队():
    若 front = rear: 返回“空”
    x ← Q[front]
    front ← (front+1) mod M
    返回 x
```

操作都为 O(1) 时间；数组占 O(M)，单次操作辅助空间 O(1)。此伪代码仅适用于单线程或已有同步的环境。

案例：循环任务缓冲

背景与输入：单个消费者按到达顺序处理任务，使用 M=4、牺牲一槽的队列，最多保留 3 个任务。依次入队 A、B、C，出队一次，再入队 D；尝试再入队 E。

操作	front	rear	队列逻辑次序	说明
初始	0	0	空	—
入 A	0	1	A	Q[0]=A
入 B	0	2	A,B	Q[1]=B
入 C	0	3	A,B,C	满，因为 (3+1) mod 4=0
出队	1	3	B,C	返回 A
入 D	1	0	B,C,D	Q[3]=D，rear 回绕到 0
尝试入 E	1	0	B,C,D	满，拒绝；状态不变



这个模型确保队列内任务的出队次序为 B、C、D；如果多个消费者并发执行，**完成顺序**不一定与出队顺序一致。选择环形数组是因为容量固定、空间可复用且操作简洁；代价是必须有满时策略和同步机制。

链队列与双端队列

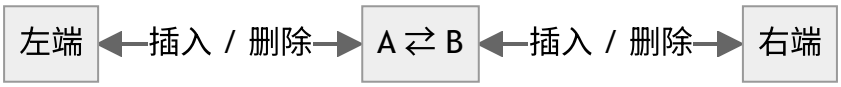
带头结点的链队列，front 指向哨兵、rear 指向尾结点；空时 front=rear。入队将新结点接到 rear 后并更新 rear；出队删除哨兵后第一个结点。删除最后一个数据结点时必须将 rear 重新指回哨兵。已知两个指针时操作 O(1)，n 个数据结点占 O(n)；内存不足仍会失败。

双端队列允许两端插入、删除。受限版本可只允许一端插入或一端删除；栈和普通队列可看作某些操作受限的用法。

下面为便于比较，约定输入受限队列只能在**右端插入**，输出受限队列只能在**左端删除**；具体题目也可以选择相反端点。

类型	左端插入	右端插入	左端删除	右端删除
普通双端队列	允许	允许	允许	允许
输入受限双端队列	不允许	允许	允许	允许
输出受限双端队列	允许	允许	允许	不允许

“受限”限制的是允许执行的操作，不改变当前已有元素的相对次序。初始化为空时，任何一端的删除都必须报告失败。



注意：四个元素依次输入时无约束双端队列恰能得到全部 24 种排列，但不能推广为任意 n 都能得到 n! 种。比如输入 1–5 允许两端入出且可交错时，输出 `5,1,3,2,4` 不可达；这是一条反例，不是判断任意序列的完整算法。

案例：滑动窗口最大值

问题：传感器依次产生数组 `A=[1,3,-1,-3,5,3,6,7]`，窗口大小 k=3。每次窗口向右移动一格，求当前窗口内的最大值；数组下标从 0 开始。

可以用一个**普通双端队列**存放候选元素的下标，并通过额外规则保持这些下标对应的值从队头到队尾严格递减。这种用双端队列维护单调性的做法称为**单调队列技巧**；普通双端队列本身不要求元素有序。

处理下标 i 时先从队头移除已经离开窗口的下标（≤ i-k）；再从队尾移除值小于或等于 `A[i]` 的下标，最后将 i 加到队尾。因为新元素更晚离开窗口，而被移除的旧元素值不大于新元素，只要旧元素仍在窗口内，新元素也在且不比它小，旧元素就不可能是此后窗口中**必须保留的最大值候选**。值相等时保留较新的下标，同样能得到正确最大值；若需要保留最早最大值的位置，应另定相等值处理规则。

```
滑动窗口最大值(A, k):              // 前提 1 ≤ k ≤ |A|
    D ← 空双端队列                   // 保存下标，队头在左
    result ← 空列表
    对 i 从 0 到 |A|-1:
        当 D 非空 且 D.front ≤ i-k: D.pop_front()
        当 D 非空 且 A[D.back] ≤ A[i]: D.pop_back()
        D.push_back(i)
        若 i ≥ k-1: result.append(A[D.front])
    返回 result
```

i	新值	当前窗口	操作完成后的队列下标（头→尾）	当前最大值
0	1	窗口尚未填满	`[0]`	—
1	3	窗口尚未填满	`[1]`	—
2	-1	`[1,3,-1]`	`[1,2]`	3
3	-3	`[3,-1,-3]`	`[1,2,3]`	3
4	5	`[-1,-3,5]`	`[4]`	5
5	3	`[-3,5,3]`	`[4,5]`	5
6	6	`[5,3,6]`	`[6]`	6
7	7	`[3,6,7]`	`[7]`	7

例如 i=4 时，下标 1 已离开窗口，先从队头移除；然后队尾下标 3、2 对应值分别为 -3、-1，都不大于新值 5，依次从队尾移除，最后加入下标 4。结果为 `[3,3,5,5,6,7]`。

每个下标恰好入队一次，至多从某一端出队一次，尽管内层有循环，总时间仍为 O(n)。队列中至多保存 k 个窗口内下标，辅助空间 O(k)；返回的 n−k+1 个结果另占 O(n−k+1) 输出空间。若 k≤0 或 k>n，本案例的前提不成立，调用方应报告参数无效。

工程应用：消息与背压

生产者向缓冲区提交任务，消费者取出处理；缓冲可以吸收短时速率波动。若生产持续快于消费，有限队列最终会满，系统必须选择阻塞生产者、拒绝、丢弃或扩容等策略；**背压**是让上游感知并降低发送速率的一类机制。环形数组只描述内存缓冲，不自动提供线程安全、持久化、确认、重试、去重或跨机器顺序。具体消息系统的保证须以其协议为准。

树的层次遍历与图的 BFS 使用队列保存待访问节点，分别见树的遍历与图的遍历。双端队列也可支持工作调度或滑动窗口，但是否线程安全取决于实现。

分级练习

基础

1. **自编**：M=6、牺牲一槽，实际容量是多少？答：5。
2. **自编**：上述方案 front=4、rear=1，元素有多少？答：(1−4+6) mod 6=3。
3. **自编**：带头结点链队列删除最后一个元素后 rear 应指向哪里？答：头结点。

进阶

1. **平台题**：LeetCode 622 设计循环队列。题目的 k 是可用容量；若牺牲一槽，应分配 k+1 个槽或改用计数法。
2. **平台题**：LeetCode 232 用栈实现队列。当出栈为空时转移入栈元素；单次可能 O(n)，一系列操作每个元素至多转移一次，均摊 O(1)。
3. **自编**：计数法 M=4，front=rear 且 size=4 时队列空还是满？答：满；模公式单独会误得 0。

综合

1. **自编**：生产者每秒稳定提交 10 个任务，消费者每秒完成 8 个；有限队列长期会怎样？设计满时策略并说明是否会丢任务。提示：不考虑其他因素时积压平均每秒增加 2，有限容量终会用尽；阻塞、拒绝和丢弃有不同语义。

第 04 章　串与模式匹配

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

串的表示

> 前置：顺序表、单链表。本文除特别说明外以“字符序列”讨论，并用 0 基、左闭右开区间 `[1,r)` 表示子串。

从搜索文本开始

日志分析器需要从一段文本中找出 `ERROR` 是否出现，并在展示时截取上下文。文本中的字符有顺序，搜索内容也由连续字符构成；这正是串和模式匹配要解决的问题。

串是零个或多个字符组成的有限序列。长度为 0 的空串与包含一个空格的串不同。子串是连续片段：`abcde[1,4)` 为 `bcd`；不连续选取 `ace` 是子序列，不是子串。两个串相等要求长度相同、对应字符均相等；字典序按首个不同字符比较，若一方是另一方前缀，则较短者通常排前，具体比较规则须明确。

串 ADT 描述赋值、求长、比较、连接、子串、定位、插入、删除和替换的语义。`substring(S,l,r)` 在 `0≤l≤r≤n` 时返回 `S[l,r)`；`l=r` 返回空串，非法边界报告失败。`index(S,P)` 的结果表示 P 首次出现的位置，不存在时返回 -1；本教材约定空模式首次出现位置为 0。不同 API 或题目可有不同约定。

顺序、动态与块链存储

方式	表示	优势与局限
定长顺序串	固定容量连续槽位和长度	可 O(1) 按槽访问；容量不足时按接口约定拒绝或截断，不能默认静默丢失
动态顺序串	运行时分配的连续缓冲，记录长度及可能的容量	可扩容；扩容和插入可能复制内容，是否可变取决于具体类型
块链串	每个结点存一块字符、长度和下一链接	块内连续、块间相连；定位需走过前面的块，结点和尾块有额外开销

顺序串 n、m 表示输入长度；若结果需新分配并复制，连接时间和结果空间均 O(n+m)，复制长度为 k 的子串为 O(k) 时间和 O(k) 输出空间；若采用共享视图，复制和生命周期成本会不同。串比较最多比较 O(min(n,m)) 个字符（假定单字符比较为 O(1)）。固定容量原地删除需移动后缀，最坏 O(n)，不额外分配缓冲时辅助空间 O(1)。

块链的**数据槽位利用率**与整个结点的**存储密度**不是同一口径。若 n 个字符、每块 B 个槽、共 k=⌈n/B⌉ 块，则槽位利用率为 n/(kB)（n>0）；若每槽占 w 字节、每个结点含 h 字节元数据和指针，忽略填充时整体密度为 nw/[k(Bw+h)]。实际结构还可能有对齐填充；不能忽略尾块空槽而断言结点越大密度一定单调增大。

“字符”在工程中可能指字节、Unicode 码点或用户可见字符簇；UTF-8 中一个汉字常占多个字节。按字节索引、按码点索引和显示宽度并不等价。文中 BF/KMP 示例使用 ASCII，避免在算法状态表中混淆单位；具体编码与规范化按系统要求处理。

案例：提取日志中的状态字段

背景与输入：日志内容是 `"ID=7;ERROR;END"`；已由其他步骤确定 `ERROR` 的起点 l=5 和终点 r=10（0 基、右端不包含）。要求取出这一字段。

步骤	状态	说明
校验	0≤5≤10≤14	字符长度为 14
截取	`S[5,10)`	依次取下标 5、6、7、8、9
结果	`ERROR`	长度 10−5=5

```
取子串(S, l, r):              // 返回新串，长度单位为字符槽
    若 l < 0 或 r < l 或 r > S.length: 返回“边界错误”
    新建长度 r-l 的结果 T
    对 i 从 l 到 r-1: T[i-l] ← S[i]
    返回 T
```

题设采用新串复制，时间 O(r−l)、输出空间 O(r−l)、除输出外辅助空间 O(1)。若取 `[10,10)` 得到空串；若 r>n 应拒绝而不是越界。真实日志常采用分隔符解析或索引定位，再进行截取；本例不把已知起点的截取误称为字符串搜索。

模式匹配与工程取舍

模式匹配是查找一个模式串 P 在文本 T 中首次或所有出现的位置。例如在 `"ababa"` 中查 `"aba"`，首次为 0，可重叠出现位置为 0 和 2。普通搜索与 KMP 的实现、空模式和重叠处理见 BF 与 KMP，此处不重复其推导。

文本编辑器和日志系统需要查找、截取及显示上下文：连续字符串便于顺序扫描和利用局部性；频繁在大文本中间插入可能造成复制，块链或其他分块结构可减少局部搬移，但引入块管理和跨块搜索成本。真实实现还要处理编码、换行、规范化及多字节边界。

分级练习

基础

1. **自编**：`" "` 和空串长度是否相同？答：不相同；前者有一个空格字符。
2. **自编**：`"abcde"[1,4)` 是什么？答：`bcd`。
3. **自编**：`ace` 是 `abcde` 的子串吗？答：不是连续子串，是子序列。

进阶

1. **平台题**：LeetCode 28 找出字符串中第一个匹配项的下标。注意平台约束模式非空，返回 0 基位置或 -1。
2. **自编**：`"aaaa"` 中 `"aa"` 的可重叠出现位置？答：0、1、2。
3. **自编**：复制 `S[2,2)` 的结果是什么？答：空串；按本文约定合法。

综合

1. **自编**：日志由很多短行组成，比较把所有内容连接成一个大串与按块保存的优缺点；讨论跨块匹配、复制成本和下标含义。答案取决于读取、修改和搜索负载，不能只依据一种大 O 结论。

BF 与 KMP：字符串匹配

> 前置：串的表示。本页匹配伪代码统一采用 0 基下标、右开区间，`n=|T|`、`m=|P|`；约定空模式首次匹配位置为 0，找不到返回 -1。比较单位是字符，示例使用 ASCII。

问题：在日志中找固定标记

在文本 T 中寻找模式 P 的第一次连续出现。简单方法在每个可能起点逐字符比较；当文本和模式有较多重复前缀时，许多已经知道的字符又被比较。KMP 利用模式自身的前后缀信息，失配后只调整模式中的位置，不让文本下标回退。

BF：逐个起点检查

```
BF(T, P):
    n ← |T|; m ← |P|
    若 m = 0: 返回 0
    对 start 从 0 到 n-m:            // 若 m > n，不进入循环
        j ← 0
        当 j < m 且 T[start+j] = P[j]: j ← j+1
        若 j = m: 返回 start
    返回 -1
```

非空模式时最多检查 `max(0,n−m+1)` 个起点，每个起点最多比较 m 次，最坏 O(nm)，额外空间 O(1)。当首次位置就完整匹配时仍要读取 m 个字符；平均表现依赖文本、模式和字符分布，不能笼统保证 KMP 在每次输入都更快。

前缀函数 π：失配后保留什么

`π[i]` 是子串 `P[0..i]` 的**最长相等真前后缀长度**；真前缀和真后缀不等于整个串。若已匹配 j 个字符，当前文本字符与 P[j] 失配，则可保留长度 `π[j−1]` 的前缀，把 j 改为该长度，文本位置不动；若 j=0 仍不等，才推进文本。

以 P=`"ababd"` 为例：

i	P[i]	前缀 P[0..i]	最长相等真前后缀	π[i]
0	a	a	空	0
1	b	ab	空	0
2	a	aba	a	1
3	b	abab	ab	2
4	d	ababd	空	0

```
构造前缀函数(P):
    m ← |P|
    若 m = 0: 返回空数组
    π[0] ← 0; j ← 0
    对 i 从 1 到 m-1:
        当 j > 0 且 P[i] ≠ P[j]: j ← π[j-1]
        若 P[i] = P[j]: j ← j+1
        π[i] ← j
    返回 π
```

j 每次回退都会减小，而整个构造过程中增加次数有线性上界，因此构造 O(m) 时间、O(m) 数组空间。

KMP 查找伪代码

```
KMP首次匹配(T, P):
    若 |P| = 0: 返回 0
    π ← 构造前缀函数(P)
    j ← 0
    对 i 从 0 到 |T|-1:
        当 j > 0 且 T[i] ≠ P[j]: j ← π[j-1]
        若 T[i] = P[j]: j ← j+1
        若 j = |P|: 返回 i-|P|+1
    返回 -1
```

不变量：处理下一个文本字符之前，`P[0..j)` 与已经扫描文本的某个最长可延续后缀相等。每次回退使 j 减少，文本 i 只向前推进，累计匹配 O(n)；含预处理为 O(n+m)，辅助空间 O(m)。若模式比文本长，仍可安全返回 -1。若需要全部可重叠匹配，在完整匹配后先记录位置，再令 `j←π[m−1]` 并继续；不允许重叠时可令 j←0。

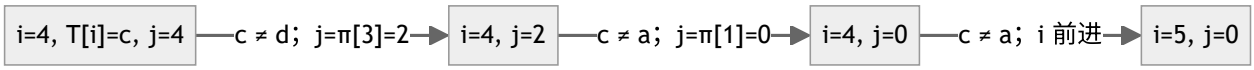
完整案例：失配后的两次回退

背景与输入：日志文本 `T="ababcababd"`，搜索固定标记 `P="ababd"`。BF 可以逐起点回试；KMP 预处理 `π=[0,0,1,2,0]`，适合模式重复查询或文本中有重复前缀的情况。下表 `j` 是比较**之前**已匹配字符数；箭头表示处理当前字符后状态。

文本 i	T[i]	比较前 j	比较/动作	比较后 j
0	a	0	等于 P[0]	1
1	b	1	等于 P[1]	2

文本 i	T[i]	比较前 j	比较/动作	比较后 j
2	a	2	等于 P[2]	3
3	b	3	等于 P[3]	4
4	c	4	不等于 P[4]=d，回退至 π[3]	2
4	c	2	不等于 P[2]=a，回退至 π[1]	0
4	c	0	不等于 P[0]=a，推进 i	0
5	a	0	等于 P[0]	1
6	b	1	等于 P[1]	2
7	a	2	等于 P[2]	3
8	b	3	等于 P[3]	4
9	d	4	等于 P[4]，完成匹配	5

结果起点 `9-5+1=5`，对应 `T[5..10)="ababd"`。文本位置 i=4 在两次回退期间没有前移，直到 j=0 仍不匹配才推进；已经匹配的 `abab` 的后缀 `ab` 可作为候选前缀。



本例包括预处理 O(m) 和扫描 O(n)，辅助 O(m)。若只搜索一个很短的模式，BF 的简单性可能更合适；同一模式搜索多个文本时可以复用 π。

与教材式 next、nextval 的对照

下面另列原资料常见的**1 基教材约定**，只用于对照；不能直接把这些数组放入上面的 0 基 π 伪代码。

- 模式字符编号为 1..m；定义 `next[1]=0`，对 `2≤j≤m`，`next[j]=π[j-2]+1`。`0` 是哨兵，表示第一个字符失配后要推进文本；`1` 才表示失配前的最长相等真前后缀长度为 0。
- 教材式匹配若在第 j 个字符失配，则令 `j←next[j]`；当 j=0 时推进文本并把 j 设回 1。这个 j 是**下一待比较字符的 1 基位置**，不是上文已匹配长度。
- `nextval[1]=0`；对 j≥2，令 k=next[j]：若 k>0 且 `P[j]=P[k]`，令 `nextval[j]=nextval[k]`，否则 `nextval[j]=k`。它跳过已知会重复失配的相同字符；使用其匹配循环也必须遵守 1 基哨兵约定。

j（1 基）	P[j]	next[j]	nextval[j]
1	a	0	0
2	b	1	1
3	a	1	0
4	b	2	1
5	d	3	3

例如 j=3 时 P[3] 和 P[next[3]]=P[1] 都为 a；若文本当前字符已与 P[3] 失配，再比较相同的 a 仍会失配，所以可以跳到 nextval[1]=0。其他教材对 next 的定义可能不同，不能只看数组名套公式。

周期与边界

对非空串 s，长度 n，`p=n−π[n−1]` 是其最小正周期长度；若 `n mod p=0`，它可由长度 p 的前缀完整重复 `n/p` 次。若要求**至少重复两次**，还须 `p<n`；例如 `ab` 的 π 为 `[0,0]`、p=2，不是两个 `a`。周期不要求整除：`ababa` 的最小周期为 2，但它不是由 `ab` 完整重复整数次得到。

空模式返回 0 是本教材 API 约定；某些题目不接受空模式。单字符、全相同字符、模式长于文本、失配发生在首字符以及重叠匹配都要测试。前缀函数的 `π[m−1]` 与教材式 `next[m]` 不是同一含义：后者只涉及失配位置之前的前缀，不能直接将 `m-next[m]` 当通用周期公式。

工程应用

搜索引擎、日志过滤或编辑器都可能反复扫描文本。预处理可复用的模式能避免重复工作；流式处理还需保存跨块的 j 状态。真实系统要确认大小写、编码、Unicode 规范化和下标单位，库函数也可能采用其他高效算法；不应假设每种输入下 KMP 都是最快方案。

分级练习

基础

1. **自编**：P=`aba` 的 π 是什么？答：`[0,0,1]`。
2. **自编**：本文 KMP 中空模式首次匹配位置？答：0；这是 API 约定。
3. **自编**：P=`ababd` 的 1 基 next[4] 是多少？答：2，对应 π[2]+1。

进阶

1. **平台题**：LeetCode 28 找出字符串中第一个匹配项的下标。平台限制模式非空，返回 0 基位置或 -1。
2. **平台题**：LeetCode 459 重复的子字符串。可用 π 求 p，检查 p<n 和 n mod p=0，时间 O(n)、辅助空间 O(n)。
3. **自编**：在 `aaaa` 中找 `aa`，允许重叠时位置为多少？答：0、1、2；匹配后令 j=π[1]=1。

综合

1. **自编**：将文本分成 `ababc` 与 `ababd` 两块，逐块执行同一次 KMP 扫描，说明需要跨块保留什么。答：保存模式 π、当前匹配长度 j 和全局偏移；本例在第一块处理完 j=0，结果全局起点仍为 5。另试把块切在 `ab` 后，说明 j 不应无条件清零。

第 05 章　树与二叉树基础

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

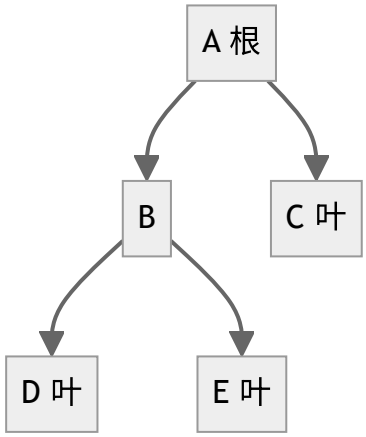
树与二叉树基础

> 前置：算法基础与复杂度、数组与矩阵。本文根为第 1 层，树高按层数计；空树高度为 0。结点数记为 n，树高记为 h。

从文件夹层级开始

文件系统中的文件夹可以包含子文件夹，路径反映层级归属。若每个非根目录只有一个上级，就可用树描述这种关系；文件系统中的符号链接可能形成一般图，不能不加区分地把所有路径关系都当作树。

树是结点的有限集合：空集合为空树；非空时有唯一根，其余结点分为互不相交的若干子树。结点的**度**是孩子数；度为 0 的是叶子。双亲、孩子、兄弟描述相邻层的关系，祖先与后代描述沿多条边向上或向下的关系。树的度是所有结点度的最大值，层数从根为 1；高度取最大层数。空树没有根、叶子和非空树的最大度，约定高度 0。



图中 A 的度为 2，B、C 是兄弟，D、E 的双亲是 B；总计 5 个结点、4 条边、高度 3。非空树每个非根结点恰有一条来自双亲的边，故边数是 n−1，所有结点度之和也为 n−1。它不适用于 n=0 的空树。

二叉树与形态

二叉树为空或由一个根、左子树和右子树组成；左右位置有区别，即使只有一个孩子，也要区分左孩子和右孩子。不要把“度不超过 2 的普通树”直接等同于不区分左右的二叉树。

按根和子树是否存在，二叉树有五种基本结构：空树；只有根；根只有左子树；根只有右子树；根同时有左、右子树。后面三种中的子树本身也可以继续有自己的孩子。这五种结构是递归定义的基本情形，与下述“满”“完全”等整体形态分类是不同角度。

- **满二叉树**：按本文教材约定，h 层都排满，共 2^h-1 个结点；英语 full 在不同资料中也可能指每个结点有 0 或 2 个孩子，读英文时应看定义。
- **完全二叉树**：结点位置与同高度满二叉树按层、从左到右编号的前 n 个位置一致。最底层可能未满，但已有结点靠左连续排列。满二叉树必为完全二叉树，反之不一定。
- **正则二叉树**：每个结点有 0 或 2 个孩子，不要求叶子全在同一层。它与完全二叉树都不是对方的必要条件。
- **BST、AVL** 在此是以键顺序或高度平衡为约束的二叉树，性质和操作在 BST 与 AVL 讲解，不将它们与“满/完全”混为一谈。

重要性质与推导

非空二叉树中，设 n_0、n_1、n_2 分别为度数 0、1、2 的结点数。按结点数与边数计数：

$n=n_0+n_1+n_2$，$n-1=n_1+2n_2$，两式相减得到 **$n_0=n_2+1$**。这个公式要求树非空；空树不满足 0=0+1。

性质	结论与条件
第 i 层最多结点	2^{i-1}，$i\geq 1$；每个结点至多产生两个孩子
高度为 h 的二叉树最多结点	2^h-1；逐层求和
非空完全二叉树高度	$\text{floor}(\log_2 n)+1$，等于 $\text{ceil}(\log_2(n+1))$
完全二叉树的叶子数	$\text{ceil}(n/2)$；编号 $\text{floor}(n/2)+1..n$ 为叶子
1 基层序编号的完全二叉树	结点 i 的双亲 $\text{floor}(i/2)$（$i>1$），左孩子 $2i$、右孩子 $2i+1$；孩子编号不超过 n 才存在
空链接数	对 n 个真实结点的二叉链表，2n 个孩子域中有 $n-1$ 条边，故空孩子域 $n+1$；$n>0$。

有的教材将高度按边数而不是层数定义，公式需要相应调整。对一般二叉树，1 基堆式编号仍能表达位置，但数组可能出现大量空洞；例如高度为 h 的右斜树需要的最大编号随 h 指数增长。

存储与操作成本

顺序存储用数组按层序编号保存完全二叉树，直接计算孩子位置，空间 Θ(n)，常用于堆；稀疏或退化形态可能浪费大量槽位。**二叉链表**结点保存值、左孩子和右孩子引用，空间 Θ(n)，访问已知结点的孩子 O(1)，但找任意值通常需要遍历 O(n)。如果额外保存双亲引用，可 O(1) 向上移动，但更新时须维护它。遍历和构造方法见 遍历与线索。

案例：检查一棵完全二叉树

仓库按层排列 6 个货架位置，编号从 1 开始，判断叶子与孩子位置。将货架按编号放在数组中：



编号为 1..6 且没有中间空洞，是完全二叉树。高度 $\text{floor}(\log_2 6)+1=3$；编号大于 floor(6/2)=3 的 4、5、6 是叶子，共 3 个；结点 3 的左孩子编号 6 存在，右孩子编号 7 超过 6，不存在。用数组判断一个已知编号的孩子只需一次乘法和边界比较，为 O(1)；整个数组占 O(n)。如果输入是一棵链式二叉树，不能只看结点数，需要层次遍历并检查空位置

后的结点，详见遍历章。

工程取舍

树适合目录层级、表达式语法结构以及堆的组织；数组堆适合接近完全二叉树且需按位置访问的场景，链式结点适合结构变化和稀疏形态。已知局部连接的修改可以很快，但查找目标位置、内存分配和并发同步仍有成本；不要将某个操作的 O(1) 扩展为整棵树所有操作均为 O(1)。

分级练习

基础

1. **自编**：只有根的一棵树高度、边数和叶子数各是多少？答：1、0、1。
2. **自编**：非空二叉树 $n_2=4$ 时 n_0 是多少？答：5。
3. **自编**：完全二叉树 n=10，编号 4 的左右孩子？答：8、9。

进阶

1. **平台题**：LeetCode 104 二叉树的最大深度，空树按题设返回 0，递归遍历时间 O(n)、栈 O(h)。
2. **自编**：判断编号 1、2、3、5 的形态是否为完全二叉树。答：否，编号 4 有空洞而 5 存在。
3. **自编**：高度为 4 的二叉树最多多少结点？答：15；至少 4 个结点。

综合

1. **自编**：比较用连续数组和链式结点保存 100 个结点的完全树与极度倾斜树，讨论空间、找孩子和插入位置成本；说明必须先明确编号和操作条件。

二叉树遍历与线索

> 前置：树与二叉树基础、栈与递归、队列。n 为结点数，h 为树高；空树遍历结果为空序列。结果列表若需保存，另占 O(n) 输出空间。

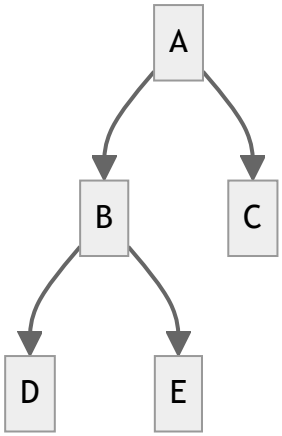
为什么访问次序重要

目录树中的每个文件都要被检查；表达式树则可能需要先处理操作数，再计算运算符。遍历是按确定规则访问每个结点一次，选择不同次序可满足不同任务。假设孩子的左右次序已确定：

- **先序**：根 → 左子树 → 右子树；适合先处理容器再处理内部内容。
- **中序**：左子树 → 根 → 右子树；对严格 BST 会得到递增键序列。
- **后序**：左子树 → 右子树 → 根；适合先计算孩子结果再合并到父结点。
- **层序**：从根开始逐层、每层从左至右；使用队列保存待访问结点。

案例：同一棵树的四种遍历

树表示一个根任务 A，下含 B、C，B 下含 D、E：



规则	执行次序	解释
先序	A B D E C	先根 A，再依次处理 B 子树、C
中序	D B E A C	先 B 的左、B、右，再 A、C
后序	D E B C A	B 的子任务 D、E 先于 B；最后处理 A
层序	A B C D E	根层、第二层、第三层

递归中序伪代码：

```
中序(p):
    若 p = null: 返回
    中序(p.left)
    访问(p)
    中序(p.right)
```

递归栈保存“返回后继续做什么”，是 函数调用栈 的应用；深度可达 h，最坏 O(n)。显式栈版中序按左链入栈，弹出时访问，再转向右子树：

```
中序_显式栈(root):
    S ← 空栈; p ← root
    当 p ≠ null 或 S 非空:
        当 p ≠ null:
            S.push(p); p ← p.left
        p ← S.pop(); 访问(p)
        p ← p.right
```

入栈后按 LIFO 弹出，例如先入 A、B、D，首先弹出 D；空树时外层循环不进入。先序可用栈弹根、先压右再压左；后序要确保两棵子树先访问，例如用显式栈记录“是否已访问右子树”的状态，不能简单地把中序的访问位置后移。层序使用 队列：非空根入队，反复出队访问并依左、右顺序入队非空孩子。

一种非递归后序方法在栈中保存“结点、是否已展开”两个状态；先展开时把稍后访问自身的标记压栈，再依次压入右、左孩子，让左孩子最先出栈。这里只处理非空结点：

```
后序_显式栈(root):
    S ← 空栈
    若 root ≠ null: S.push((root, 未展开))
    当 S 非空:
        (p, 状态) ← S.pop()
        若 状态 = 已展开: 访问(p)
        否则:
            S.push((p, 已展开))
            若 p.right ≠ null: S.push((p.right, 未展开))
            若 p.left ≠ null: S.push((p.left, 未展开))
```

每个结点最多压栈两次，时间 O(n)；待处理条目与树高成正比，辅助空间 O(h)，不计输出。空树不入栈，返回空序列。

层序案例的队列状态：

动作	队列（头→尾）	输出
A 入队	A	空
出 A，入 B、C	B,C	A
出 B，入 D、E	C,D,E	A,B
出 C	D,E	A,B,C
出 D、E	空	A,B,C,D,E

四种遍历均对每个结点处理常数次，时间 O(n)。先/中/后序的递归或显式栈为 O(h) 辅助空间；层序队列最坏 O(n)，实际取决于最大宽度。以上不计结果输出空间；“显式栈先压右再压左”仅指上述先序方案，不自动适用于所有非递归 DFS 写法。

遍历序列重建

若结点标识**互不相同**且序列来自同一棵树，先序+中序可唯一重建：先序第一个是根，在中序中找到根，把中序分成左右子树，再按左右结点数切分先序并递归。上例先序 ABDEC、中序 DBEAC：根 A；中序左段 DBE 有 3 个结点，对应先序左段 BDE，其余 C 为右子树。每个递归子问题按相同规则继续，得到图中结构。

中序+后序、在相同唯一标识条件下的中序+层序也能确定；仅有先序+后序一般不唯一：单个孩子可能是左孩子或右孩子。重复键若没有额外结点身份或约束，重建可能不唯一。用哈希表索引中序位置、传下标范围时，平均 O(n) 时间、索引 O(n) 与递归栈 O(h) 辅助空间；逐次线性查找最坏 O(n^2)。若输入序列不一致，应报告失败，不能随意构

造。

线索二叉树

二叉链表中的空孩子域可用于保存某种遍历次序的前驱或后继；必须另设标志区分孩子和线索。以**中序线索**为例：`ltag=0` 表示 left 是左孩子，`ltag=1` 表示 left 是中序前驱；`rtag=0` 表示 right 是右孩子，`rtag=1` 表示 right 是中序后继。首结点前驱、末结点后继可为 null（也有带头结点的其他约定）。建线索时先按中序遍历，保存上一个访问结点 prev：当前结点无左孩子时设前驱线索；prev 无右孩子时设其后继线索。遍历结束要按约定处理最后结点。建线索时间 O(n)，递归构建辅助空间 O(h)，标志和线索存于原结点。

以上例的中序 `D B E A C` 为准：D 的后继线索指向 B；E 的前驱线索指向 B、后继线索指向 A；C 的前驱线索指向 A，后继为 null。查某结点的中序后继：若 rtag=1，直接取 right；否则进入右子树，沿**真正左孩子**走到最左结点，不能把 ltag=1 的线索当孩子继续走。前驱对称处理。单次查找最坏 O(h)，连续线索中序遍历全部结点总时间 O(n)、不计输出的额外工作空间 O(1)。

工程应用与练习

表达式树常用后序计算：先算两个子表达式，再处理当前运算符；语法树的具体结点、类型检查和异常处理由语言决定。层序访问适合按层展示组织结构，但队列峰值随树宽增长；链式树的递归遍历对极深树可能耗尽调用栈，显式栈只是把状态转移到可控的存储中。

基础

1. **自编**：上图中序是什么？答：D B E A C。
2. **自编**：空树的层序输出是什么？答：空序列。
3. **平台题**：LeetCode 94 二叉树的中序遍历。

进阶

1. **平台题**：LeetCode 102 二叉树的层序遍历。输出按层分组；每轮先记录当前队列长度。
2. **平台题**：LeetCode 105 从前序与中序遍历序列构造二叉树。题目保证元素不重复。
3. **自编**：上图 E 的中序前驱、后继是谁？答：B、A；说明如何区分线索和孩子。

综合

1. **平台题**：LeetCode 236 二叉树的最近公共祖先。先明确题目给定的两个结点都存在；可用后序信息合并，时间 O(n)、递归栈 O(h)。

树、森林与存储表示

> 前置：树与二叉树基础、遍历与线索。本页的树为有序树，孩子从左到右有确定次序；空指针为 null。

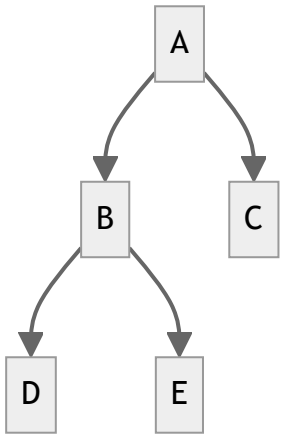
为什么一棵树有多种表示

一个组织有部门，每个部门又有不同数量的团队。若要频繁找上级，保存双亲方便；若要列出下属，保存孩子关系方便。表示方式是实现选择，不改变树本身的父子关系。

表示	记录内容	主要代价
双亲表示	每个结点记录双亲下标；根用特殊值如 -1	已知结点找双亲 O(1)；无额外索引时找全部孩子需扫描 O(n)
孩子表示	每个结点指向其孩子列表	枚举结点的 d 个孩子 O(d)；若未另存双亲，找双亲可能 O(n)
孩子兄弟表示	`firstChild` 指向第一个孩子，`nextSibling` 指向下一个兄弟	O(n) 存储；逐个访问孩子 O(d)，不能把兄弟指针当作原树的孩子

案例：部门树转成二叉链式表示

输入与目的：组织 A 下有 B、C，B 下有 D、E，C、D、E 无孩子。要用统一的两条链接保存任意数量的孩子。选孩子兄弟表示：第一个孩子走左链接，下一个兄弟走右链接。



原树结点	第一个孩子 → 二叉左孩子	下一个兄弟 → 二叉右孩子
A	B	null
B	D	C
C	null	null
D	null	E
E	null	null

结果二叉树链接为 A.left=B、B.left=D、B.right=C、D.right=E。注意二叉树中的 B→C 是右孩子链接，但在原树中 C 是 B 的**兄弟**，不是 B 的下属。每个结点检查一次、每条原树边处理常数次，转换时间 O(n)，结果结点空间 O(n)；如果新建表示，递归或栈的额外空间视实现而定。

```
构建孩子兄弟(T):                // 输入是有序树；输出为新表示
    若 T 为空：返回 null
    为每个原结点建立对应结点
    对每个结点 p:
        若 p 有孩子:
            p.firstChild ← 第一个孩子的对应结点
            按原顺序将相邻孩子的 nextSibling 相连
        最后一个孩子的 nextSibling ← null
    返回根的对应结点
```

实现需先建立原结点与新结点的对应关系，或采用递归返回新结点；映射表为 O(n) 额外空间，不能把它漏掉。伪代码未赋值的空链接应初始化为 null。

遍历的对应关系

上面的原树先根序为 A B D E C，后根序为 D E B C A。孩子兄弟表示所得二叉树的先序和中序恰好分别是这两个序列：先序先访问结点再依次访问孩子；二叉中序先处理第一个孩子及兄弟链，再访问该结点，对应原树后根。

森林是一组有序的不相交树。先将各棵树转成孩子兄弟形式，再把各棵树的根依次用右兄弟链接连接；这时得到的是整片森林对应的一棵二叉树。森林先序对应二叉树先序，常见教材定义的森林中序对应二叉树中序。对单棵树，先根/后根分别对应二叉树先序/中序；不要把原树后根误写成对应二叉树后序。

例如森林由 A(B,C) 和 D(E) 构成：二叉链接为 A.left=B、B.right=C、A.right=D、D.left=E。二叉先序为 A B C D E，二叉中序为 B C A E D，分别对应森林先序、中序。只转一棵树时，树根没有同层兄弟，其 right 为 null；森林中后续根要通过右链接连接。

工程应用

文档解析器或目录系统常有可变数量的子项。孩子列表直接适合逐项枚举，双亲指针便于沿路径向上；孩子兄弟表示用固定两个链接支持多叉结构，但按第 k 个孩子访问需要沿兄弟链行走 O(k)，不适合频繁随机访问。真实数据如果存在共享子结点、循环引用或跨链接，则可能是图，应先定义访问去重与环检测。

分级练习

基础

1. **自编**：上例 B 的第一个孩子和下一个兄弟分别是谁？答：D、C。
2. **自编**：双亲数组中的根需要什么约定？答：用 -1 等不与合法下标冲突的标记。
3. **自编**：孩子兄弟表示中，原树叶子一定没有二叉树右孩子吗？答：不一定，右链接可能指向它的兄弟。

进阶

1. **自编**：树 R 的孩子依次为 X、Y、Z，列出 firstChild 和 nextSibling。答：R.firstChild=X，X.nextSibling=Y，Y.nextSibling=Z，Z.nextSibling=null。
2. **自编**：仅保存双亲数组，列出某结点所有孩子最坏要多久？答：扫描 n 个结点为 O(n)，若另建索引则另论。

3. **自编**：森林 A(B,C)、D(E) 的森林先序是什么？答：A B C D E。

综合

1. **自编**：设计文档目录的表示，要求频繁列出全部子项与偶尔回溯上级；比较孩子列表和额外双亲指针的空间、维护及访问成本，并说明出现快捷链接时为什么需要图模型。

第 06 章　查找树、哈夫曼与并查集

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

二叉排序树与 AVL 树

> 前置：二叉树基础、遍历、复杂度。本文严格 BST 不存重复键；高度按层数计，空树高度为 0。比较成本假设为 O(1)。

为什么既要有序又要动态更新

应用维护一组不断增减的订单号，需要按键查找，也可能按顺序输出。排序数组支持 O(log n) 折半查找，但中间插入需移动元素；二叉排序树通过链接组织有序键，操作成本取决于树的高度。若更新顺序使树严重倾斜，性能也会恶化。

二叉排序树（BST）是空树，或每个结点都满足：左子树所有键小于当前键，右子树所有键大于当前键，左右子树本身也为 BST。只检查直接孩子不够：根 10 的左孩子为 5、5 的右孩子为 20，虽然局部有 5<10、20>5，20 却位于 10 的左子树，违反全局约束。严格 BST 中序遍历得到严格递增序列。

查找、插入和删除

```
查找(root, key):
    p ← root
    当 p ≠ null:
        若 key = p.key: 返回 p
        若 key < p.key: p ← p.left
        否则: p ← p.right
    返回 null
```

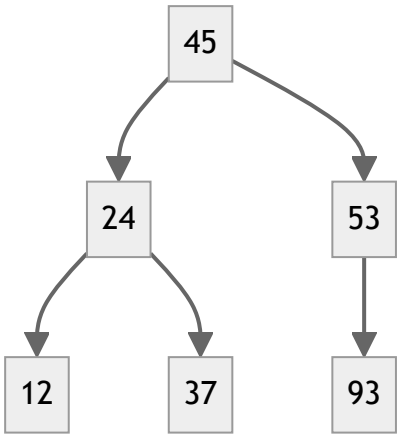
插入从根沿同一比较路径向下，在遇到的空链接处接入新叶；键已存在时本页约定不插入。查找、插入均为 O(h)。空树查找返回 null，空树插入需更新根引用。

删除分三类：叶结点直接摘除；只有一个孩子时让该孩子接到原位置；有两个孩子时，可复制左子树最大结点（直接前驱）的键到目标，再从左子树删除该前驱；前驱没有右孩子，因此后续至多有一个孩子。也可对称使用右子树最小结点作为后继。若数据记录与键绑定，替换时必须维护完整记录及外部引用语义。

高度修正：对上述普通 BST 的前驱/后继替换删除，不进行旋转，新树高度不会比原树更高；它可能不变或降低。复制键不改变结构，摘除前驱/后继至多把其唯一孩子上移。AVL 的删除另有重新平衡过程，不能把这个推理不加说明地套到任意自平衡实现。

案例：构造 BST 并计算查找长度

按顺序插入 `45,24,53,12,37,93`，每次从根比较：24 小于 45 到左；53 大于 45 到右；12、37 分别成为 24 的左右孩子；93 为 53 的右孩子。



这棵树的高度为 3；查找 37 比较 45、24、37 共 3 次；查找 30 比较 45、24、37 后落到空链接，共 3 次**真实键比较**。若六个已有键等概率被成功查询，比较次数依次是 `1,2,2,3,3,3`，成功 ASL 为 `14/6=7/3`。

失败 ASL 必须给出失败情况的概率。本例共有 7 个空链接；若人为约定七个键区间**等概率**，按仅计真实键比较，结点 53 左空贡献 2 次，其余 6 个空链接的双亲都在第 3 层各贡献 3 次，失败 ASL 为 `(2+6×3)/7=20/7`。若将最后的空链接检查也单独计一次，则该特定计数为 27/7，不能把它叫作完全相同的“关键字比较次数”。实际请求不等概率时要以各区间概率加权。

若高度 h=O(log n)，查找、插入、普通删除都是 O(log n)；递增序列插入可生成高度 n 的单链，最坏均为 O(n)。构造 n 个结点最坏 $O(n^2)$，不能无条件写 O(n log n)。递归操作栈 O(h)，迭代查找辅助 O(1)，均不含树本身 O(n) 存储。

AVL：用旋转限制高度

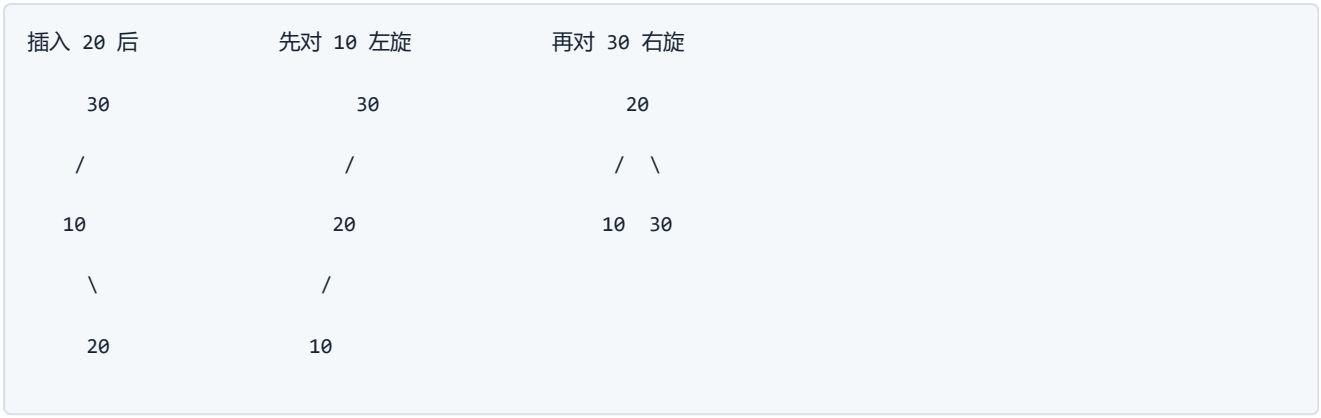
AVL 是满足 BST 性质的二叉树，并要求每个结点左右子树高度差的绝对值不超过 1。本文平衡因子 `BF=左高-右高`。插入后从下往上检查第一个失衡祖先：

类型	插入方向	修复
LL	左孩子的左侧	对失衡结点右旋
RR	右孩子的右侧	对失衡结点左旋
LR	左孩子的右侧	先左旋左孩子，再右旋失衡结点
RL	右孩子的左侧	先右旋右孩子，再左旋失衡结点

旋转只改变局部链接，不改变中序键次序；需先更新旋下结点高度，再更新新子树根的高度。删除也可能失衡，并可能沿多层祖先继续调整；不能把“插入修复一次后高度恢复”的性质无条件套给删除。

完整案例：LR 双旋

插入序列 30、10、20：插入前两项后，30 的左孩子为 10；插入 20 后，它位于 10 的右边，30 的 BF=2、左孩子 BF=-1，属于 LR。



中序在三幅图中始终为 `10,20,30`；完成后根 20 的 BF=0，两个叶子 BF=0。已定位失衡位置后的每次旋转改常数条链接，O(1)；从根查找、沿路径更新高度为 O(log n)。要把旋转后的**新子树根**接回其父结点，若旋转的是整棵树的根还要更新 root。

令 N(h) 为高度 h 的 AVL 树最少结点数，`N(0)=0,N(1)=1,N(h)=1+N(h-1)+N(h-2)`（h≥2）：为满足平衡且有高度 h，两子树最小高度可取 h−1、h−2。这个递推随 h 指数增长，因此高度 O(log n)。插入与删除的查找和沿路径重平衡最坏 O(log n)，维护高度使用每结点常数额外字段；递归实现工作栈 O(log n)。

红黑树与工程取舍

红黑树也是自平衡 BST：通常用黑色空叶哨兵、根为黑色、红结点不能有红孩子、从任一结点到其后代空叶的路径黑结点数相同等规则限制高度。查找和更新最坏 O(log n)，但颜色与旋转维护规则不同于 AVL，不能将 AVL 的 BF 判断直接套用。AVL 的严格高度约束利于控制查找路径，但实际性能还取决于更新比例、内存布局和实现；数据库磁盘页索引常采用多路结构，详见 B 树与 B+ 树。

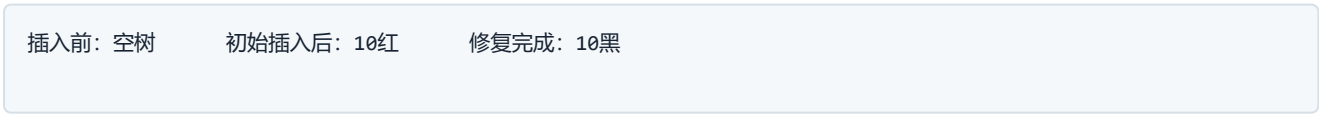
红黑树插入案例：重着色与旋转

本例从空树依次插入 **10、5、15、1、3**，键不重复；按普通 BST 定位后，先将新真实结点染红，再按需要修复。插红色结点不会像插黑色结点那样直接使经过该位置的路径多一个黑结点，但可能与红父结点相邻；首个结点作为根时还须染黑。

红黑性质：根为黑；每个 NIL 外部叶为黑；红结点的孩子均为黑；从任一结点到其后代 NIL 的各条路径黑结点数相同。图中省略的每个空孩子均表示黑色 NIL。本例的黑高 `bh(x)` 按**不计起点 x、计入终点黑色 NIL**统计；例如红或黑的真实叶结点到其 NIL 孩子的黑高均为 1。NIL 是逻辑上的黑色外部叶，程序可用哨兵或以空指针配合判色规则表示，但不可解引用空指针。

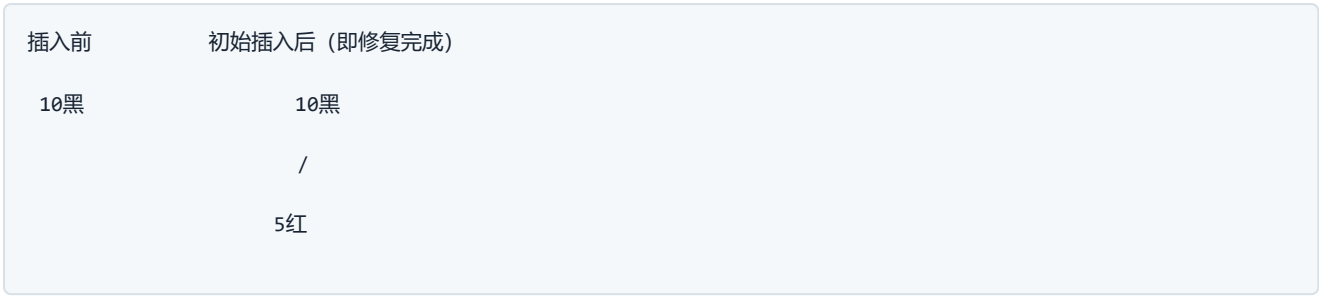
图中 黑、红 标记真实结点颜色。下面“初始插入后”尚未完成修复，允许临时出现不合法状态；每步“修复完成”才要求整棵树满足性质。

1. 插入 10：初始红根违反根黑，将根染黑。

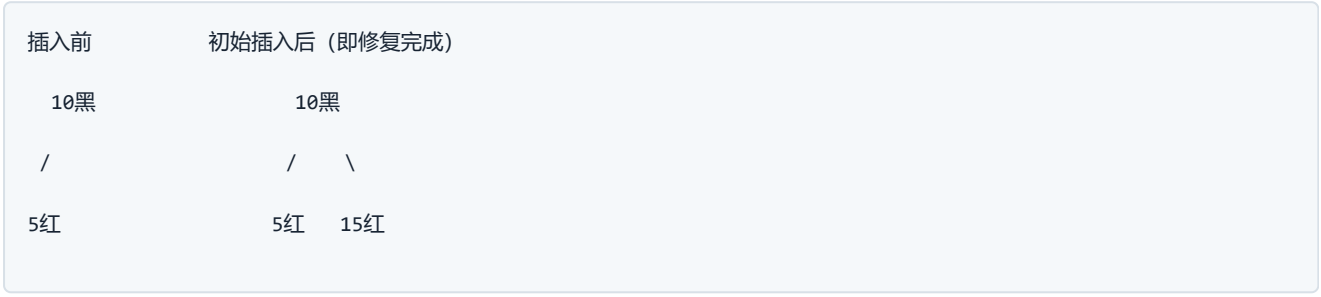


根黑高为 1；左右空孩子均为黑 NIL。

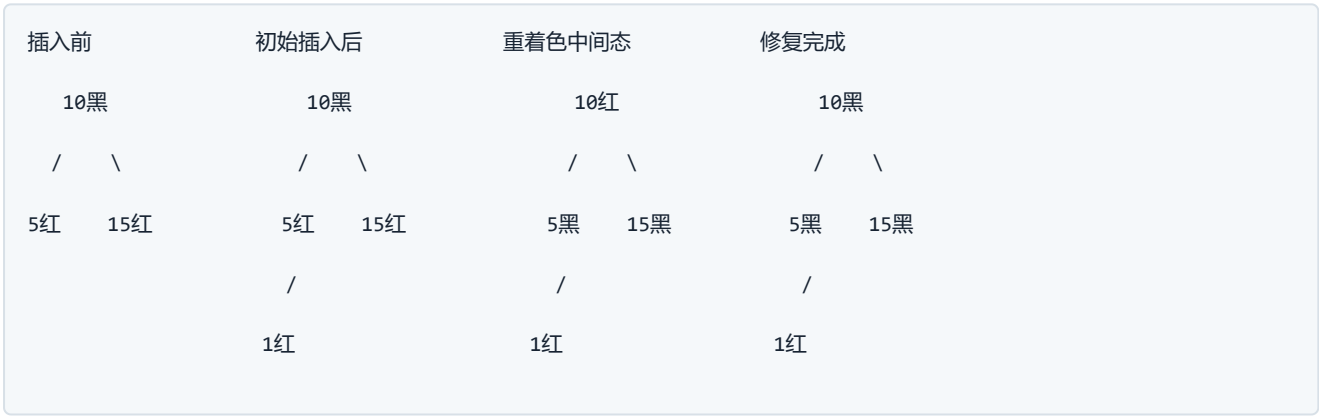
2. 插入 5：父 10 为黑，不产生红红冲突，无需修复。



3. 插入 15：父 10 为黑，仍无需修复。

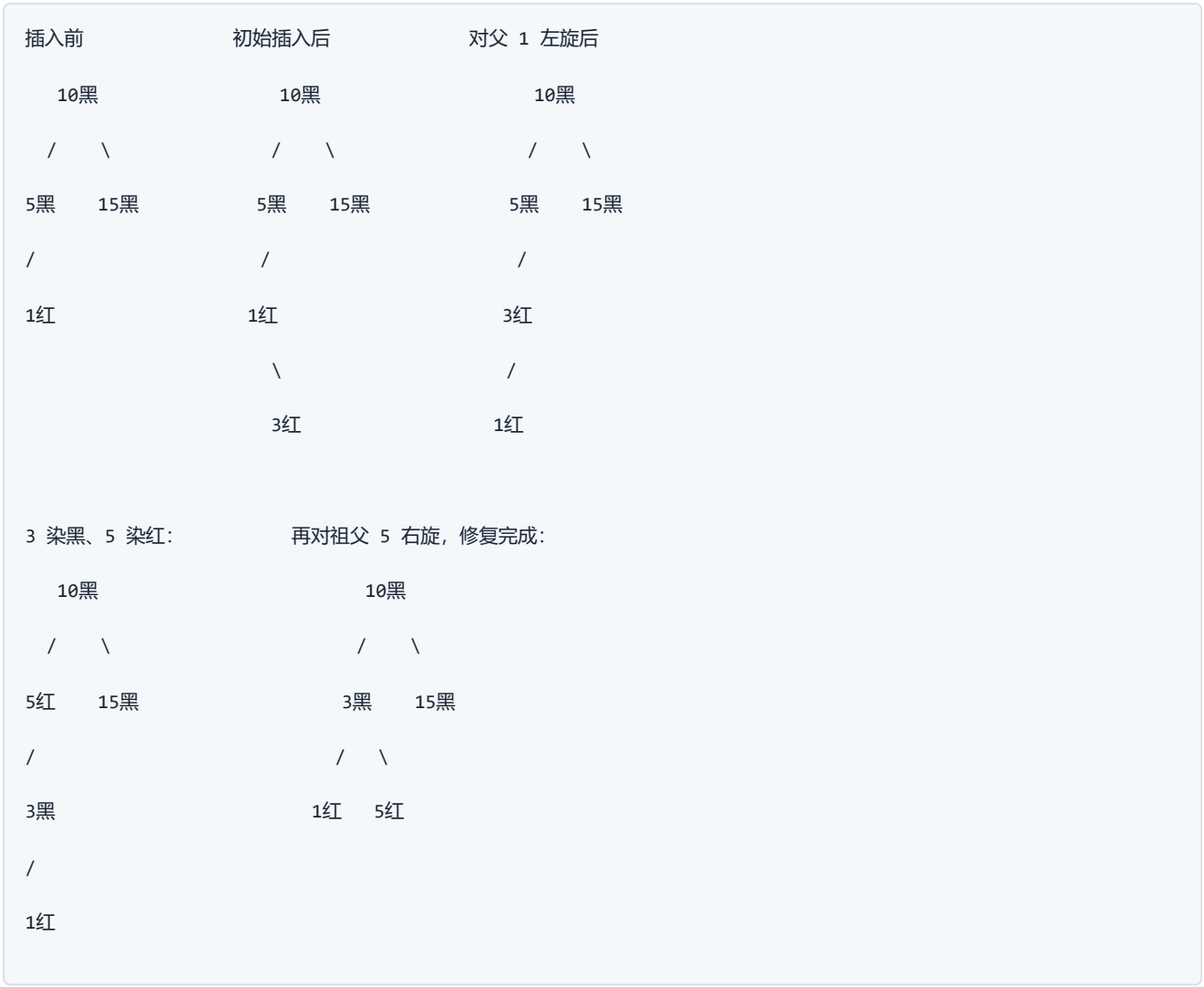


4. 插入 1：初始状态中 1红 的父 5红 也是红，违反红结点无红孩子；叔叔 15红。将父 5 和叔叔 15 染黑，祖父 10 染红，修复位置上移；因为 10 是根，再将根染黑。



中间态 10红 作为根暂时违反根黑；最终根染黑，各性质恢复。本步不旋转。

5. 插入 3：3红 是 1红 的右孩子，出现红红冲突；祖父是 5黑，叔叔是 5 的右孩子黑 NIL，属于祖父左子树中的 LR 情况。



左旋使 1 的右孩子 3 上升、1 成为 3 的左孩子；再对 5 右旋，使 3 上升、5 成为 3 的右孩子。一般旋转还需正确转接被提升结点的中间子树；本例该子树是 NIL。左旋后、重着色后的中间态不必单独满足全部红黑性质；应在右旋完成后整体检查。两次旋转前后中序均为 `1,3,5,10,15`。

最终根 10 为黑，所有 NIL 为黑；红结点 1、5 的孩子都是黑 NIL，没有红红父子。根 10 的每条后代 NIL 路径都经过一个真实黑结点（3 或 15）和一个黑 NIL，故 `bh(10)=2`；`bh(3)=bh(15)=bh(1)=bh(5)=1`。同时中序严格递增，BST 性质保持。

方向速查：以下均指插入修复时父红、叔黑，方向按祖父→父→新结点判断；实际修复还须配合重着色。本例只实际演示 LR，其他三种是规则对照。

形态	局部旋转方向	颜色处理的要点
LL	对祖父右旋	父染黑、祖父染红
RR	对祖父左旋	父染黑、祖父染红
LR	对父左旋，再对祖父右旋	第一旋转后提升的结点染黑、祖父染红
RL	对父右旋，再对祖父左旋	第一旋转后提升的结点染黑、祖父染红

红叔叔先重着色并向上检查，不直接套用黑叔叔旋转表。红黑树依据颜色和黑高约束修复；AVL 依据左右子树高度差，二者都可旋转，但触发条件不同。常见错误是漏掉根最终染黑、把 NIL 当作无颜色、只核验根而忽略其他结点的黑高，或把本例 LR 双旋误称覆盖了所有修复情况。

分级练习

基础

1. **自编**：依次插入 2、1、3，中序序列？答：1、2、3。
2. **自编**：在上述约定下，空树查找结果？答：null。
3. **自编**：左子树高 3、右子树高 1 时 BF？答：2，当前结点失衡。

进阶

1. **平台题**：LeetCode 98 验证二叉搜索树。必须传播祖先上下界或检查严格递增中序；O(n) 时间、O(h) 栈。
2. **平台题**：LeetCode 110 平衡二叉树。自底向上同时返回高度和失衡状态，O(n) 时间、O(h) 栈；其题设是高度平衡判断，不要求 BST。
3. **自编**：依次插入 10、30、20，第一个失衡结点的类型？答：RL，先对 30 右旋、再对 10 左旋。

综合

1. **自编**：给出只含 1..n 的插入顺序，使普通 BST 高度为 n；再讨论需要有序动态更新时 AVL 与排序数组的查找、插入和范围遍历权衡。答：升序插入会右斜；数组查找 O(log n) 而中间插入 O(n)，AVL 查找与更新最坏 O(log n)。

哈夫曼树与并查集

> 前置：二叉树基础、树的遍历。本页是并查集原理的唯一主讲位置；图模块的 Kruskal 引用这里的合并和判连通操作。

一、哈夫曼树：让高频符号使用较短路径

假设通信系统需要编码不同字符，出现频率不同。固定长度编码容易随机定位，但对高频字符也使用相同长度；若需要无分隔地逐位解码，可采用**前缀编码**：任何字符的编码都不是另一个字符编码的前缀。哈夫曼算法为已知非负权值构造带权路径长度最小的二叉前缀编码树。

叶子权值 w_i 可为出现次数或概率；叶子深度 l_i 是根到该叶的**边数**。带权路径长度 $WPL=\Sigma w_i l_i$；以出现次数作权时，消息总位数的期望模型由权值代表的频率决定。每次取两棵权值最小的树合并为新树，新根权值为两者之和；重复至只剩一棵树。

```
构造哈夫曼树(权值):                // 至少两个叶子，权值非负
    将每个权值建立为一棵叶子树，放入最小优先队列
    当队列大小 > 1:
        a ← 取出最小树; b ← 取出次小树
        新根.weight ← a.weight + b.weight
        新根.left ← a; 新根.right ← b
        将新根放回队列
    返回队列中唯一的树
```

左右分配 0/1 可不同，权值相等时合并选择也可能不同；编码不是唯一的，但在满足条件的构造中最小 WPL 相同。若只有一种符号，树的边长与空码字/至少一位码的约定要单独规定，实际编码还需保证消息可解码；不能直接把一般多符号结论机械套入单符号。

案例：六种字符的编码

背景与输入：A、B、C、D、E、F 的频次依次是 5、9、12、13、16、45。需要一种可即时解码的前缀编码；选择哈夫曼树以最小化这些频次下的总带权路径长度。

轮次	取出的最小权值	合并结果	结束后的权值集合
初始	—	—	5,9,12,13,16,45
1	5,9	14	12,13,14,16,45
2	12,13	25	14,16,25,45
3	14,16	30	25,30,45
4	25,30	55	45,55
5	45,55	100	100

若小权值子树取 0、另一子树取 1，可得到 F=0、C=100、D=101、A=1100、B=1101、E=111。各叶权×深度之和为 `45×1+12×3+13×3+5×4+9×4+16×3=224`，也等于五次新根权值和 `14+25+30+55+100=224`。6 个叶结点且每个内部结点有两个孩子，因 $n_0=n_2+1$，结点总数为 11。

用这组编码发送 `BADFACE`：它是 **7 个字符**，码长依次 4、4、3、1、4、3、3，总计 22 位；若对 6 种字符采用定长 3 位码，该条消息为 7×3=21 位。哈夫曼对给定权值的**加权总量**最优，不保证任意一条具体消息都比定长编码短；这也修正原例把消息称为 8 个字符的错误。

小根堆初始化若逐项插入为 O(n log n)，每次合并执行两次取最小、一次插入，共 O(n log n) 时间；树和堆占 O(n) 空间。若构造后还要显式保存所有码字，空间取决于码字总长度，最坏可能为 O(nh)，不能一概算成 O(n)。编码、解码的成本还与消息和输出的实际位数有关。

二、并查集：维护不断合并的集合

社交系统中用户通过关系逐步连接，需要回答两人是否属于同一连通组。并查集维护不相交集合，支持 `find(x)` 找代表元，`union(a,b)` 合并所在集合。以下采用 parent 指向双亲、根满足 `parent[root]=root`；代表元只是集合标识，不保证是最小编号，也不支持直接回答路径或拆分集合。

```
find(x):
    若 parent[x] ≠ x:
        parent[x] ← find(parent[x])   // 路径压缩
    返回 parent[x]

union(a,b):                          // 按秩合并
    ra ← find(a); rb ← find(b)
    若 ra = rb: 返回
    若 rank[ra] < rank[rb]: 交换 ra, rb
    parent[rb] ← ra
    若 rank[ra] = rank[rb]: rank[ra] ← rank[ra]+1
```

初始化 parent[i]=i、rank[i]=0；调用需保证下标合法。按秩合并把较低秩根接到较高秩根；相等时选择一个根并增加其秩。路径压缩会缩短路径，因此 rank 是用于合并的上界信息，不一定等于当前真实高度。也可用集合大小代替秩，但两种字段和更新规则不可混用。

对 n 个元素初始化 O(n)；随后 m 次操作同时采用按秩和路径压缩时，总成本可表示为 O((n+m)α(n))，所以操作序列的均摊代价接近常数；α 是增长极慢的反阿克曼函数。**均摊不是单次操作的最坏常数保证**。parent、rank 占 O(n)；上面递归 find 还使用与当前路径长度有关的调用栈，若需规避递归可用迭代两遍压缩。

案例：合并连接关系与检查根

元素 0..5 初始各成一个集合，依次考虑边 `(0,1),(1,2),(3,4),(4,5),(2,3)`。每次比较两个端点的根并合并，不需要保存整张图即可维护当前连通分量。

步骤	边	合并后的集合划分
初始	—	{0},{1},{2},{3},{4},{5}
1	(0,1)	{0,1},{2},{3},{4},{5}
2	(1,2)	{0,1,2},{3},{4},{5}
3	(3,4)	{0,1,2},{3,4},{5}
4	(4,5)	{0,1,2},{3,4,5}
5	(2,3)	{0,1,2,3,4,5}

最终有 1 个集合；根据秩相同的选根策略不同，具体 parent 数组可能不同。例如 `parent=[0,0,0,0,3,3]` 是合法的一种状态：0 指向自身，是唯一根；3 的父结点为 0，4、5 经 3 到达 0。对 4 执行带压缩的 find 后，parent[4] 可直接变成 0。不能把 parent[4]=3 理解为“3 一定是根”，也不能在没调用 find 时把直接父结点当作代表元。

图的 Kruskal 算法可把每条候选边的两端交给 find：根相同表示已有连接，再加入该无向边会成环；根不同则可合并。算法流程见最小生成树。并查集适合只增连接的动态连通性；若要删除边、列出实际路径或分析有向可达性，需要其他结构或算法。

工程应用与取舍

哈夫曼编码可作为压缩格式的一部分，但真实压缩系统还要存储或共享码表、处理字节对齐、输入分布变化和单符号边界；频率估计、码表开销和其他压缩步骤影响总收益。并查集可用于批量连通分组和去重的等价关系合并，但不能仅凭它恢复具体连接路径，也不适合直接处理任意删除。

分级练习

基础

1. **自编**：频次 2、3、4，先合并哪两个？答：2、3，得到 5。
2. **自编**：`parent=[0,0,1]` 中 2 的根是谁？答：0，不是直接父结点 1。
3. **自编**：前缀码 `0,10,11` 是否存在一个编码是另一个编码的前缀？答：不存在。

进阶

1. **自编**：权 2、3、4 的 WPL 是多少？答：先 2+3=5，再 4+5=9，WPL=5+9=14。
2. **平台题**：LeetCode 547 省份数量。可按矩阵连接合并集合；扫描 $n\times n$ 邻接矩阵为 $O(n^2)$，还需计并查集操作，额外 parent/rank 空间 $O(n)$。
3. **自编**：按秩合并时两根秩相同，为什么只增加新根的秩？答：合并后最高路径可能增加一层，另一根已不再是根。

综合

1. **自编**：对包含相同权值的字符集，构造两种左右 0/1 分配，比较 WPL、前缀性和逐位解码；说明码表本身及消息分布可能如何影响实际压缩收益。

第 07 章　图

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

图的基本概念与存储

> 前置：数组与矩阵、链表。本文默认有限简单图：无自环、无平行边；顶点数 V=|V|，边数 E=|E|。允许自环或重边时要重新说明计数与表示。

从城市路线开始

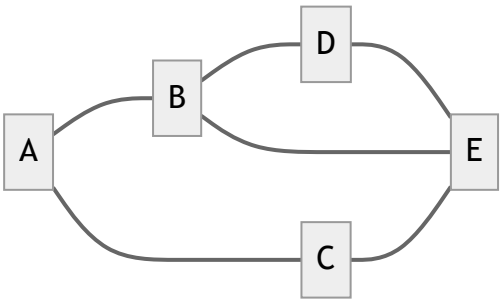
城市之间的道路既可能双向通行，也可能是单行线；两个城市之间还可能有多条路线。线性表只能直接表达一个前后次序，树要求层级关系，而**图**用顶点和边表达一般的多对多关系。建模时先说明“顶点代表什么、边代表什么、方向和权重代表什么”。

图 G=(V,E) 由顶点集合 V 和边集合 E 组成。**无向边** $\{u,v\}$ 没有方向；**有向边** $u\to v$ 只能沿指定方向走。带权图的边可附带距离、费用或时间等权；无权图可以只记录连接关系。

- 无向图的**度**是与顶点相接的边数；有向图的**出度**是从该点出发的弧数，**入度**是到该点的弧数。简单无向图满足度数之和 $2E$；有向图入度之和、出度之和分别都是 E。
- **路径**表示按边连接的一串顶点；有向图必须遵守箭头。路径长度若未特别说明，通常指边数，带权路径的代价则是权值之和。教材中“路径”是否允许重复顶点的用词可能不同，本章明确称允许重复的序列为**游走**。
- 无向图任意两点可达称连通图；其极大连通子图称连通分量。有向图中任意两点相互可达称强连通图；极大强连通子图是强连通分量。单向可达不代表强连通。
- 连通无向图的生成树含所有 V 个顶点和 $V-1$ 条边；非连通图没有覆盖全部顶点的生成树，但各连通分量各选一棵树可组成生成森林。仅有 $E=V-1$ 不保证图是树：三角形加一个孤立点有 4 顶点、3 条边，却有环且不连通。

案例：同一张无向图的两种表示

假设五个站点之间的双向道路为 $\{AB,AC,BD,BE,CE,DE\}$。顶点顺序固定为 A、B、C、D、E，供下一节遍历复用。



各顶点度数分别为 A=2、B=3、C=2、D=2、E=3，总和 12=2×6。无向邻接矩阵按相同顶点顺序为：

	A	B	C	D	E
A	0	1	1	0	0
B	1	0	0	1	1
C	1	0	0	0	1
D	0	1	0	0	1
E	0	1	1	1	0

矩阵关于主对角线对称。等价的邻接表（邻居按字母升序）为 A:`B,C`；B:`A,D,E`；C:`A,E`；D:`B,E`；E:`B,C,D`。无向边在常见邻接表中出现两次，故本例存放 12 个邻接项。构建矩阵先初始化 V^2 个槽，再填 E 条边，为 $O(V^2+E)$ 时间、$O(V^2)$ 空间；按已知顶点映射构建邻接表 $O(V+E)$ 时间和空间。

对带权图，无边不能一概用 0 表示：若存在权重 0 的合法边，会与无边混淆。最短路中常用 ∞ 表示不可达、对角线按算法定义初始化为 0，并在加法前防止哨兵溢出。

选择存储结构

结构	表示方法	空间	典型操作及限制
邻接矩阵	A[u,v] 标记边或保存权	$O(V^2)$	判断指定边 O(1)，扫描一个顶点的邻居 O(V)；适合稠密图和 Floyd
邻接表	每个顶点一列出边/邻居	$O(V+E)$	扫描 u 的出边 O(outdegree(u))；判断指定边通常需扫该表；适合稀疏图
逆邻接表	每个顶点保存入边来源	$O(V+E)$	扫入边方便，普通出边需要另建索引或全图扫描
边集数组	每条边存端点及可选权	O(E)，另算顶点存储	便于按边排序，如 Kruskal；找某顶点邻居通常需扫描所有边

有向图普通邻接表按出边建立；若只存出边，要统计单个顶点的入度通常需扫描所有顶点表和边，$O(V+E)$，不是 O(入度)。无向简单图每条边在普通邻接表记录两次，仍为 $O(V+E)$。

十字链表用于有向图。每条弧结点只存一次，包含弧尾 tail、弧头 head、指向同弧尾下一条弧的 `tlink`、指向同弧头下一条弧的 `hlink`；每个顶点保存首条出弧与首条入弧。比如 A→B 的弧同时在 A 的出弧链和 B 的入弧链中。它便于按方向访问入弧和出弧，代价是维护两套链接。

邻接多重表用于无向图。每条边只存一个结点，记录两个端点 u、v 和各自的下一条关联边链接；从 u 遍历时沿 u 对应链接走，从 v 遍历时沿 v 对应链接走。边的访问标记可放在共同边结点，但插删时必须维护两个端点的链；是否 O(1) 删除取决于是否已经取得对应前驱等信息。

若 A 是无权图的 0/1 邻接矩阵，普通算术矩阵幂的 `(A^k)[i,j]` 计数长度恰为 k 的**游走**数，允许重复顶点和边；它不一般等于不重复顶点的简单路径数。

工程应用与练习

交通、依赖和社交关系都可建模为图，但道路方向、权限、实时状态与权重定义可能不同。稀疏网络通常优先考虑邻接表；密集且频繁判断任意两点是否相连时矩阵可能更方便，代价是 V^2 空间。不要仅凭稀疏或稠密标签推断实际系统速度。

基础

1. **自编**：有向边 A→B 对 A 的出度和 B 的入度各贡献多少？答：各 1。
2. **自编**：本案例中 B 的度和图的边数？答：3 和 6。
3. **自编**：权值 0 能否一律当作无边？答：不能，可能是合法零权边。

进阶

1. **平台题**：LeetCode 1971 寻找图中是否存在路径。注意题目是无向图，可用遍历；邻接表 O(V+E) 时间和空间。
2. **自编**：图由三角形和孤立点组成，为什么不是树？答：既有环又不连通，虽然 E=V−1。
3. **自编**：有向图若需要频繁列出某个顶点的入边，普通出边邻接表与十字链表有什么差异？提示：前者可能全图扫描，后者直接沿入弧链走。

综合

1. **自编**：设计一个包含单行路、双向路和旅行时间的站点网络。比较邻接矩阵、邻接表和边集数组对查询邻居、查询指定边、按费用排序的成本；说明如何区分零权边和无边。

图的遍历：DFS 与 BFS

> 前置：图的表示、栈与递归、队列。V 为顶点数，E 为边数；空图无起点，整图遍历输出空序列。

搜索站点网络

需要从某个站点找出所有可达站点。图可能含环：如果不标记访问过的顶点，搜索就会在环中反复走。深度优先搜索（DFS）沿一条路线尽量深入再回退；广度优先搜索（BFS）先访问距起点一条边、再两条边的顶点。

递归 DFS 与队列 BFS

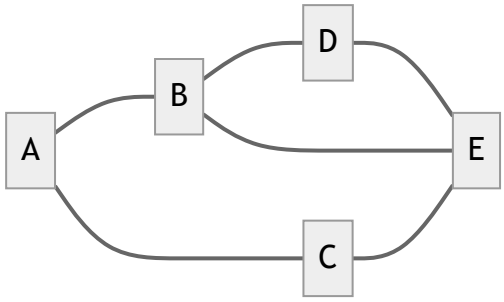
```
DFS(u):                          // visited 初始均为 false
    visited[u] ← true; 访问 u
    按约定顺序枚举 u 的邻居 v:
        若 visited[v] = false: DFS(v)


BFS(s):                          // visited 初始均为 false
    visited[s] ← true; s 入队
    当队列非空:
        u ← 出队; 访问 u
        按约定顺序枚举 u 的邻居 v:
            若 visited[v] = false:
                visited[v] ← true    // 入队时标记，避免重复入队
                v 入队
```

DFS 的递归栈记录回退位置；要模拟与递归一致的次序，显式栈须按具体压栈与标记策略设计，不能只笼统说“逆序压栈就一定相同”。BFS 的队列实现详见前置章节。对于不连通无向图，从一个起点只能访问其连通分量；整图遍历需按顶点顺序再次从未访问顶点启动。对有向图，外层启动次数不等于强连通分量数。

案例：同一张图的 DFS 和 BFS

使用图的表示中的无向图，边为 AB、AC、BD、BE、CE、DE；邻居按字母升序。从 A 开始。



DFS 每次进入最小的未访问邻居：A→B→D→E→C。栈中记当前未返回的递归调用；读到已访问邻居时跳过。

首次访问	递归调用栈（底→顶）	解释
A	A	起点
B	A,B	A 的首个未访问邻居
D	A,B,D	B 的未访问邻居，跳过已访问 A
E	A,B,D,E	D 的未访问邻居
C	A,B,D,E,C	E 的未访问邻居；其余邻居最终均已访问

之后 C、E、D、B、A 依次返回。DFS 序列 `A,B,D,E,C`；若邻接表顺序改变，合法序列也可能改变。

BFS 在入队时标记，状态如下：

出队访问	本轮新入队	操作后队列（头→尾）	已输出
—	A	A	空
A	B,C	B,C	A
B	D,E	C,D,E	A,B
C	无	D,E	A,B,C
D	无	E	A,B,C,D
E	无	空	A,B,C,D,E

BFS 序列为 `A,B,C,D,E`。例如 C 检查到 E 时，E 已在 B 访问时标记并入队，因此不重复入队。

无权图中 BFS 的层号给出源点到各可达点的最少边数：A=0，B、C=1，D、E=2。若记录首次发现每个顶点时的前驱，可以回溯一条最短路径；多条等长路径时结果依赖邻居顺序。权重不等时最少边数不一定对应最小权值，需看最短路径。

成本与适用条件

采用邻接表并对整张图遍历，初始化 visited、扫描每个顶点及邻接项，时间 $O(V+E)$；递归 DFS 的栈、BFS 队列、visited 最坏各 $O(V)$，不含图本身。采用邻接矩阵并遍历整图，每个顶点可能扫描整行，时间 $O(V^2)$，辅助空间仍为 $O(V)$，矩阵本身占 $O(V^2)$。仅从起点遍历且不初始化全局数组时，可以按实际可达顶点与扫描的边计数。

工程上，依赖分析、连通区域搜索和爬虫发现链接都可使用遍历；DFS 深度可能很大，应考虑调用栈容量；BFS 在宽图上可能保存很多待访问顶点。线上图可能变化或有权限限制，还需要一致性、去重与资源上限策略。

分级练习

基础

1. **自编**：本例 A 的 BFS 第 1 层有哪些点？答：B、C。
2. **自编**：BFS 为什么在入队时标记？答：避免不同父顶点重复将同一顶点入队。
3. **自编**：从 A 在本例递归 DFS 的第三个访问顶点是什么？答：D。

进阶

1. **平台题**：LeetCode 1971 寻找图中是否存在路径。邻接表遍历的 O(V+E) 时间包括建表或按题目输入实际说明。
2. **平台题**：LeetCode 200 岛屿数量。把上下左右相邻陆地看作边；m×n 网格时间 O(mn)，访问标记或栈最坏 O(mn)。
3. **自编**：给本例加孤立点 F，从 A 开始 BFS 会访问 F 吗？答：不会；整图遍历外层需再从 F 启动。

综合

1. **自编**：地图只允许经过双向道路，如何用 BFS 输出 A 到 E 的一条最少边数路线？提示：首次入队时记录前驱；本例按字母顺序得到 A→B→E，距离 2。若道路有不同费用，解释为何要改选其他最短路算法。

最小生成树：Prim 与 Kruskal

> 前置：图的表示、并查集。本文讨论有限、连通、无向带权图；顶点数 V≥1，边数 E。权可相等，也可为 0 或负数。

连接所有站点，费用尽量少

要用线缆连接所有站点，边权表示铺设费用。**生成树**覆盖全部顶点、连通且无环，有 V−1 条边；**最小生成树（MST）**是所有生成树中权值总和最小者。它优化总铺设费用，不保证任意两点之间在树上的路径最短。

非连通无向图没有覆盖所有顶点的生成树；可分别求各连通分量的最小生成树，合为最小生成森林。边权互不相同是 MST 唯一的充分条件，但不是必要条件；有相同权值不必然产生多个 MST。

Prim：从顶点集合向外扩张

从一个起点开始，维护已选顶点集合 S；每轮选择一条一端在 S、一端在外部的最小权边，加入边和外部顶点。这个选择使用切分性质：跨某个切分的最小权边可属于某棵 MST；权值相同时需确定一个可复现的并列规则。

```
Prim(G, s):                          // 非空无向图；矩阵版
    S ← {s}；其他顶点的 best ← 从 s 到该顶点的边权或 ∞
    parent[v] ← s 或 null
    重复 V-1 次:
        从 S 外选择 best 最小的顶点 v
        若 best[v] = ∞: 返回“不连通”
        选边 (parent[v], v)；将 v 加入 S
        对 S 外每个顶点 x:
            若存在边 (v,x) 且 w(v,x) < best[x]:
                best[x] ← w(v,x); parent[x] ← v
    返回所选边
```

best 是“从已选集合到某个未选顶点的一条最小边的权”，不是从源点的路径长度。若 V=1，直接返回空边集；边权相等时本伪代码保留先发现的候选。

Kruskal：按边排序并避免环

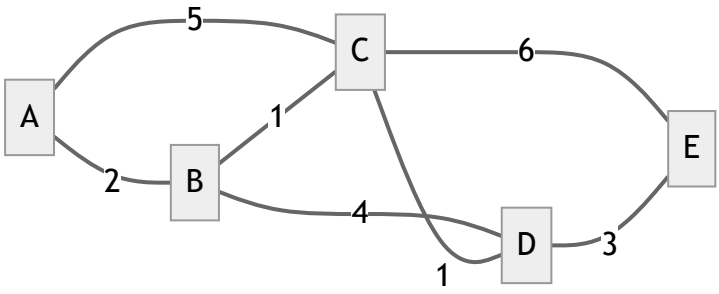
把边按权值非递减排序，依次考虑：若边两端尚未连通则选中，否则跳过。用第三批并查集维护当前连通分量，这里只调用其 find/union，不重复实现其原理。

```
Kruskal(G):
    为每个顶点初始化独立集合
    将边按权值非递减排序
    selected ← 空集合
    对每条边 (u,v,w):
        若 find(u) ≠ find(v):
            selected 加入该边；union(u,v)
            若 selected 的边数 = V-1: 停止
    若 selected 的边数 < V-1: 返回“不连通及当前森林”
    返回 selected
```

若 V=1，可以直接返回空边集。权值相等时按端点名称排序只是示例约定；其他并列顺序可能得到不同的最小生成树。

案例：同一张图对比两种算法

连接五个站点，边权为 AB=2、AC=5、BC=1、BD=4、CD=1、CE=6、DE=3。图为无向图。



Prim 从 A 出发，每一步选取跨越已选和未选顶点的最小边：

步骤	已选顶点 S	本轮选边	已选边权之和	未选顶点的 best（括号内为来源）
初始	A	—	0	B=2(A), C=5(A), D=∞, E=∞
1	A,B	AB=2	2	C=1(B), D=4(B), E=∞
2	A,B,C	BC=1	3	D=1(C), E=6(C)
3	A,B,C,D	CD=1	4	E=3(D)
4	A,B,C,D,E	DE=3	7	无

Kruskal 按权值排序，权值相同时按边名字母顺序；每次比较的是当前已选边形成的连通分量：

考虑边	结果	累计选边	权值和
BC=1	选择	BC	1
CD=1	选择	BC,CD	2
AB=2	选择	BC,CD,AB	4
DE=3	选择，已达 V−1 条	BC,CD,AB,DE	7

两者均得到边集 {AB,BC,CD,DE}，总权 7。Kruskal 在已选够四条边后结束；BD、AC、CE 未再检查。若为教学演示继续检查，这三条都会连接已在同一集合中的顶点，应跳过。Prim 不是全图中任意选最小边，它只能选当前切分上的边。

实现成本与工程取舍

实现	时间	空间及说明
Prim，邻接矩阵与线性选最小	$O(V^2)$	矩阵 $O(V^2)$，best、parent 等额外 O(V)；常用于稠密图
Prim，邻接表与带索引的二叉堆	$O((V+E)\log V)$	图 O(V+E)，堆及数组 O(V)；稀疏图可能合适，具体取决于实现
Kruskal，边集排序与并查集	$O(E \log E + V + E\alpha(V))$	边集 O(E)、并查集 O(V)，排序辅助空间取决于排序算法；边少或边集已给定时方便

边权分布、图是否稠密、排序是否已有序、堆实现和内存限制都会影响实际成本；“Prim 一定比 Kruskal 快”不成立。施工费用还可能受预算、地形或节点可靠性约束，单一边权的 MST 不能直接解决所有工程规划要求。

分级练习

基础

1. **自编**：5 个顶点的生成树有几条边？答：4。
2. **自编**：非连通无向图是否有覆盖全部顶点的生成树？答：没有。
3. **自编**：边权可否为负？答：最小生成树可以使用负权边。

进阶

1. **平台题**：LeetCode 1584 连接所有点的最小费用。题目使用曼哈顿距离构成完全图，讨论矩阵式 Prim 的 $O(V^2)$ 时间。
2. **自编**：本例 Prim 从 A 首先选哪条边？答：AB，权 2。
3. **自编**：Kruskal 已选 BC、CD、AB、DE 后再看 BD，应否加入？答：否，两端已连通。

综合

1. **自编**：将上例 AC 的权值改为 2，重新计算两种算法；说明边权相同时最小树可能不唯一，但不能仅凭存在相同边权就断言一定不唯一。比较铺设总费用和两点最短路的不同目标。提示：选 BC、CD、DE 后，连接 A 可选 AB 或 AC，两种树的总权均为 7。

最短路径：Dijkstra 与 Floyd

> 前置：图的表示、BFS。V 为顶点数、E 为边数；不可达记为 ∞，前驱不存在记为 –。以下权值加法假设不会溢出，且任何与 ∞ 的加法先判断可达性。

先选对问题

路线规划需要最小化从起点到终点的距离；它和最小生成树最小化所有铺设边的总费用不同。若每条边代价相同，BFS 可求最少边数；若边权均非负，可用 Dijkstra 求单源最短路径；Floyd 可计算所有顶点对的最短路径。**标准 Dijkstra 不适用于存在负权边的一般情形**：它可能在含负边的某些特定图上碰巧正确，但没有通用正确性保证。

Dijkstra：逐步确定单源距离

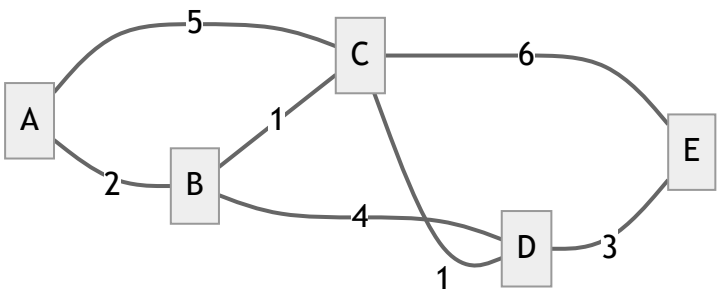
从源点 s 出发，`dist[v]` 是已发现的最短距离上界，`pred[v]` 记录对应路径的前驱。每轮从尚未确定的可达顶点中选 dist 最小者 u，将它确定，再用边 u→v 松弛其他顶点。边权非负保证一旦 u 被选中，后续经过尚未确定顶点的路径不会再缩短它。

```
Dijkstra(G, s):                          // 每条边权均 ≥ 0，s 合法
    dist 全设为 ∞；pred 全设为 −；settled 全设为 false
    dist[s] ← 0
    重复至多 V 次：
        u ← 未确定顶点中 dist 最小者
        若不存在 u 或 dist[u]=∞：结束
        settled[u] ← true
        对 u 的每条出边 (u,v,w)：
            若 v 未确定 且 dist[u]+w < dist[v]：
                dist[v] ← dist[u]+w；pred[v] ← u
    返回 dist，pred
```

若存在并列最短路，用 < 保留先发现的前驱；不同遍历次序可能得到另一条同长路径。未到达顶点距离仍为 ∞，前驱为 —；s 的前驱也为 —。回溯时应从可达终点沿 pred 回到 s 并反转，不可将不可达点当作存在路径。

完整案例：同一张带权图

使用 最小生成树 中的无向图：AB=2、AC=5、BC=1、BD=4、CD=1、CE=6、DE=3，源点 A。无向边在计算时可视为两个方向的弧。图如下：



表中每轮为**选择 u 并完成松弛后**；距离和前驱均按 A、B、C、D、E 顺序列出：

轮次	已确定顶点	dist[A,B,C,D,E]	pred[A,B,C,D,E]	本轮变化
初始	空	[0,∞,∞,∞,∞]	[−, −, −, −, −]	源点距离为 0
选 A	A	[0,2,5,∞,∞]	[−, A, A, −, −]	更新 B、C
选 B	A,B	[0,2,3,6,∞]	[−, A, B, B, −]	C 经 B 变 3，D 经 B 为 6
选 C	A,B,C	[0,2,3,4,9]	[−, A, B, C, C]	D 经 C 变 4，E 经 C 为 9
选 D	A,B,C,D	[0,2,3,4,7]	[−, A, B, C, D]	E 经 D 变 7
选 E	A,B,C,D,E	[0,2,3,4,7]	[−, A, B, C, D]	无改进

从 E 沿前驱回溯 E←D←C←B←A，反转得到 A→B→C→D→E，距离 2+1+1+3=7。本图恰好与所选生成树上的路径一致，但这只是本例现象，MST 不保证最短路。

邻接矩阵、线性选最小并扫描行：O(V²) 时间，图 O(V²) 空间，dist/pred/标记额外 O(V)。邻接表加支持 decrease-key 的索引二叉堆：O((V+E) log V) 时间、图 O(V+E) 及工作 O(V)；若使用可重复入堆的普通堆，堆中可能有 O(E) 条目，界与实现不同。未指定实现时不能混写复杂度。

Floyd：所有顶点对之间的距离

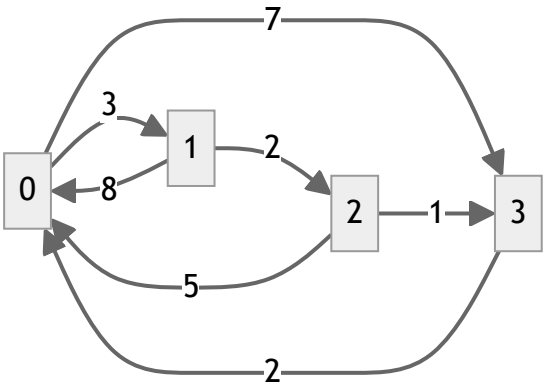
设 D^(k)[i,j] 为只允许编号不大于 k 的顶点作为**中间顶点**时，从 i 到 j 的最短距离。转移为 min(D[i,j], D[i,k]+D[k,j])。初始化直接边权、对角线为 0、无边为 ∞；若存在负权自环，对角线应保留更小的自环权，以便检测。每次只在两段均可达时相加。

```
Floyd(D):                              // D 已按上面的约定初始化
    对 k 从 0 到 V-1:
        对 i 从 0 到 V-1:
            对 j 从 0 到 V-1:
                若 D[i,k] ≠ ∞ 且 D[k,j] ≠ ∞:
                    D[i,j] ← min(D[i,j], D[i,k]+D[k,j])
    返回 D
```

k 必须为最外层，确保本轮允许的中间顶点集合符合递推；路径还原需要另外维护 next 或 predecessor 矩阵。经典矩阵实现时间 O(V³)、距离矩阵空间 O(V²)；若原图以邻接表给出，初始化矩阵仍需 O(V²+E)。

完整案例：逐轮距离矩阵

考虑另一个**有向图**，顶点 0..3，弧 0→1:3、0→3:7、1→0:8、1→2:2、2→0:5、2→3:1、3→0:2。表中行是起点、列是终点；每个矩阵都在该轮**全部 i、j 更新完成后**给出。∞ 代表不可达。



```
初始 D（仅直接边）        k=0 完成后             k=1 完成后

     0  1  2  3             0  1  2  3             0  1  2  3

0 [  0  3  ∞  7 ]     0 [  0  3  ∞  7 ]     0 [  0  3  5  7 ]

1 [  8  0  2  ∞ ]     1 [  8  0  2 15 ]     1 [  8  0  2 15 ]

2 [  5  ∞  0  1 ]     2 [  5  8  0  1 ]     2 [  5  8  0  1 ]

3 [  2  ∞  ∞  0 ]     3 [  2  5  ∞  0 ]     3 [  2  5  7  0 ]


k=2 完成后             k=3 完成后

     0  1  2  3             0  1  2  3

0 [  0  3  5  6 ]     0 [  0  3  5  6 ]

1 [  7  0  2  3 ]     1 [  5  0  2  3 ]

2 [  5  8  0  1 ]     2 [  3  6  0  1 ]

3 [  2  5  7  0 ]     3 [  2  5  7  0 ]
```

例如 k=1 时，0→1→2 将 D[0,2] 从 ∞ 改为 3+2=5；k=2 时 0→1→2→3 将 D[0,3] 从 7 改为 5+1=6；k=3 时 1→2→3→0 将 D[1,0] 从 7 改为 3+2=5。最终 D[0,3]=6。

负权边与负权环

Floyd 可以处理负权边；若不存在相关的负权环，有限最短距离有定义。完成 Floyd 后若 `D[k,k]<0`，说明存在从 k 可到达并回到 k 的负权闭合游走。对于能从 i 到达该负环、且能从负环到达 j 的顶点对，可绕环任意多次使代价无下界，其最短距离不是有限值，应标为负无穷或报告未定义；其他与该负环无关的顶点对仍可能有有限最短路或不可达。仅检测对角线并不会自动把受影响的每一对标为负无穷。

有负权边的单源问题可考虑 Bellman–Ford：对所有边反复松弛，最多 V−1 轮后再检查是否仍有**从源点可达**的边可松弛，从而检测源点可达的负权环；复杂度通常 O(VE)，还需根据输入表示及路径需求计空间。

工程应用与练习

地图路径、网络延迟和服务依赖距离都需要先定义权值：时间是否随时段变化、有向道路是否可通、负权是否有意义。Dijkstra 的静态非负模型不自动处理实时变化；全源 Floyd 适合顶点数较小且需要大量点对查询的情形，代价是立方时间和平方空间。

基础

1. **自编**：无权图求最少边数通常可用什么算法？答：BFS。
2. **自编**：Dijkstra 的不可达顶点距离和前驱如何表示？答：∞ 和 —。
3. **自编**：Floyd 是否允许负权边？答：允许，但相关负权环会使部分点对不存在有限最短距离。

进阶

1. **平台题**：LeetCode 743 网络延迟时间。有向边权非负；从指定源求所有可达距离，并处理不可达节点。
2. **平台题**：LeetCode 1091 二进制矩阵中的最短路径。题目允许八方向，路径长度按单元格数计，不是边数。

3. **自编**：本页 Dijkstra 案例中，B 被确定后 dist[C] 和 pred[C] 是什么？答：3 和 B。

综合

1. **自编**：给出有向边 A→B=2、A→C=5、C→B=−10；说明标准 Dijkstra 为何不能可靠确定 B，并比较使用 Bellman–Ford 的条件。提示：B 可能先被确定为 2，但经 C 的路径为 −5。

拓扑排序与关键路径

> 前置：图的表示、队列。本页示例均为有限有向图；V、E 分别为顶点数、边数。拓扑排序判断先后依赖，关键路径分析有持续时间的工程工期，两者不是同一个问题。

AOV 网：安排任务的先后顺序

AOV（Activity on Vertex）网用顶点表示活动，用有向边表示先后约束：u→v 表示 u 必须先于 v。拓扑序是顶点的一个线性排列，使每条有向边的起点都在终点之前；可能存在多个合法序列。有向图存在拓扑序当且仅当它没有有向环。

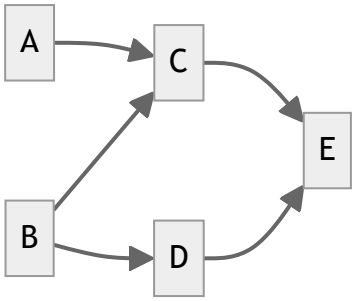
Kahn 算法维护每个顶点当前入度：将所有入度为 0 的顶点放入队列，反复取出并输出，删除其出边（用入度减一模拟）；若邻居入度变 0 则入队。最后输出数小于 V 表示图中有环。空图返回空序列，是合法的拓扑序。

```
拓扑排序(G):
    根据全部弧计算 indegree[0..V-1]
    Q ← 所有入度为 0 的顶点
    order ← 空列表
    当 Q 非空:
        u ← Q.出队(); order.append(u)
        对 u 的每条出边 u→v:
            indegree[v] ← indegree[v]-1
            若 indegree[v]=0: Q.入队(v)
    若 order 长度 < V: 返回“存在环”
    返回 order
```

使用邻接表时，计算入度 O(V+E)，每个顶点和边最多处理一次，总时间 O(V+E)，入度与队列等辅助空间 O(V)，不计图和输出。若需要每次从所有可选顶点中取字母序最小值，可用最小堆，但成本应重新计算；本例仅对初始入队和每次同轮新入队按字母序排列，使用普通 FIFO。

案例：项目任务依赖

任务 A、B 可立即开始；C 依赖 A、B，D 依赖 B，E 依赖 C、D。边为 A→C、B→C、B→D、C→E、D→E。



轮次	出队任务	更新后仍未输出顶点的入度	操作后队列（头→尾）	输出
初始	—	A=0,B=0,C=2,D=1,E=2	A,B	空
1	A	B=0,C=1,D=1,E=2	B	A
2	B	C=0,D=0,E=2	C,D	A,B
3	C	D=0,E=1	D	A,B,C
4	D	E=0	E	A,B,C,D
5	E	无	空	A,B,C,D,E

输出 A、B、C、D、E 是一个合法序列；B、A、D、C、E 也是合法序列。拓扑序只约束前后关系，不给出任务时长或最短工期，也不意味着所有任务必须串行执行。若添加 E→B，就出现 B→C→E→B，算法不能输出全部顶点。

AOE 网：活动时间与关键路径

AOE（Activity on Edge）网用**顶点表示事件**、有向边表示活动，边权为活动持续时间。以下采用单源单汇、所有事件均处于源到汇的某条路径上、持续时间非负的 DAG；若有多个起点或终点，可在建模允许时增加零时长虚拟源、虚拟汇。所有活动按依赖约束进行，假设资源足够、可并行，关键路径长度才给出该模型中的最短完成工期；资源争用、工作日历和延误等需要额外模型。

对事件 v：`ve[v]` 是最早发生时间，源点为 0，按拓扑序 `ve[v]=max(ve[u]+w(u,v))`；`vl[v]` 是不延迟汇点完成的最迟发生时间，汇点初始化为 `ve[汇]`，按逆拓扑序 `vl[u]=min(vl[v]-w(u,v))`。

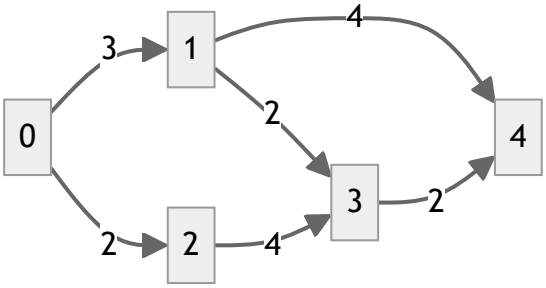
对活动 u→v：最早开始 `e=ve[u]`，最迟开始 `l=vl[v]-w(u,v)`，总时差 `l-e`；时差为 0 的是关键活动。源到汇的最长路径是关键路径，可能有多条；只缩短其中一条路径上的活动，不一定缩短工期。

```
关键路径(G, source, sink):          // 满足上述单源单汇 DAG 前提
    order ← 拓扑排序(G)
    若存在环: 返回“输入不符合 DAG 前提”
    ve 全设 -∞; ve[source] ← 0
    按 order 遍历 u:
        对每条 u→v, 权 w: ve[v] ← max(ve[v], ve[u]+w)
    vl 全设 +∞; vl[sink] ← ve[sink]
    按 order 逆序遍历 u:
        对每条 u→v, 权 w: vl[u] ← min(vl[u], vl[v]-w)
    对每条 u→v, 权 w:
        e ← ve[u]; l ← vl[v]-w
        若 e=l: 标记该边为关键活动
    返回 ve, vl, 所有关键活动
```

除拓扑排序结果外，若不计图和输出，ve、vl 等辅助空间 O(V)；邻接表上总时间 O(V+E)。上面前提排除了无法从源到达或不能到达汇的顶点；若输入不保证这一点，须先检查或重新规定虚拟源汇的建模。

案例：项目工期与关键活动

项目事件编号 0..4，活动及持续时间为 `0→1:3`、`0→2:2`、`1→3:2`、`2→3:4`、`1→4:4`、`3→4:2`，源为 0、汇为 4。边上的数字是时长，不是事件编号；事件 3 需要等待 1→3 和 2→3 均完成。



按拓扑序 0、1、2、3、4 更新 ve：

已处理事件	ve[0,1,2,3,4]	本轮关键更新
初始	[0,−∞,−∞,−∞,−∞]	源点从 0 开始
0	[0,3,2,−∞,−∞]	更新 1、2
1	[0,3,2,5,7]	更新 3、4
2	[0,3,2,6,7]	2→3 得 2+4=6，大于原 5
3	[0,3,2,6,8]	3→4 得 6+2=8

已处理事件	ve[0,1,2,3,4]	本轮关键更新
4	[0,3,2,6,8]	无出边

因此最早工期为 8。逆序计算 vl：

已处理事件	vl[0,1,2,3,4]	计算
初始	[∞,∞,∞,∞,8]	汇点最迟为 8
4	[∞,∞,∞,∞,8]	无出边
3	[∞,∞,∞,6,8]	8−2=6
2	[∞,∞,2,6,8]	6−4=2
1	[∞,4,2,6,8]	min(6−2,8−4)=4
0	[0,4,2,6,8]	min(4−3,2−2)=0

活动	时长	e=ve[弧尾]	l=vl[弧头]−时长	时差	关键?
0→1	3	0	1	1	否
0→2	2	0	0	0	是
1→3	2	3	4	1	否
2→3	4	2	2	0	是
1→4	4	3	4	1	否
3→4	2	6	6	0	是

关键路径是 0→2→3→4，长度 2+4+2=8。例如 0→1 的总时差为 1，并不意味着所有非关键活动可各自同时延误其全部时差而互不影响；多个活动可能共享时差。

工程应用与练习

构建系统可用拓扑顺序安排先决任务，并检测循环依赖；项目排期可在依赖和估计时长已知、资源不受限制时用关键路径识别影响工期的活动。真实项目的资源约束、人工分配、工作日历、时间不确定性和多条关键路径会改变结果，不能把拓扑序本身当作进度计划。

基础

1. **自编**：A→B 时拓扑序能否将 B 放在 A 前？答：不能。
2. **自编**：有向环是否存在拓扑序？答：不存在。
3. **自编**：本例工程的最早完成时间？答：8。

进阶

1. **平台题**：LeetCode 207 课程表，注意 [a,b] 表示 b→a。
2. **平台题**：LeetCode 210 课程表 II，需要返回一个可行顺序；无解返回空数组。
3. **自编**：本 AOE 案例活动 1→3 的最早开始、最迟开始及总时差？答：3、4、1。

综合

1. **平台题**：LeetCode 2050 并行课程 III。题目用顶点表示课程及持续时间，与本页边表示活动的 AOE 模型不同；可按拓扑序更新课程完成时间，并遵守题目可并行的假设。

第 08 章　查找

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

查找基础：顺序、折半与分块

> 前置：复杂度与概率口径、顺序表。本文默认键互不相同，数组下标从 0 开始；每次比较完整关键字的成本视为 O(1)。

查找与平均查找长度

图书管理员需要根据编号找到一本书。查找以关键字为条件定位记录：找到称成功，确认不存在称失败。**平均查找长度 ASL** 是在指定查找情形的概率分布下，查找代价的期望。若第 i 个成功键被查询的条件概率为 p_i、需 C_i 次关键字比较，则 $ASL_{成功}=\sum p_i C_i$，且 $\sum p_i=1$。失败情形按其概率 q_j 和代价 C'_j 加权；成功和失败的条件概率须分别说明。整体期望还要用成功、失败发生概率加权，不能把两个 ASL 直接平均。

“比较次数”要声明是否计入哨兵、空槽检查或边界判断；与记录的键比较、访问槽位和执行机器指令不是同一口径。各键、各失败区间**不必然等概率**，以下算例只在明确声明下取等概率。

顺序查找与哨兵

无序记录 `[12,7,19,5]` 按从左至右比较，查 19：12、7 不等，第三次比较 19 命中。若四个成功键等概率，成功比较次数分别为 1、2、3、4，ASL=(1+2+3+4)/4=2.5；失败必与 4 个真实记录比较，所以只计真实键比较的失败 ASL=4。

哨兵写法把待查键临时放在数组 0 号槽，真实记录放在 1..n，从后向前查找。哨兵确保循环一定遇到相等值，省去循环中的下标越界判断，但不免费减少所有比较。

```
带哨兵顺序查找(A, n, x):       // A[1..n] 为真实记录；A[0] 专供哨兵
    A[0] ← x
    i ← n
    当 A[i] ≠ x: i ← i-1
    若 i = 0: 返回“未找到”
    返回 i                      // 返回 1 基位序
```

例：A[1..4]=`[12,7,19,5]`，查 7 时依次比较 5、19、7，返回位序 2。查不存在的 8 时依次比较 5、19、7、12 和哨兵 8：若**连哨兵键比较也计入**共 5 次；若只计真实记录的键比较则是 4 次。这不是两种不同数据的平均概率，而是两种统计对象。n=0 时仅比较哨兵一次并返回未找到；A[0] 会被覆盖，不可指向只读内存或有效记录。

折半查找与判定树

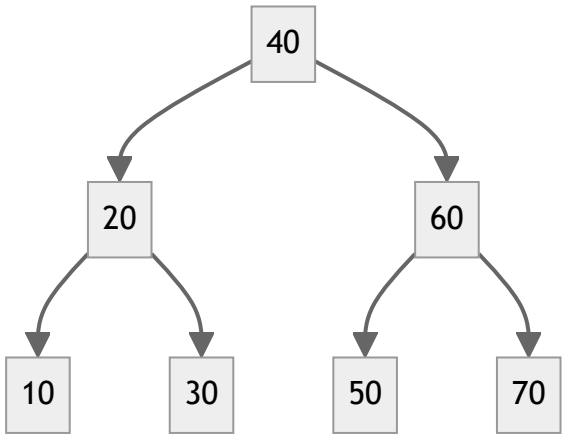
升序顺序表能 O(1) 随机访问中点。闭区间版维持不变量：若目标存在，必在 `A[low..high]`；每轮将中点及不可能的一半排除。

```
折半查找(A, x):                // A 升序，0 基；可为空
    low ← 0; high ← |A|-1
    当 low ≤ high:
        mid ← low + floor((high-low)/2)
        若 A[mid] = x: 返回 mid
        若 A[mid] < x: low ← mid+1
        否则: high ← mid-1
    返回 -1
```

数组 `[10,20,30,40,50,60,70]` 查找 50：

轮次	low	high	mid	比较	下一步
1	0	6	3	A[3]=40 < 50	low=4
2	4	6	5	A[5]=60 > 50	high=4
3	4	4	4	A[4]=50	返回下标 4

将每次可能比较的中点画为判定树：



七个键各等概率成功，比较次数按结点层数计：ASL成功=`(1+2×2+4×3)/7=17/7`。若八个失败间隙也**各等概率**，并且只数真实键比较，本例每个失败都比较 3 次，ASL失败=3；实际查找可能有不同概率。若把最后的空区间判断也计作一次额外操作，将得到另一种代价，必须标明，而不能称为同一口径。

每轮保留至多约一半元素，迭代时间 O(log n)，辅助空间 O(1)；n=0 时初始 high=-1 并直接返回 -1。重复键时该算法返回某个匹配位置，不保证第一个或最后一个；需重新设计边界。对链表，折半比较次数可为 O(log n)，但链路遍历成本不具备数组的随机访问优势：某些迭代器实现可用 O(n) 次链接遍历，不应一概断言 O(n log n)。Oracle Collections 文档列出了这类实现的区分。

分块查找：索引与块内扫描

将 n=9 条记录分成 3 个等长块，**块间有序、块内可无序**：

```
块 1 [3,1,2]    块 2 [6,4,5]    块 3 [9,7,8]

索引最大键   3             6             9
```

查 5：顺序检查索引，5≤3 为假、5≤6 为真，定位第 2 块；依次比较 6、4、5 后命中。计索引键比较 2 次、块内键比较 3 次，共 5 次；若某题将两种比较赋予不同代价，不能直接相加。

设 b 块、每块 s 个且 n=bs，索引顺序扫描、块内顺序扫描、每个记录成功查找概率相同，且按上例将一次索引比较和一次块内比较各计一次，则 ASL成功=(b+1)/2+(s+1)/2。例中为 2+2=4。将 b=n/s 代入，连续近似在 s=√n 处最小；实际整数块数、最后一块不满、非等概率或索引改为二分时结论会改变。索引表额外 O(b) 空间；顺序索引最坏 O(b+s)，折半索引加块内顺序扫描为 O(log b+s)。失败情形须根据可能查询值与概率单独计算。

工程选型

无序小列表或链表可顺序查找，不要求预排序；有序数组适合静态或少更新的随机访问，但维护有序性和插入可能 O(n)。块索引适合块间有序、块内需要局部扫描的场景；索引本身也有内存与更新成本。动态有序查找的 BST、AVL 和红黑树已在 BST 与 AVL 主讲，这里不重复实现。

分级练习

基础

1. **自编**：无序表 5 个不同键等概率成功查找，顺序扫描成功 ASL？答：3 次真实键比较。
2. **自编**：空数组闭区间折半查找的初始 low、high？答：0、-1，循环不进入。
3. **自编**：哨兵查找失败，n=4、计入哨兵比较为几次？答：5 次。

进阶

1. **平台题**：LeetCode 704 二分查找。按平台有序且无重复的题设查找，时间 O(log n)、辅助空间 O(1)。
2. **平台题**：LeetCode 35 搜索插入位置。确定边界后返回应插位置；不能只在找到相等时结束。
3. **自编**：本页分块例查找 5，索引比较和块内比较各几次？答：2 次、3 次。

综合

1. **平台题**：LeetCode 34 在排序数组中查找元素的第一个和最后一个位置。分别求左、右边界；题目允许空数组和重复值，时间 O(log n)、辅助空间 O(1)。

B 树与 B+ 树

> 前置：查找与 ASL、BST 与 AVL。本页键互不相同；同名结构存在多种“阶”和分隔键定义，下述示例明确采用的约定。n 为真实记录或键数，m 为最大分支数。

为什么磁盘索引需要多路结构

订单系统经常需要按编号查单条记录，也需要查询一个编号范围。若数据大到不能全部驻留内存，一次磁盘或存储页访问的代价可能远大于一次内存比较。多路树让一个节点容纳多个有序键，以较少层级定位目标；节点更新、页分裂和并发控制也有成本。

B 树：每层可保存真实键

本节 m=4 的 B 树约定：每个节点最多 4 个孩子、3 个键；非根节点至少 1 个键（内部节点至少 2 个孩子）；有 c 个孩子的内部节点有 c−1 个有序键；所有真实叶节点同层。根若非叶至少两个孩子；空树单独表示。键可出现在内部或叶节点。教材有时把空孩子视为“不含键的失败叶”，本页将**含键的终端节点**称真实叶节点，避免混淆。

查找每层先在节点内比较键：相等则结束，否则进入相应的键区间子树。插入首先进入真实叶节点；若插入使节点超出 3 个键，就分裂并将分隔键**上移**到父节点，必要时继续向上传播。本例在临时出现 4 个键时选第 2 个键上移。删除内部键可先用前驱或后继替换，再到真实叶节点删除；非根节点下溢时先视兄弟是否有余量借位，否则与兄弟及父分隔键合并；根空时若有唯一孩子，将其提升为新根。

案例：查找、分裂、合并与借位

依次插入 10,20,30,40,50,60。中括号表示一个节点，斜线为子树，空树插入首键直接形成根：

```
插 10,20,30:    [10 20 30]

插 40:          [10 20 30 40]  暂时溢出；20 上移

                     [20]

                    /    \

                  [10]  [30 40]

插 50: 右叶变成 [30 40 50]

插 60: 右叶暂为 [30 40 50 60]；40 上移

                  [20 | 40]

                 /     |     \

              [10]   [30]  [50 60]
```

查 60：根节点先比较 20、40，进入大于 40 的右叶，再比较 50、60，命中。若节点内采用顺序比较，这条路径一共 4 次键比较、访问 2 个节点；换成节点内折半比较或缓存节点，比较次数和页 I/O 口径会变。

从上图**依次**删除 10、30、20：

```
删 10：左叶空；相邻兄弟 [30] 只有最低 1 键，不能借。

       左叶 + 父键 20 + 兄弟 [30] 合并，父节点移除 20：

                  [40]

                 /    \

              [20 30] [50 60]

删 30：左叶保留 [20]，仍满足下限。

删 20：左叶空；右兄弟 [50 60] 有 2 键，可借。

       父键 40 下移左叶，右兄弟最小键 50 上移为新父键：

                  [50]

                 /    \

              [40]   [60]
```

每次分裂或删除修复完成后的节点均满足本例规则；图中标明的临时溢出状态是修复过程，不是最终合法状态。删除时如果父节点继续低于下限，要继续向上修复；本例没有发生父节点继续下溢。不存在的键应报告未找到而不改变树；不同合法分裂位置可能形成不同但仍有效的树。

```
B树查找(node, key):

    当 node ≠ null:

        在 node 的有序键中找第一个 ≥ key 的位置 i

        若位置 i 存在 且 node.key[i] = key: 返回命中记录

        若 node 是叶节点：返回“未找到”

        node ← 对应区间的第 i 个孩子

    返回“未找到”
```

该伪代码默认孩子位置和键均从 0 开始；若所有键都小于目标，i 等于键数，选择最后一个孩子。不要在节点键不存在时访问 key[i]。

B+ 树：内部索引与有序叶链

本节采用一种常见的**右子树最小键作为分隔符**的四阶教学变体：内部节点最多 4 个孩子，c 个孩子有 c−1 个分隔键；非根内部节点至少 2 个孩子。叶节点最多 3 条记录，非根叶至少 2 条，真实数据记录只在叶节点，所有叶节点在同一层并按键用 next 相连。根有特殊规则：根为叶时可只有 1 条记录，空树另表示；根为内部节点至少 2 个孩子。别的教材可能让内部节点的键数等于孩子数或采用不同叶容量，不可把不同变体的分裂和下限公式混用。

例如根分隔键为 `30,40`：小于 30 走第一个孩子，30≤key<40 走第二个，key≥40 走第三个；等于分隔键也要继续向右子树的叶节点寻找。查找命中内部键时不能提前当作找到记录。

案例：从插入到分裂、合并、借位

依次插入 10,20,30,40,25,35,50；叶节点满后插入使 4 条暂时溢出，按 2+2 分裂，将右叶最小键**复制**到父节点，原记录仍留在叶中。

```
插 10,20,30: 叶根 [10 20 30]

插 40 后叶溢出 [10 20 30 40], 分为 [10 20] 和 [30 40]:

                    <30>

                   /    \

              [10 20] → [30 40]

插 25: 左叶 [10 20 25]; 插 35: 右叶 [30 35 40]

插 50: 右叶 [30 35 40 50] 分裂, 复制 40 到根:

                    <30 | 40>

                  /      |       \

       [10 20 25] → [30 35] → [40 50]
```

例如查 35：根比较分隔键，30≤35<40，进入中间叶 [30 35] 后命中。范围查询 [25,45]：先从根定位 25 所在叶，依次读到 25，沿叶链读 30、35、40；遇到大于 45 的 50 时停止，结果为 25、30、35、40。

继续从上图**依次**删除 10、20、50：

```
删 10: 左叶 [20 25], 仍有 2 条, 无需修复。

删 20: 左叶变 [25], 低于 2 条; 右邻叶 [30 35] 恰好为下限, 不能借。

       两叶合并为 [25 30 35], 从根删除分隔符 30:

                        <40>

                       /    \

             [25 30 35]  →  [40 50]

删 50: 右叶变 [40], 低于 2 条; 左叶有 3 条, 可借出最大键 35。

       移动 35 到右叶, 并把父分隔键更新为右叶最小键 35:

                        <35>

                       /    \

               [25 30]  →  [35 40]
```

插入叶分裂时是复制右叶最小键到父节点；删除时分隔键变化必须沿必要的祖先路径维护。若父节点因此下溢，还需要按内部节点规则借位或合并，必要时收缩根；本例未触发。叶链也必须在分裂、合并后更新，范围扫描才能不漏记录。

B 树与 B+ 树的选择与成本

维度	B 树	本页 B+ 树变体
数据键	内部和叶节点均可能保存真实键及记录关联	内部为分隔索引，真实记录位于叶节点
相等查找	可在内部节点命中结束	必须继续至叶节点
范围查询	可通过有序遍历实现	先定位起始叶，再沿叶链扫描，顺序访问方便
内部节点	c 个孩子、c−1 个真实键	c 个孩子、c−1 个分隔键；其他约定也存在

树高为 O(log_m n) 的节点访问量只在节点装载度受约束、m 等参数明确时使用；若把 m 当可变参数，节点内线性比较可能需 O(m) 每层，节点内折半比较为 O(log m)；插入和删除在节点内移动或分裂可能涉及 O(m) 数据，不能忽略。磁盘页访问次数还受缓存命中、页布局和实现影响，不能简单等同于比较次数。

工程中 B+ 树式索引适合大量数据的点查与范围扫描，内部页只用于导航、叶页可顺序访问；但叶链和分隔键要维护，写入可能触发分裂，索引还占空间。B 树**也支持**范围查询，不能说完全不支持。具体数据库索引命名和实现会不同：例如 PostgreSQL 官方 B-tree 文档 描述内部页指向下一层、叶页记录指向表行以及页之间链接；其产品实现细节不能完全等同于本页的教学约定。

分级练习

基础

1. **自编**：本页四阶 B 树一个节点最多几个键？答：3。
2. **自编**：本页 B+ 树查找命中内部 30 后是否可以直接返回记录？答：不能，进入对应右子树叶节点。
3. **自编**：本页非根 B+ 叶节点最少几条记录？答：2。

进阶

1. **自编**：本页 B 树初始 `[20|40]` 的右叶 `[50,60]` 查 60，节点访问几次？答：2 次；键比较数取决于节点内搜索方法。
2. **自编**：本页 B+ 树插入 40 后，为什么 30 同时出现在父节点和右叶？答：父节点复制分隔键，叶中保留真实记录。
3. **自编**：上述 B+ 删除 50 后为什么父分隔键从 40 改为 35？答：借位后右叶最小键为 35。

综合

1. **自编**：订单表有大量等值查询和编号范围扫描，比较散列表与本页 B+ 树；讨论页缓存、叶链、索引维护和写入分裂。不要声称散列绝不可以通过额外排序支持范围查询，也不要假定数据库必然选择某个索引。

散列表：用关键字直接定位

一个用户资料缓存每天要按用户编号进行大量查询。如果每次都从头比较，数据量增长后平均查找次数也会增长。散列表先把关键字映射为数组位置，再处理不同关键字落在同一位置的情况。映射可以很快，但它通常不能保持关键字大小顺序，也不能保证每次操作都快。

基本概念与散列函数

散列函数 `h(key)` 将关键字映射到表下标；表长为 `m` 时，下标必须在 `0..m-1`。不同关键字得到同一下标称为**冲突**，发生冲突的关键字互为同义词。冲突无法仅凭函数彻底避免：当可能的关键字多于槽位时，必有不同关键字映射到同一位置。散列函数应计算方便，并结合实际数据尽量使结果分布均匀；例如整数可取 `h(k)=k mod m`，负数须按所用语言规则归一化到有效下标。字符串可由各字符逐步计算散列值，但不同字符串仍可能有相同值，因此最终必须比较原关键字。

装填因子 α = n/m，其中 n 为当前有效记录数、m 为槽位或桶数。开放定址法每槽至多存一条，α≤1；拉链法每桶可存多条，α 可以大于 1。α 影响冲突，但它不是唯一因素：散列函数、键分布及冲突处理方式也会影响性能。

> 原资料中“表长以下最大素数使冲突最少”不能作为普遍结论。例如表长 16、关键字 13、26、39：取模 13 时三者都落到 0；取模 11 时分别落到 2、4、6。实际分布要结合关键字与散列函数评估。

开放定址：线性探测

冲突时按探测序列寻找同一数组中的其他槽位。线性探测依次访问 `(h(k)+i) mod m`，i=0,1,…,m−1，最多检查 m 个槽；必须回绕，不能无限循环。查找遇到相同键即成功；遇到**从未使用过的空槽**即可失败，扫描完 m 槽仍未找到也失败。删除不能简单地把槽改成从未使用，因为这可能截断后续键的查找路径，应使用“已删除”标记（墓碑）。

```
查找(T, key):
    对 i 从 0 到 m-1:
        p ← (h(key)+i) mod m
        若 T[p] 是从未使用的空槽：返回 未找到
        若 T[p] 是有效记录 且 T[p].key = key：返回 p
        // 已删除的槽不能终止查找
    返回 未找到
```

插入时须先确认键不存在：记录遇到的第一个墓碑，继续查找，直至遇到原始空槽、已有相同键或完成一圈；若已存在则按字典语义更新；否则优先使用记录的墓碑，其次使用原始空槽。整圈无空槽和墓碑时表满，需扩容或报告失败。墓碑过多会延长探测，可重建表清理；扩容后必须按新表长重新散列，不能只复制数组下标。

完整案例：冲突、回绕与删除

表长 m=7，h(k)=k mod 7；依次插入 10、17、24、9、31。表初始全为空；槽位从 0 开始。

插入键	起始位置	依次探测的槽	放入位置	槽访问次数
10	3	3	3	1
17	3	3→4	4	2
24	3	3→4→5	5	3
9	2	2	2	1
31	3	3→4→5→6	6	4

最终表为：

```
下标   0   1   2   3   4   5   6

内容   ∅   ∅   9  10  17  24  31
```

例如查找 31，依次检查 3、4、5、6，共 4 次。删除 17 后槽 4 标为“已删除”，查 24 时仍依次经过槽 3、4、5，不能在槽 4 停止。若之后插入 38，起点为 3，先记住墓碑 4，继续检查 5、6、0，遇到从未使用的槽 0 且确认不存在后，才将 38 放入槽 4。这一过程也展示了从 6 回绕到 0。

ASL 必须先说清查找分布和比较口径。在上面未删除的静态表中，5 个现有键被等概率查找，按与真实键的相等比较次数计，成功 ASL = (1+2+3+1+4)/5 = 11/5。若失败查找的起始槽 0..6 等概率，且从未使用过的空槽检查也计一次槽访问，则对应访问次数分别为 1、1、6、5、4、3、2，失败平均槽访问次数 = 22/7；若只算与真实键的比较，则为 15/7。这一失败结果依赖题设的起始槽分布，不代表所有输入分布，也不适用于改动后有墓碑的表。一般 ASL 定义见算法基础与复杂度。

线性探测容易产生连续占用的**聚集**，使后续探测变长。二次探测可用平方偏移，双重散列用另一个散列值作为步长；但不恰当的参数可能不能访问所有槽。比如双重散列要使步长与表长互素，才能在一圈内覆盖全部槽。具体实现仍须限制探测次数。

拉链法

每个桶保存一个链表或其他容器；同义词进入同一个桶。仍取 m=7、h(k)=k mod 7，使用上例关键字并按插入顺序追加：

```
桶 2 → 9

桶 3 → 10 → 17 → 24 → 31

其余桶 → 空
```

查 24 时先算出桶 3，再依次比较 10、17、24，共 3 次关键字比较；若查到链尾仍未匹配则失败。删除时从桶中移除节点即可，不需要墓碑。若桶内记录等概率被查找，上表成功 ASL=(1+1+2+3+4)/5=11/5；这一数值也取决于链内顺序及查找概率，不能据此断言两种方法总是一样快。链结构及边界操作见单链表。

方案	主要收益	局限
开放定址	数组连续，避免每条记录单独分配链节点	聚集、墓碑和高装填因子可能增加探测；需要空槽或扩容
拉链法	删除方便，记录数可超过桶数	额外指针或容器开销；链长不均时退化

假设散列计算与键比较为 O(1)，装填因子受控且键均匀分布时，两者查找、插入、删除通常具有期望 O(1) 成本；碰撞严重时单次操作最坏 O(n)，不应把期望当作最坏保证。表本身占 O(m+n) 空间；开放定址有效记录存入 m 个槽，额外槽表为 O(m)，拉链法桶和节点合计 O(m+n)。字符串散列或比较还需计入键长成本，扩容的单次重散列通常为 O(n+m)；均摊结论需要给出具体扩容策略。

工程应用：缓存与键值查询

缓存通过哈希表由键定位条目，避免每次遍历所有缓存项；需要最近最少使用淘汰时，还可与双向链表配合，使定位和调整顺序各司其职。收益取决于较均匀的散列、适当的容量管理和键的比较成本；碰撞、扩容停顿、内存开销以及并发同步都可能影响实际延迟。若需求是按键排序或区间扫描，应考虑有序索引（见 B 树与 B+ 树），普通哈希表不提供自然的有序遍历。

分级练习

以下自编题为练习设计，不标为历年真题；平台链接为官方题面。

基础

1. **自编**：m=7，h(k)=k mod 7，解释 10 与 17 为何冲突，并计算 5 条记录的装填因子。
2. **自编**：为什么开放定址删除必须区分从未使用和已删除？用上面的 17、24 说明。
3. **自编**：比较拉链法与线性探测在存储位置、删除处理和 α 上的区别。

进阶

4. **自编**：依次插入 6、13、20、7 到长度 7 的线性探测表，写出各次探测和最终表；假设等概率查找已插入键，计算成功 ASL 并说明比较口径。
5. **平台题（已核实）**：LeetCode 706 设计哈希映射：考虑更新已有键、删除、冲突和空表。思路：使用桶与链，按键比较后更新或删除；均匀散列且负载受控时期望 O(1)，碰撞最坏 O(n)，空间 O(m+n)。
6. **平台题（已核实）**：LeetCode 1 两数之和：扫描时查找目标与当前值之差，并保存已访问元素的位置；先查再存可避免重复使用同一元素。期望 O(n) 时间、O(n) 空间，哈希操作的最坏情况取决于实现与输入。

综合

7. **自编**：为一个缓存设计键到值的查询和容量限制方案。比较开放定址、拉链法，并说明扩容、删除、键分布及需要按顺序淘汰时的额外结构；分别分析期望成本和碰撞严重时的最坏成本。

第 09 章　内部排序

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

排序基础与插入排序

仓库要按商品价格从低到高排列记录，同价商品原有的上架先后次序是否保留，会影响顾客看到的结果。排序不仅要得到升序，还要考虑稳定性、内存及更新成本。

排序评价标准

对记录按关键字排列；若任意两个关键字相等的记录，排序前后的相对次序都不改变，排序算法才是**稳定**的。例如输入 `[2甲,1,2乙]`，稳定升序必须让 2甲仍在 2乙前。单次输出恰好未改变次序不能证明算法稳定；一个合法反例足以证明不稳定。

内部排序指待排序数据可在内存中处理；外部排序的数据不能全部装入可用内存，需协调外存读写。额外空间不含输入本身，但必须说明是否计递归栈、输出数组；原地通常指 O(1) 辅助空间，并非“不修改输入”。复杂度的基本定义见算法基础。

以下各内部排序的共同输入为 `[4,2,5,1,3]`，数组和区间均为 0 基。只按数字大小比较；实际带记录时需一起移动整条记录。

直接插入排序

像整理手牌：先把 `A[0..i-1]` 看作已排好，把 A[i] 保存为待插记录，从右向左移动严格大于它的记录，再放入空位。相等时不移动，因而稳定。

```
直接插入(A):
    对 i 从 1 到 n-1:
        x ← A[i]; j ← i-1
        当 j ≥ 0 且 A[j] > x:
            A[j+1] ← A[j]; j ← j-1
        A[j+1] ← x
```

i	待插入	已排序区间（插入前）	一趟后整个数组
1	2	[4]	[2,4,5,1,3]
2	5	[2,4]	[2,4,5,1,3]
3	1	[2,4,5]	[1,2,4,5,3]
4	3	[1,2,4,5]	[1,2,3,4,5]

本例关键字比较次数依次为 1、1、3、3，共 8；只数 `A[j]>x`，不数 j 的边界检查。移动旧记录次数依次 1、0、3、2，共 6；暂存和最终写入不计为旧记录移动。空数组与单元素不进入循环。已升序时比较 n−1 次、时间 Θ(n)；逆序时比较和移动均 n(n−1)/2、时间 Θ(n²)；在互异记录的所有排列等概率下平均 Θ(n²)。辅助空间 O(1)。

折半插入排序

已排序前缀允许二分定位，但移动元素仍须逐个进行。寻找第一个**大于** x 的位置（即插在已有相等记录之后），以保留稳定性。

```
折半插入(A):
    对 i 从 1 到 n-1:
        x ← A[i]; lo ← 0; hi ← i       // 在 [lo,hi) 找第一个 > x
        当 lo < hi:
            mid ← lo + floor((hi-lo)/2)
            若 A[mid] ≤ x: lo ← mid+1
            否则: hi ← mid
        对 j 从 i-1 递减到 lo: A[j+1] ← A[j]
        A[lo] ← x
```

共同输入依次插入后的数组与上表相同；例如 i=4、前缀 `[1,2,4,5]`，x=3，检查 mid=2 的 4，再检查 mid=1 的 2，得到位置 2，把 5、4 向右移动后写入 3。每趟二分比较 O(log i)，总比较 Θ(n log n)，但最好时间仍为 Θ(n log n)、最坏和等概率排列平均时间 Θ(n²)，因为移动量可能二次；辅助 O(1)。即使已排好，本版也仍执行二分，不能套用直接插入的最好 Θ(n)。

希尔排序

选择递减间隔，对每个间隔形成的子序列分别做插入排序；最后间隔必须为 1 才保证完全有序。下面使用间隔 2、1。j 每次减少 gap，故各组不会混合。

```
希尔(A, gaps):                 // 正整数 gaps 最后一项为 1
    对每个 gap ∈ gaps:
        对 i 从 gap 到 n-1:
            x ← A[i]; j ← i
            当 j ≥ gap 且 A[j-gap] > x:
                A[j] ← A[j-gap]; j ← j-gap
            A[j] ← x
```

共同输入，gap=2 时下标组为 (0,2,4) 和 (1,3)，分别由 `[4,5,3]`、`[2,1]` 排成 `[3,4,5]`、`[1,2]`，交织结果 `[3,1,4,2,5]`；gap=1 对整个序列插入，得到 `[1,2,3,4,5]`。跨组移动可颠倒相等记录：`[2甲,2乙,1]` 在 gap=2 时变为 `[1,2乙,2甲]`，因此不稳定。

辅助空间 O(1)；时间取决于增量序列和输入，不存在适用于所有序列的单一 Θ(n log n) 保证。以上每趟为间隔插入，一趟最坏 O(n²)，两趟示例亦至多 O(n²)；实际随 n 增长的增量策略须单独分析。

工程应用

小规模或几乎有序的事件列表适合直接插入：移动量与逆序对数量有关，且实现简洁、稳定；列表很大且乱序时，平方移动成本可能不可接受。折半插入可减少昂贵的关键字比较，但不能消除数组搬移；若比较成本很低、移动昂贵，收益可能有限。希尔排序无需递归栈或大辅助数组，但稳定性和复杂度保证不如稳定归并；应按内存、延迟与排序键分布选择。

分级练习

基础

1. **自编**：`[2甲,1,2乙]` 稳定排序后两条 2 的顺序如何？答：甲在乙前。
2. **自编**：直接插入已升序的 n 个键，关键字比较多少次？答：n−1（n≥1）。
3. **自编**：折半插入要把新记录放在相等键之前还是之后才能稳定？答：之后。

进阶

1. **自编**：对共同输入手算 gap=2 和 gap=1 两趟希尔排序。
2. **自编**：解释为什么折半定位不把最坏移动次数降至 O(n log n)。
3. **自编**：用 `[2甲,2乙,1]` 验证希尔排序的不稳定性。

综合

1. **自编**：比较几乎有序且关键字比较昂贵的列表采用直接插入、折半插入或归并的成本；说明额外空间、稳定性和移动次数。

交换排序与选择排序

前置：排序基础与稳定性。本页统一对 `[4,2,5,1,3]` 升序，数组 0 基；表中“比较”若出现，只统计关键字之间的比较，不统计循环判断。

冒泡排序

从左向右比较相邻记录，仅当左键严格大于右键才交换；每趟最大者到达当前区间右端。无交换则可提前结束；相等时不交换，因此稳定。

```
冒泡(A):
    对 end 从 n-1 递减到 1:
        changed ← false
        对 j 从 0 到 end-1:
            若 A[j] > A[j+1]: 交换 A[j],A[j+1]; changed ← true
        若 changed = false: 结束
```

趟	本趟区间	结束状态	比较	交换
1	0..4	[2,4,1,3,5]	4	3
2	0..3	[2,1,3,4,5]	3	2
3	0..2	[1,2,3,4,5]	2	1
4	0..1	[1,2,3,4,5]	1	0，提前结束

共 10 次比较、6 次交换；第 3 趟虽已有序，但程序只有到第 4 趟无交换时才知道。n≤1 无循环。带提前终止时最好 Θ(n)，互异键排列等概率平均及逆序最坏 Θ(n²)，空间 O(1)。例如 `[2甲,2乙,1]` 排成 `[1,2甲,2乙]`，相等记录从未互换。

快速排序

取区间末尾为枢轴，Lomuto 分区：已处理部分中左侧 ≤ `pivot`，其右侧 > `pivot`；扫描完后把枢轴放在两部分之间。这里 `i` 指下一个小于等于枢轴记录的目的位置。相等元素可能跨过彼此，故不稳定。

```
快排(A, lo, hi):                // 闭区间；lo ≥ hi 返回
    若 lo ≥ hi: 返回
    pivot ← A[hi]; i ← lo
    对 j 从 lo 到 hi-1:
        若 A[j] ≤ pivot:
            交换 A[i],A[j]; i ← i+1
    交换 A[i],A[hi]; p ← i
    快排(A,lo,p-1); 快排(A,p+1,hi)
```

共同输入，以下按上述递归先左后右：

区间	枢轴	分区过程（交换下标；含自交换）	分区后	继续处理
0..4	3	j=1 交换(0,1)→[2,4,5,1,3]；j=3 交换(1,3)→[2,1,5,4,3]；枢轴交换(2,4)	[2,1,3,4,5]	0..1、3..4
0..1	1	j=0 不交换；枢轴交换(0,1)	[1,2,3,4,5]	0..-1、1..1
3..4	5	j=3 交换(3,3)；枢轴交换(4,4)	[1,2,3,4,5]	3..3、5..4

关键字比较分别 4、1、1，共 6；交换语句分别执行 3、1、2 次，共 6 次，其中 2 次为自交换，若只计不同位置交换则为 4。空区间和单元素不分区。相等键反例：`[2甲,2乙,1]` 以末尾 1 分区后为 `[1,2乙,2甲]`。

平衡划分最好 Θ(n log n)，在互异键的均匀随机排列或相应随机枢轴模型下期望 Θ(n log n)；每次划分极不均衡时最坏 Θ(n²)。递归辅助栈最好/平衡 O(log n)、最坏 O(n)；只说分区原地 O(1) 不能忽略栈。本版将全部相等元素划到左侧，可能 Θ(n²)；三路分区是另一实现。

简单选择排序

每趟从未排序后缀找第一个最小键，与后缀首元素交换。相等时不更新最小位置；远距离交换仍可能颠倒相等记录。

```
选择(A):
    对 i 从 0 到 n-2:
        p ← i
        对 j 从 i+1 到 n-1:
            若 A[j] < A[p]: p ← j
        若 p ≠ i: 交换 A[i],A[p]
```

i	最小值下标 p	一趟后
0	3	[1,2,5,4,3]
1	1	[1,2,5,4,3]
2	4	[1,2,3,4,5]
3	3	[1,2,3,4,5]

比较 4+3+2+1=10，实际不同位置交换 2 次。最好、平均、最坏均 $\Theta(n^2)$，辅助 O(1)；`[2甲,2乙,1]` 首趟后 `[1,2乙,2甲]`，故不稳定。

堆排序

构造最大堆（父键不小于孩子），每轮把堆顶与未排序区末尾交换，缩小堆后向下调整。堆以数组表示，0 基节点 i 的孩子为 2i+1、2i+2。下列 end 是**开区间**终点。

```
下沉(A, i, end):
    当 2i+1 < end:
        c ← 2i+1
        若 c+1 < end 且 A[c+1] > A[c]: c ← c+1
        若 A[i] ≥ A[c]: 返回
        交换 A[i],A[c]; i ← c
堆排序(A):
    对 i 从 floor(n/2)-1 递减到 0: 下沉(A,i,n)
    对 end 从 n-1 递减到 1:
        交换 A[0],A[end]
        下沉(A,0,end)
```

建堆依次下沉 i=1 和 i=0：`[4,3,5,1,2]` → `[5,3,4,1,2]`。

待放置位置	交换堆顶后	下沉完成后（右侧为已排序区）
4	[2,3,4,1,5]	[4,3,2,1 | 5]

待放置位置	交换堆顶后	下沉完成后（右侧为已排序区）
3	[1,3,2,4,5]	[3,1,2 | 4,5]
2	[2,1,3,4,5]	[2,1 | 3,4,5]
1	[1,2,3,4,5]	[1 | 2,3,4,5]

自底向上建堆 Θ(n)；此版本对全相等键的最好情况为 Θ(n)，互异键均匀随机排列下平均 Θ(n log n)，最坏 Θ(n log n)，辅助 O(1)。不稳定反例 `[2甲,2乙,1]`：建堆不换，第一次堆顶与末尾交换后 `[1,2乙,2甲]`，下沉得 `[2乙,1,2甲]`，最终 `[1,2乙,2甲]`。堆的结构另见树模块。

工程选择

冒泡易观察相邻逆序，但大量数据通常不划算；快速排序可在数组上取得较好缓存局部性，但枢轴与重复键影响最坏情况。堆排序能给出 O(n log n) 最坏时间和 O(1) 辅助空间，却不稳定、访问位置跳跃；若需要稳定排序，可考虑归并排序。

分级练习

基础

1. **自编**：为什么严格大于才交换的冒泡稳定？答：相等记录不会互换。
2. **自编**：数组 0 基下，节点 2 的两个孩子下标？答：5、6，存在与否取决于长度。
3. **自编**：对 5 个元素做简单选择，比较次数？答：10。

进阶

1. **自编**：按本页末元素枢轴重做共同数组的三次分区，列出区间。
2. **自编**：对 `[2甲,2乙,1]` 模拟堆排序，说明不稳定。
3. **平台题（已核实）**：LeetCode 75 颜色分类：三种颜色可用三路区间划分；一趟扫描时间 O(n)、辅助空间 O(1)，注意换入未知区的元素仍须检查。

综合

1. **平台题（已核实）**：LeetCode 912 排序数组：题目要求不用内置排序、时间 O(n log n)；可用堆排序保证最坏 O(n log n)、辅助 O(1)，或用归并排序及 O(n) 辅助空间。

归并排序与基数排序

对大批按时间顺序到达的记录排序时，既希望结果有序，也可能需要保持相同时间的原始次序。归并排序利用有序段合并；基数排序利用关键字的数字位分配。前置：稳定性与比较口径。

二路归并排序：先划分，再合并

对数组 `[4,2,5,1,3]`，递归把区间分至单元素，随后按递归返回顺序合并。先展示**完整划分关系**，它不是数组实际移动顺序：

```
划分：[4,2,5,1,3]

          /            \

     [4,2,5]          [1,3]

      /    \          /   \

   [4,2]   [5]      [1]   [3]

   /   \

 [4]   [2]
```

实际深度优先执行会在左子树划分完成后先归并左子树，再处理右子树；上图仅总结所有区间划分。**合并阶段**按先左后右的递归返回顺序为：

顺序	左段	右段	合并结果	键比较次数
1	[4]	[2]	[2,4]	1
2	[2,4]	[5]	[2,4,5]	2
3	[1]	[3]	[1,3]	1
4	[2,4,5]	[1,3]	[1,2,3,4,5]	3

合计 7 次键比较；一边耗尽后直接复制另一边，不算额外键比较。最后一趟依次选择 1、2、3，随后复制 4、5。合并时相等取左段首记录，故同键的原始先后得到保留。

```
归并排序(A, lo, hi):                  // 闭区间

    若 lo ≥ hi: 返回

    mid ← lo + floor((hi-lo)/2)

    归并排序(A,lo,mid); 归并排序(A,mid+1,hi)

    将两段合并到临时数组；相等先取左段；复制剩余部分

    将临时数组复制回 A[lo..hi]
```

n=0 可调用 `(0,-1)`，立即返回。每层合并总工作 Θ(n)，有 Θ(log n) 层，因此最好、平均、最坏均 Θ(n log n)；此数组实现额外空间 O(n)，递归栈 O(log n)，合计 O(n)。链表归并可重连已有节点，但是否额外计递归栈取决于实现。例 `[2甲,1,2乙]` 左段最终 `[1,2甲]` 与右段 `[2乙]` 合并，遇相等先取左侧，结果 `[1,2甲,2乙]`。

多路归并

若有 k 个已有序段，每次从各段当前头部选择最小值，输出后仅推进该段。例 `[1,7]`、`[2,5]`、`[3,4]` 的头部依次从 `(1,2,3)` 变为 `(7,2,3)`、`(7,5,3)`、`(7,5,4)`、`(7,5,∞)`、`(7,∞,∞)`，输出 `1,2,3,4,5,7`。∞ 表示该段已耗尽，不是记录。堆或败者树可在 O(log k) 内调整候选者；共 n 个输出的调整成本 O(n log k)，还需建结构 O(k)，以及 O(k) 的头部/选择结构空间（不含外存缓冲）。有相等键需按来源顺序等规则一致打破平局才能整体稳定；稳定多趟外排还要保留原始顺序信息。外存 I/O 和缓冲预算见外部排序。

LSD 基数排序：低位到高位

用补零后的两位十进制非负键 `[24,12,25,11,23]`，每轮按该位分配到 0..9 桶，**保持同桶输入顺序**，再按桶号收集。这个数组保留共同输入 `[4,2,5,1,3]` 的相对次序并加入十位，用以观察两轮。

轮	非空桶（从小到大）	收集结果
个位	1:[11]；2:[12]；3:[23]；4:[24]；5:[25]	[11,12,23,24,25]
十位	1:[11,12]；2:[23,24,25]	[11,12,23,24,25]

```
LSD(A,d,r):                        // 等长、非负 r 进制键；不足位补零
    对 pos 从最低位到最高位共 d 次:
        清空 r 个桶
        按 A 当前顺序将记录追加到其 pos 位对应的桶
        按桶号递增、桶内原顺序收集到 A
```

每位必须稳定，否则高位相同时已排好的低位次序可能丢失。相等完整键每轮进同桶且次序不变，因此稳定。这个实现每轮需 O(n+r)，共 Θ(d(n+r))；计输出数组和桶头需 O(n+r) 辅助空间。若复用已有节点的链域，仅另计 r 个桶头、尾指针可为 O(r)；若直接预分配 r×n 个桶槽则是 O(rn)，不能统称 O(r)。d、r 与 n 的关系会影响比较，不可笼统称线性。负数、变长字符串需要另定义映射和顺序规则。

MSD 基数排序：高位到低位

同一输入先按十位稳定分配得到 `1:[12,11]`、`2:[24,25,23]`；只在各桶内部按个位递归：1 桶变 `[11,12]`，2 桶变 `[23,24,25]`，按桶顺序连接为 `[11,12,23,24,25]`。每桶独立递归；空桶和单记录直接结束。若各位稳定分配且桶的拼接保持原序，整体可稳定；具体实现若原地交换桶内记录则可能不稳定。以每层总共扫描至多 n 条、每个非空递归节点初始化 r 个桶计，时间 O(dn+rdn) 是宽松上界；采用分配方案和桶初始化优化后常写 O(d(n+r))，不能不加实现条件照搬。空间也取决于缓存复用和递归实现。

算法比较与选型

以下为本套教材中相应版本，n≥2；平均列对于比较排序指互异键均匀随机排列，快排也可采用独立均匀随机枢轴模型。空间计辅助数组和递归栈。

方法	最好	平均	最坏	辅助空间	稳定
直接插入	Θ(n)	Θ(n²)	Θ(n²)	O(1)	是
折半插入	Θ(n log n)	Θ(n²)	Θ(n²)	O(1)	是
希尔	依增量序列	依增量序列	依增量序列	O(1)	否
提前终止冒泡	Θ(n)	Θ(n²)	Θ(n²)	O(1)	是
末键枢轴快排	Θ(n log n)	Θ(n log n)	Θ(n²)	O(log n) 平衡、O(n) 最坏	否
简单选择	Θ(n²)	Θ(n²)	Θ(n²)	O(1)	否
堆排序	Θ(n)（全相等键）	Θ(n log n)	Θ(n log n)	O(1)	否
数组二路归并	Θ(n log n)	Θ(n log n)	Θ(n log n)	O(n)	是

方法	最好	平均	最坏	辅助空间	稳定
上述 LSD	Θ(d(n+r))	Θ(d(n+r))	Θ(d(n+r))	O(n+r)	是

外部文件太大时应优先考虑页 I/O 而非单纯比较数；内存中大量记录需要稳定性时可选择归并；整数键位数较少且有足够桶空间时 LSD 可避免关键字之间的比较，但处理负数和长键须改变规则。具体排序库常采用混合策略，应查其版本与实现，不能根据名称推断。

分级练习

基础

1. **自编**：合并两个有序段时相等先取哪一段才能稳定？答：原顺序在前的左段。
2. **自编**：LSD 为什么每一位都要稳定？答：保留已处理低位的顺序。
3. **自编**：长度为零的归并排序如何终止？答：空区间直接返回。

进阶

1. **平台题（已核实）**：LeetCode 88 合并两个有序数组：从两个有效段的末尾向前合并，时间 O(m+n)、辅助 O(1)；空段可直接处理。
2. **自编**：按本文两位数案例分别演示 LSD 和 MSD，并比较其分组方向。
3. **自编**：r 个长度为 n 的预分配桶与复用链域的 O(r) 额外桶头有何区别？

综合

1. **平台题（已核实）**：LeetCode 23 合并 K 个升序链表：以小根堆维护非空链表头，总节点数为 n，时间 O(k+n log(k+1))、堆辅助空间 O(k)；空列表直接返回空。

第 10 章　外部排序与综合应用

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

外部排序：归并段、缓冲与 I/O

当日志文件超出可用内存，不能假设所有记录同时驻留内存。外部排序先在内存生成若干有序段，再多路归并；设计重点从纯比较次数扩展到外存读写、内存缓冲和数据块边界。本文以块为 I/O 单位时，读一块和写一块各算 1 次块 I/O；记录级搬运另行计算。

大文件案例：两阶段与四路平衡归并

假设文件有 1000 条记录，恰好 10 块、每块 100 条。阶段一排序工作区只能容纳 100 条，顺序读取每一块、内存排序并写回初始段，产生 10 个各 100 条的有序段。阶段二单独配置至少 500 条记录的缓冲预算：4 个整块输入缓冲、1 个整块输出缓冲；段头、归并控制和其他工作空间还需另外预算。阶段一的 100 条容量不等于阶段二总内存。

```
原文件 10 块 → 内存排序 → 10 个初始段

10 个段 → 四路归并：4段、4段、2段 → 3 个段

3 个段  → 最后一趟归并 → 1 个有序文件
```

阶段	输入与输出	读块	写块
初始段生成	10 块 → 10 个 1 块段	10	10
第 1 趟归并	10 段 → 3 段（长度 4、4、2 块）	10	10
第 2 趟归并	3 段 → 1 段（10 块）	10	10
合计	每趟均完整落盘，含最终写出	30	30

因此共 **60 次块 I/O**；归并阶段为 40 次，初始生成另 20 次。这里每段均为整块、无部分块、无缓存命中，不计元数据和中间文件额外开销。如果最终输出直接流向后续计算而不落盘，统计会改变。k 路、初始段数 R>1、每趟按 k 个分组直到一段，趟数为 `ceil(log_k R)`；R≤1 无归并趟。并不是归并路数越多越好：输入缓冲、输出缓冲和选择结构需要内存。

败者树：维护各路当前最小值

归并时各路各保留一个当前头；比赛树内部保存对手中的败者，根外保存总冠军。输出冠军后，只将它所在路的新头沿原路径与记录的败者逐级重赛，其余路不重新比较。耗尽的路可用概念上的 +∞ 或专门的失效标记；实际存在最大值时不能把同值普通记录误当作哨兵。

例四路 `A=[1,7]`、`B=[2,5]`、`C=[3,6]`、`D=[4,8]`：初始比较 1 对 2（败者 2）、3 对 4（败者 4），再 1 对 3（败者 3），冠军 1，建树 3 次键比较。输出 1、A 补 7：7 对 A/B 的败者 2，2 胜；2 再对上层败者 3，2 胜。冠军变为 2，共 2 次调整比较；下一次输出 2 并补 5，路径上的比较继续更新对应败者。相等键可按段次序等额外规则决定胜者，

以支持稳定归并。

初始化：各路读入头记录；构造比赛树并保存各节点败者与冠军

当冠军有效：

输出冠军；从其所在路补下一个头（耗尽则标记无效）

沿该叶到根逐层与保存的败者比赛，更新败者与冠军

对 4 路补齐的完全二叉树，全部叶到根深度为 2，若每层都执行一次比较且计入无效标记比较，每次调整恰 2 次；有效键比较可能更少。不补齐时不同叶深可不同，如 3 路树叶深为 1、2、2，5 路树可有 2、2、2、3、3，不能一概说每路都恰 $\text{ceil}(\log_2 k)$ 次。需分别计建树、结束哨兵和调整；通常构建 O(k)，每个输出调整 O(log k)，共 O(n log k)，选择结构 O(k)，缓冲空间另计。

置换选择：生成更长的初始段

固定工作区 w=3，输入 `4,1,3,2,6,0,5`；先读 4、1、3。输出当前活动记录最小值，并读一个新记录补位；新值若小于刚输出的值，则冻结到下一段，否则留在当前段。当前段无活动记录时，解冻剩余记录并开始新段。

输出	新读入	活动记录（无顺序要求）	冻结记录
1	2	{2,3,4}	∅
2	6	{3,4,6}	∅
3	0	{4,6}	{0}
4	5	{5,6}	{0}
5	无	{6}	{0}
6	无	∅	{0}

第一段 `[1,2,3,4,5,6]`；解冻后第二段 `[0]`。工作区有限时可用堆组织活动记录；读完后不再补充。某些独立随机输入模型下平均段长常近似 2w，但它不是任意数据保证：降序等输入可产生短得多的段。段数减半也不必然减少归并趟：k=4，16 段和 8 段都需 2 趟。

最佳归并树：不等长段与计数口径

已有段长 2、3、4、5 条，允许任意两段二路归并且只优化归并阶段的记录级读写量；不要求保持跨段原始稳定顺序。先合 2+3=5，再合 4+5=9，最后合 5+9=14。树叶权为段长、深度为参与的归并次数，`WPL=5+9+14=28` 条记录层次搬运量。每次归并均读入各记录并写出一次，所以归并阶段记录级读写总量为 `2×WPL=56`；不含生成初始段。块 I/O 还取决于每段如何占块以及缓冲，不能直接把 56 当作块次数。若要求稳定合并并限制只能合并原始相邻段，普通哈夫曼合并未必仍是该约束下的最优解。多路最佳归并树中是否补零权虚段，也须写明分叉规则。

工程考虑

数据库排序、日志归档和批处理常把有序段顺序写盘再归并，顺序 I/O 能减少反复随机访问；但存储介质、缓存、临时空间和写放大都会影响实际耗时。败者树减少多路候选的比较，代价是选择结构及每路缓冲；置换选择可能减少段数，但数据特征与内存限制决定效果。不同来源的同键记录要稳定输出，必须明确跨段平局顺序，并在生成段和后续归并中一致保持。

分级练习

基础

1. **自编**：10 个初始段采用 4 路平衡归并，需几趟？答：2 趟。
2. **自编**：按一块一输入缓冲及一块输出缓冲的模型，4 路最少几块缓冲？答：5 块，控制信息另计。
3. **自编**：当前段输出 5，补入 3，应怎样处理？答：冻结至下一段。

进阶

1. **自编**：复算本文 1000 条案例，分别写出生成段、每一归并趟及总块 I/O。
2. **自编**：复算四路败者树输出 1、补入 7 后的两次比赛。
3. **自编**：计算段长 2、3、4、5 的 WPL 与记录级读写量，并说明是否包含初始段生成。

综合

1. **自编**：在给定输入缓冲、输出缓冲和内存预算下设计多路归并，并讨论负载不均、记录稳定性、末尾部分块与统计口径。独立题解提供一个明确预算的练习。

线性结构综合：从题设到算法

本章迁移原 `Day10` 的算法设计方法与已有题型；基础结构操作分别见顺序表、单链表和栈与队列。数组默认 0 基，链表默认有限、无环；有特殊约定时以各题为准。

来源说明：以下题型与案例依据原资料 Day10 的相应小节整理，原资料中提及的真题年份尚未逐题核实，因此本章不将它们标为历年真题；来源状态统一记为**来源待核实**。题设的明确化和伪代码是为自学所作的整理，不能据此推断原题出处。

K101：算法设计题的三段式

1. **问题分析**：说明输入结构、合法范围、输出、要求保留的性质与极端情况。例如循环左移要先确定 n=0 是否允许、p 的取值，以及是否允许修改原数组。
2. **算法设计**：解释为什么选择该结构和步骤，写出可执行的关键伪代码与不变量。例如下文 K103 把数组分成 X、Y 两段，先分别逆置再整体逆置，达到 XY→YX。
3. **复杂度分析**：按实现统计操作次数及辅助空间，说明是否计输入、输出、递归栈，以及是否包含建结构的成本。不能只写一个结果而不说明它对应什么规模。

以 K103 的 `A=[1,2,3,4,5,6,7,8]`、`p=3` 为完整示范：输入为长度 8 的数组，要求原地左移 3 位；设 `X=[1,2,3]`、`Y=[4,5,6,7,8]`，分段逆置后整体逆置得到 YX，结果 `[4,5,6,7,8,1,2,3]`。具体伪代码和各中间状态见 K103；三次扫描累计 O(n) 时间、仅常数变量 O(1) 辅助空间。空数组先返回以避免对零取模，p=0 不改变顺序。考场应写清每次逆置的区间和终止条件，不能只给“翻转三次”或只写复杂度。

K102：高频题型如何相互关联

数据特征 / 问题	关键工具	案例位置
数组两段换位	双指针区间逆置	本文 K103
两个有序序列找指定次序	有序性与按秩淘汰	本文 K104

数据特征 / 问题	关键工具	案例位置
主元素候选	抵消不变量与第二遍计数	本文 K105
单链表分组 / 交点	摘链与尾插、结点身份、长度对齐	本文 K107、K108
树的叶权和结构判定	遍历时携带深度、队列及空位置	树图综合 K106、K109
有向图依赖与判环	入度、队列和输出数	树图综合 K110

K103：数组循环左移 p 位

问题与约定：输入长度 n≥0 的数组 A、非负整数 p；原地将前 p 个元素循环移到尾部。n=0 返回空数组；n>0 时先令 p←p mod n。输出修改后的 A，不额外复制数组。

思路与推导：令 A=XY，X 是前 p 项、Y 为其余项。`reverse(reverse(X) reverse(Y))=YX`；每段都原地双指针交换。相同元素也按位置移动，不依赖关键字大小。

```
逆置(A, left, right):                  // 闭区间，left≥right 时不执行
    当 left < right:
        交换 A[left], A[right]
        left ← left+1; right ← right-1


循环左移(A, p):                        // p 为非负整数
    n ← |A|
    若 n=0: 返回 A                     // 必须在取模之前
    p ← p mod n
    逆置(A,0,p-1)
    逆置(A,p,n-1)
    逆置(A,0,n-1)
    返回 A
```

手算：n=8、p=3：

步骤	数组
初始	`[1,2,3,4,5,6,7,8]`
逆置前 3 项	`[3,2,1,4,5,6,7,8]`
逆置后 5 项	`[3,2,1,8,7,6,5,4]`
整体逆置	`[4,5,6,7,8,1,2,3]`

p=0 或 p 为 n 的倍数时结果不变；n=1 同理。p<0 不属于本题约定，若允许负数需另定义方向和取模。三次逆置总计 O(n) 时间、O(1) 辅助空间，输入数组本身不计入辅助空间。常见失误是空数组取模、闭区间端点越界、把左移误当右移。

K104：两个等长升序序列的第 n 小元素

问题与约定：A、B 各有 n≥1 个元素，均按非递减序排列，可有重复；返回合并后的 2n 个元素中的**第 n 小**，相同值按出现次数分别计数。这是此题的“中位数”约定，不取第 n 和第 n+1 小的平均值。输入不修改。

思路与推导：维护 A、B 未淘汰部分的起点 a、b 和目标秩 k；初始 k=n。若一段为空，答案在另一段的第 k 个；k=1 时取两段首项较小者。否则在两段中分别考察至多 `floor(k/2)` 个元素。设数量为 i、j，比较各段所考察的末元素。若 A 的末元素不大于 B 的末元素，可淘汰 A 的前 i 个并令 k 减 i；相反淘汰 B 的前 j 个。这里 i+j≤k。若 A 的第 i 个不大于 B 的第 j 个，则严格小于 A 第 i 个值的元素，在 A 中至多有 i−1 个、在 B 中至多有 j−1 个；因此该值不大于剩余元素中的第 k 小。淘汰 A 前 i 个后，将目标秩减 i；相等值按出现次数计数，选择一侧淘汰仍不改变目标值。每次淘汰后保持“目标在剩余元素中为第 k 小”。

```
第n小(A, B):                          // 长度都为 n≥1，均升序
    a ← 0; b ← 0; k ← n
    循环:
        若 a=|A|: 返回 B[b+k-1]
        若 b=|B|: 返回 A[a+k-1]
        若 k=1: 返回 min(A[a], B[b])
        i ← min(floor(k/2), |A|-a)
        j ← min(floor(k/2), |B|-b)
        若 A[a+i-1] ≤ B[b+j-1]:
            a ← a+i; k ← k-i
        否则:
            b ← b+j; k ← k-j
```

手算：A=[11,13,15,17,19]，B=[2,4,6,8,20]，找第 5 小。

a	b	k	比较与淘汰
0	0	5	i=j=2；13>4，淘汰 B 的 2、4
0	2	3	i=j=1；11>6，淘汰 B 的 6
0	3	2	i=j=1；11>8，淘汰 B 的 8
0	4	1	min(11,20)=11

输出 11。若 A=[1,3,5]、B=[2,4,6]，第 3 小为 3，而两个中间值平均为 3.5；二者是不同题设。n=1、重复值、某一剩余段先耗尽均由分支覆盖；两个空序列不满足 n≥1。每轮若未直接耗尽某段，则目标秩按比例下降，时间 O(log n)，辅助空间 O(1)。考场易错点是淘汰后忘记同步减少 k，或把第 n 小改写成平均值。

K105：数组主元素

问题与约定：给定长度 n≥0 的数组，若某值出现次数严格大于 n/2，输出该值；否则返回“无主元素”。使用与数组值区分的结果状态，不能把某个合法数值强行当失败哨兵。

思路与推导：不同值两两抵消，若真正的主元素存在，抵消后它仍有剩余。第一遍投票得到候选，**不能保证候选一定是主元素**，必须第二遍数真实次数。

```
主元素(A):
    若 |A|=0: 返回“无主元素”
    count ← 0
    对每个 x ∈ A:
        若 count=0: candidate ← x; count ← 1
        否则若 x=candidate: count ← count+1
        否则: count ← count-1
    occurrences ← 0
    对每个 x ∈ A:
        若 x=candidate: occurrences ← occurrences+1
    若 occurrences > |A|/2: 返回 candidate
    返回“无主元素”
```

手算：A=[3,3,4,3,5,3,3,2,3]。按顺序处理后 (候选,票数) 为 (3,1),(3,2),(3,1),(3,2),(3,1),(3,2),(3,3),(3,2),(3,3)；第二遍数出 3 出现 6 次，6>9/2，输出 3。对 [1,2,3,1,1,2]，候选需要验证；1 出现 3 次，不严格大于 6/2，因此无主元素。

空数组直接返回无；单元素必是主元素。时间 O(n)，辅助空间 O(1)，两遍不改变输入。常见错误是省略第二遍，或将“严格大于”写成“大于等于”。

K107：按结点数据值奇偶拆分单链表

问题与约定：给定有限、无环、数据域为整数的单链表首元指针 first；复用所有原数据结点，按**数据值**奇偶拆成奇数链、偶数链，分别保持原有相对次序。返回两条首元指针；输入链表的原链接会改变，不新建数据结点。

思路与推导：每次先保存后继，再把当前结点摘下，尾接到对应结果。两个尾指针记录最后结点，首次追加时同时设置首指针；循环不变量是已处理结点恰好进入一条结果链，未处理部分仍由 p 指向。

```
奇偶拆分(first):                    // odd(x) 对负整数也按奇偶定义
    oddHead ← null; oddTail ← null
    evenHead ← null; evenTail ← null
    p ← first
    当 p ≠ null:
        next ← p.next; p.next ← null
        若 odd(p.data):
            若 oddHead=null: oddHead ← p
            否则: oddTail.next ← p
            oddTail ← p
        否则:
            若 evenHead=null: evenHead ← p
            否则: evenTail.next ← p
            evenTail ← p
        p ← next
    返回 (oddHead, evenHead)
```

手算：输入 6→1→8→3→4→5→2→7：

处理后	奇数链	偶数链
6、1	1	6
8、3	1→3	6→8
4、5	1→3→5	6→8→4
2、7	1→3→5→7	6→8→4→2

空链返回 (null,null)；全奇或全偶时另一条为空；单结点正确终止。若原表带头结点，应先传入 head.next，调用者负责按接口管理头结点；不要把哨兵当数据。每个结点恰处理一次，时间 O(n)、辅助空间 O(1)，复用结点不计新链结点空间。易错点包括未先保存后继、按位置奇偶拆分、忘记终止尾链，以及误称保留原链表不变。

K108：两个单链表的第一个公共结点

问题与约定：输入两条有限无环单链表首元引用 A、B；公共结点指**同一个结点对象**，不是数据相同。若共享后缀，返回两链首次共同到达的结点；不相交返回 null。不得修改链表。

思路与推导：分别求长度 m、n，长链先走 |m-n| 步，令两个指针距各自末尾的距离相同；再同步前进。共享后缀的指针会在第一个共同结点处相遇；不相交时同时到达 null。

```
首个公共结点(A, B):
    m ← 链长(A); n ← 链长(B)
    p ← A; q ← B
    若 m>n: 令 p 向后走 m-n 步
    否则: 令 q 向后走 n-m 步
    当 p ≠ q: p ← p.next; q ← q.next
    返回 p                          // 可能为 null
```

手算：A 为 1→2→3→X(7)→Y(8)→Z(9)，B 为 4→5→X(7)→Y(8)→Z(9)；X、Y、Z 是**同一组结点。**m=6、n=5，A 先走一步至 2；同步比较 (2,4)、(3,5) 后两者都指向 X，返回 X。若另建两个值都为 7 但身份不同的结点，它们不构成交点。

任一链为空、不相交、两首指针相同均适用；若可能有环需另行处理，不能套用该长度法。求长与同步移动总时间 O(m+n)，辅助空间 O(1)。易错点是比较 data 代替结点身份、对齐方向反了，或没有处理返回 null。

C 考场表达：数组与链表

以下是与 K103、K107、K108 对应的**标准 C11**表达示例；每个 C 代码块含本块所需的头文件和类型定义，可独立作为一个翻译单元编译，测试时需另外提供调用代码。原章节的 text 块是伪代码，不可直接当 C 编译。本节不改变前述算法的题设。

数组参数与 K103 左移

数组参数 int *a 传入首元素地址，函数必须另收长度；数组元素可通过 a 修改，但形参 a 本身只是指针副本。下例区间为左闭右开 [left,right)，并约定 n>0 时 a 指向至少 n 个可写 int；n=0 时允许 a 为 NULL。p 是非负的 size_t，无效输入以 0 报告。

```c
#include <stddef.h>


static void reverse_part(int *a, size_t left, size_t right) {
    while (right - left > 1) {
        --right;
        int tmp = a[left];
        a[left] = a[right];
        a[right] = tmp;
        ++left;
    }
}


int rotate_left(int *a, size_t n, size_t p) {
    if (n == 0) return 1;       /* 不访问 a，也不对 0 取模 */
    if (a == NULL) return 0;
    p %= n;
    reverse_part(a, 0, p);
    reverse_part(a, p, n);
    reverse_part(a, 0, n);
    return 1;
}
```

三个右开区间依次对应 K103 的前段、后段、整体；调用者保证非空时指针及长度真实有效。空数组和单元素返回成功；若 n>0 却传 NULL 返回失败。三次逆置时间 O(n)、辅助空间 O(1)。不能从指针参数用 `sizeof(a)` 推断调用者数组长度，也不能在 n=0 前执行 `p %= n`。

链表结点、参数与 K107 摘链

C 的所有参数按值传递：`Node *p` 是指针副本，可以通过它修改结点，但给 p 赋新地址不会改变调用者的头指针。需要改变调用者头指针时，可**返回新的 `Node *`**，例如调用方赋 `head = reverse(head)`；需要同时返回两条链时，可用两个 `Node **` 输出参数。

```c
#include <stddef.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

/* 可选建结点函数：失败返回 NULL，成功后由调用方负责释放。 */
Node *make_node(int value) {
    Node *p = malloc(sizeof *p);
    if (p == NULL) return NULL;
    p->data = value;
    p->next = NULL;
    return p;
}

/* first 为无环链首元；odd、even 为两个不同的可写输出位置。
   输出位置不得位于输入链的结点字段中。返回 1 成功，0 参数无效。 */
int split_values(Node *first, Node **odd, Node **even) {
    if (odd == NULL || even == NULL || odd == even) return 0;
    *odd = NULL; *even = NULL;
    Node *odd_tail = NULL, *even_tail = NULL;
    for (Node *p = first; p != NULL; ) {
        Node *next = p->next;      /* 断链前保存后继 */
        p->next = NULL;
        if (p->data % 2 != 0) {   /* 负奇数也满足 */
            if (*odd == NULL) *odd = p;
            else odd_tail->next = p;
            odd_tail = p;
        } else {
            if (*even == NULL) *even = p;
```

```
            else even_tail->next = p;
            even_tail = p;
        }
        p = next;
    }
    return 1;
}
```

例如调用方使用 `Node *odd = NULL, *even = NULL;` 并传 `&odd, &even;` 函数通过 `*odd` 和 `*even` 改写调用方的指针变量。空链成功得到两个 NULL；所有结点为奇数或偶数时另一条链为空。此函数复用结点并修改链接，**不分配也不释放**；时间 O(n)、辅助空间 O(1)。K108 判断公共结点时比较的是 `p == q` 指针身份，不是 `p->data == q->data`；两链共享后缀时不能各自无条件 `free` 整条链，以免重复释放。

若题目确需动态创建结点，`malloc` 来自 `<stdlib.h>`；标准 C 可写 `Node *p = malloc(sizeof *p);`，随后先检查 `p == NULL` 再解引用。只对已动态分配、仍由当前方负责且不再使用的对象 `free`；不能释放自动变量或仍被其他结构使用的共享结点。上述拆分函数没有调用 `malloc`，故不产生分配失败分支。

栈与循环队列的最小表达

以下两个固定容量结构使用 `{0}` 初始化；栈容量为 2，环形队列有 2 个槽且牺牲一槽，**实际可存 1 个元素**。失败返回 0，成功返回 1；调用者传入已初始化结构。出栈或出队的输出地址必须有效。

```c
#include <stddef.h>


enum { CAP = 2, M = 2 };

typedef struct { int data[CAP]; size_t size; } Stack;

typedef struct { int data[M]; size_t front, rear; } Queue;


int stack_push(Stack *s, int x) {

    if (s == NULL || s->size == CAP) return 0;

    s->data[s->size++] = x;

    return 1;

}

int stack_pop(Stack *s, int *out) {

    if (s == NULL || out == NULL || s->size == 0) return 0;

    *out = s->data[--s->size];

    return 1;

}

int enqueue(Queue *q, int x) {

    if (q == NULL || (q->rear + 1) % M == q->front) return 0;

    q->data[q->rear] = x;

    q->rear = (q->rear + 1) % M;

    return 1;

}

int dequeue(Queue *q, int *out) {

    if (q == NULL || out == NULL || q->front == q->rear) return 0;

    *out = q->data[q->front];

    q->front = (q->front + 1) % M;

    return 1;

}
```

例如声明 `Stack s = {0}; Queue q = {0};`，先判满/空再访问数组。代码假定结构状态未被外部破坏，单次操作 O(1)，结构存储分别为 O(CAP)、O(M)；队列不能把 M 个槽都当作可用容量。更一般的公式及其他判满约定见队列与双端队列。

C++ 中可用 `nullptr` 表示空指针，也可用引用参数让函数改变调用者变量；它们不是 C11 语法。C++ 的对象构造和内存管理规则需另行遵守，不能把 C 的 `malloc` 当作会自动构造 C++ 对象的通用方法。

树与图综合：携带状态的遍历

本章依据原 `Day10` 的树图题型迁移，前置知识见二叉树基础、树的遍历、哈夫曼与 WPL和拓扑排序。树结点引用均可为 null；图默认有限有向图。

题源说明：以下案例和题型依据原 Day10 整理，原文所提及的真题年份未逐题核实，本章不将其认定为历年真题，题源均标为**来源待核实。**

K102：题型之间的联系

WPL 遍历要让递归携带“深度”；完全树判定用 FIFO 队列保持层次次序，并记录是否已见空位；Kahn 判环则用队列维护当前入度为 0 的图顶点。三者都在遍历中维护额外状态，但含义不同。数组与链表题型的逆置、按秩淘汰、计数和指针对齐见线性结构综合 K102—K108。

K106：二叉树带权路径长度 WPL

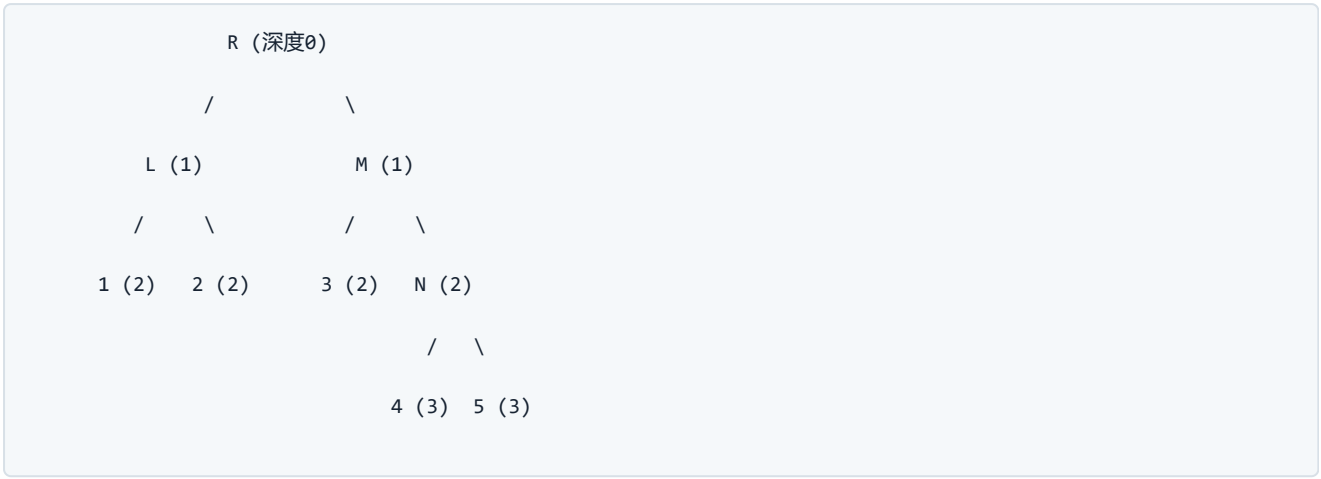
问题与约定：给定有限二叉树，每个叶结点有权值；根的深度定义为 0，其他结点深度为从根到它的**边数。**输出 `Σ(叶结点权值×叶结点深度)`；内部结点的值不计入。空树 WPL 约定为 0，树高 h 按层数计。

思路与推导：深度作为遍历参数，进入孩子时加 1；只有遇到叶结点才计 `weight×depth`。子树返回的和可直接相加。与哈夫曼树构造不同：本题计算的是**已给定树**的 WPL，不声称给定树一定最优。

```
WPL(root):
    返回 求和(root, 0)

求和(p, depth):
    若 p=null: 返回 0
    若 p.left=null 且 p.right=null:
        返回 p.weight × depth
    返回 求和(p.left, depth+1) + 求和(p.right, depth+1)
```

手算与结构核验：原例给 5 个叶权 `1,2,3,4,5`，对应深度 `2,2,2,3,3`。原文字只给出权值与深度，并未唯一指定树；以下构造**一个满足原始数据的树**，并非宣称是原文唯一树形：



叶结点分别是 1、2、3、4、5，括号为深度；R、L、M、N 为内部结点。逐项计算：

叶权	深度	贡献
1	2	2
2	2	4
3	2	6
4	3	12
5	3	15
合计	—	**39**

空树返回 0；只有根且根为叶时深度 0，按本约定贡献为 0。每个结点访问一次，时间 O(n)；递归栈 O(h)，不含输入树。易错点是把内部结点权值计入、用根层号 1 当深度 1，或把 `39` 误称为该权集的哈夫曼最优 WPL。原数据足以构造上述树并支持 39，但不足以唯一确定树形。

K109：判断二叉树是否完全

问题与约定：输入有限二叉树根 root；返回布尔值。允许队列保存 null；按层次从左至右处理，包括空孩子。空树按本题约定视为完全二叉树。

思路与推导：完全二叉树的层序位置从左到右连续。遇到第一个 null 后设置 `seenGap=true`；此后如果再次出队真实结点，就说明空位之后仍有结点，应返回 false。对真实结点按左、右顺序把孩子引用入队；null 不再展开，避免无穷入队。

```
是否完全(root):                    // 队列允许存 null
    Q ← 空队列; Q.入队(root)
    seenGap ← false
    当 Q 非空:
        p ← Q.出队()
        若 p=null:
            seenGap ← true
        否则:
            若 seenGap: 返回 false
            Q.入队(p.left); Q.入队(p.right)
    返回 true
```

手算：根 1 的孩子是 2、3，结点 2 左孩子空、右孩子 4；按队首在左表示：

出队	seenGap	入队后队列	结果
1	false	[2,3]	继续
2	false	[3,null,4]	继续
3	false	[null,4,null,null]	继续
null	true	[4,null,null]	记录空位
4	true	[null,null]	出现空位后的真实结点，返回 false

空树仅处理一次 null 后返回 true；单结点返回 true；只有右孩子返回 false。n 个真实结点最多引入 2n+1 个引用，时间 O(n+1)，队列空间 O(n+1)，n≥1 时均为 O(n)。若队列实现不允许存 null，应另用等价的“遇空孩子后检查后续孩子”实现，不能直接运行本伪代码。常见错误是从队列中过滤掉空位，丢失判定依据。

K110：Kahn 拓扑排序与判环

问题与约定：给定 V 个顶点、E 条弧的有限有向图，返回拓扑序；如果有向环存在则返回“有环”。使用邻接表及一个可变入度副本，不修改图本身；V=0 返回空序列。若图有重边，入度与邻接表都按每条弧计数并更新；自环导致不存在拓扑序。

思路与推导：任何拓扑序的第一个顶点必须入度为 0。把所有当前入度 0 的顶点入队，出队一个就输出，并对它每条出边的终点入度减一；新变为 0 的顶点入队。若最终输出 V 个，输出序列满足全部前驱关系；若小于 V，剩余顶点的依赖无法消除，图中有有向环。

```
Kahn判环(G):                        // 有向图；adj[u] 枚举 u 的每条出弧
    indegree ← 长度 V 的全零数组
    对每个 u 及每条出弧 u→v: indegree[v] ← indegree[v]+1
    Q ← 所有入度为 0 的顶点
    order ← 空列表
    当 Q 非空:
        u ← Q.出队(); order.append(u)
        对每条出弧 u→v:
            indegree[v] ← indegree[v]-1
            若 indegree[v]=0: Q.入队(v)
    若 |order| < V: 返回“有环”
    返回 order
```

手算：顶点 0..3，弧 0→1、1→2、2→3、3→1；初始入度 [0,2,1,1]。

操作	入度 [0,1,2,3]	队列	输出
初始化	[0,2,1,1]	[0]	[]
出 0，删 0→1	[0,1,1,1]	[]	[0]
队空	[0,1,1,1]	[]	[0]

输出 1 个小于 V=4，故有环 1→2→3→1。若将 3→1 删除，则依次输出 0、1、2、3，输出数为 4，图无环；不同的零入度选择可能给出不同合法拓扑序。

空图返回空拓扑序；孤立顶点入队并输出；有自环时相应依赖不能消除。邻接表建入度及处理每个顶点和弧共 O(V+E) 时间；入度副本与队列辅助 O(V)，输出序列另占 O(V)，图本身占 O(V+E)。若用邻接矩阵，需要按实现重新计扫描行的 $O(V^2)$ 时间与矩阵存储。常见错误是用无向图直接套用拓扑判环、比较输出数与边数，或重复删除边而不正确更新入度。

C 考场表达：树与队列

以下 C11 代码块有本块所需定义，可独立作为翻译单元编译；与前文 K106 的根深度为 0、仅叶结点参与加权之和的伪代码对应。NULL 判断必须先于任何 p->... 访问。

```c
#include <stddef.h>


typedef struct TreeNode {
    int weight;
    struct TreeNode *left, *right;
} TreeNode;


/* depth 非负；假定所有深度和乘加结果均可由 long long 表示。 */
long long wpl_at(const TreeNode *p, long long depth) {
    if (p == NULL) return 0;
    if (p->left == NULL && p->right == NULL)
        return (long long)p->weight * depth;
    return wpl_at(p->left, depth + 1) + wpl_at(p->right, depth + 1);
}
```

调用 `wpl_at(root, 0)`；空树及根为唯一叶结点均返回 0。每个结点访问一次，时间 O(n)，递归栈 O(h)；若计算可能超出 `long long` 范围，必须另加溢出处理，不能依赖有符号溢出。`const TreeNode *` 表示函数不通过该指针修改结点。

K109 的伪代码把**空指针也入队**。若用只增不回绕的指针数组队列且树有 n 个真实结点，最多需写入 `2n+1` 个槽（空树也需要 1 槽）；先检查 n、容量计算和分配不会溢出，并允许槽中保存 `NULL`。处理 NULL 时只记录空位，不得解引用，也不得继续扩展它；容量不足应报告错误，不能当作“不完全”。如果使用不保存空指针的等价实现，必须重新写清空位判定规则和容量。

K110 的顶点队列存的是顶点编号，而不是树结点指针；正确更新入度时每个顶点最多入队一次，线性数组容量 V 足够，V=0 应直接处理，不能假设零长度数组是通用 C 语法。入度数组需先正确初始化，并按每条弧（包括重边）递增、递减；调用者须提供足够空间并保证计数类型范围足够。

判定函数最好用返回状态区分“执行成功但图有环”和“输入无效或队列容量不足”等错误，必要时通过输出参数返回布尔判定或拓扑序长度；若接口以特殊值报告失败，须确保与合法结果不冲突。不要把分配失败、容量不足当作图有环或树不完全。只使用固定容量数组时需先证明上界；需要动态空间时检查分配失败，并按所有权释放。

结构选型与工程实践

本页把已学结构组合起来解决具体问题。成本均在明确的实现假设下讨论，实际系统还需测量数据分布、并发和资源约束。基础操作分别见顺序表、双向链表、散列表、图的表示和最短路径。

案例一：LRU 缓存

需求：服务最多缓存 c 条用户资料。读取或更新后视为最近使用；空间满时淘汰最久未访问的条目。只用数组逐项寻找要 O(c)；只有链表也需 O(c) 查找键。选择**哈希表加双向链表**：表将键映射到结点，链表从头到尾按最近到最久排列，定位后可常数次改链。哈希表期望查找 O(1)，不是无条件的最坏 O(1)。

维护头尾哨兵、键到结点映射 `H`。每个数据结点保存 key、value、prev、next。使用互斥保护或其他并发协议保证两种结构同步；以下伪代码只描述单线程。

```
get(key):
    若 key 不在 H: 返回 未命中
    p ← H[key]; 从链中摘下 p; 将 p 接到头哨兵之后
    返回 p.value
put(key, value):
    若 c = 0: 返回
    若 key 在 H:
        p ← H[key]; p.value ← value; 摘下 p
    否则:
        若 |H| = c:
            q ← 尾哨兵之前的结点; 摘下 q; 从 H 删除 q.key
        创建结点 p(key,value); H[key] ← p
    将 p 接到头哨兵之后
```

手算：c=2，依次 `put(A,1)`、`put(B,2)`、`get(A)`、`put(C,3)`；链表从新到旧分别为 `[A]`、`[B,A]`、`[A,B]`、`[C,A]`，最后淘汰 B。`get(A)` 命中返回 1，`get(B)` 未命中。摘链前后须维护双向指针；删除链结点时同步删哈希映射。

在哈希操作期望 O(1) 的假设下，get/put 期望 O(1)，最坏取决于散列实现且可达 O(c)；空间 O(c)。容量 0、重复键、失效时间、对象大小不等、并发和持久化都需额外策略，LRU 也不保证适应周期扫描。

案例二：任务调度

需求：后台先处理较高优先级任务，同优先级按到达次序处理；任务可更新优先级。FIFO 队列难以迅速找到最高优先级；使用**索引二叉堆**，比较键为 `(-priority, sequence)`，使优先级高者先出、同级 sequence 小者先出。另用映射 `pos[id]` 找到任务在堆中的位置，堆元素交换时同步更新两个 pos。

```
insert(id, priority):
    若 id 已存在：返回 失败
    seq ← 下一个不重复到达序号
    在堆尾追加 (id, priority, seq)；更新 pos
    向上调整堆并在每次交换时更新 pos；返回 成功
pop():
    若堆空：返回 空
    取走堆顶，删除其 pos；将末元素移到堆顶
    若堆非空：更新其 pos 并向下调整
    返回取走的任务
update(id, newPriority):
    若 id 不存在：返回 失败
    修改堆中对应项的 priority，保留原 seq
    按新旧比较键关系向上或向下调整，并更新 pos
```

手算：插入 A(2)、B(5)、C(5)，到达序号依次 1、2、3；B 优先于 C。执行 `update(A,6)` 后 A 升到堆顶；依次 pop 得 A、B、C。若更新希望重置排队时间，应明确改写 sequence，不能默默沿用不同规则。

有 n 个任务时，堆调整最坏 O(log n)，映射操作为期望 O(1)、最坏依实现而定；每项操作期望 O(log n)，空间 O(n)。持续高优先级任务可能使低优先级饥饿，可另设老化策略；持久队列、重试、并发和崩溃恢复不由堆自动提供。

案例三：网络路径规划

需求：从站点 S 找到所有站点的最低行程成本，路线权重表示非负耗时；无路应明确返回不可达。每个站点为顶点，有向线路为弧、耗时为权；仅在路线双向且成本相同时才各建相应反向弧。稀疏网络用邻接表 O(V+E) 空间；稠密小图或需频繁查任意边可考虑 $O(V^2)$ 的邻接矩阵。

非负权单源查询选 Dijkstra；下面用小根堆和惰性删除，`dist` 初始化为无穷，`pred` 用于回溯。负边时一般不保证正确，可改用适合负边的算法并检查负环；动态路况、时刻表或多目标成本需要进一步建模。

```
路径(S):
    所有 dist ← ∞, pred ← null; dist[S] ← 0
    H ← 小根堆，加入 (0,S)
    当 H 非空:
        (d,u) ← 删除最小值
        若 d ≠ dist[u]: 继续                  // 过期堆项
        对每条出弧 u→v，权重 w:
            若 d+w < dist[v]:
                dist[v] ← d+w; pred[v] ← u
                插入 (dist[v],v)
    返回 dist, pred
```

手算：S→A=4、S→B=1、B→A=2、A→C=1、B→C=5。初始 (S,A,B,C)=(0,∞,∞,∞)；处理 S 得 (0,4,1,∞)；处理 B 得 (0,3,1,6)；处理 A 得 (0,3,1,4)；C 无可改进项。最终 C 的前驱链 C←A←B←S，路径为 S→B→A→C，耗时 4。

邻接表与惰性堆在初始化后时间 O(V+E log(E+1))，等价地可保守写 O((V+E) log(V+E))；堆中最多 O(E) 条记录，辅助空间 O(V+E)，图存储另计 O(V+E)。空图、未知源、数值溢出和不可达顶点须先定义接口。实际导航还要考虑交通变化、数据采集误差和路线可用性。

延伸练习

上述三个案例是教材自编工程模型；可分别改变容量、任务优先级和边权自行手算。分级练习请从题目总索引进入，基础知识不在此重复。

LeetCode 206：反转链表

来源：力扣 206 反转链表。题目给出单链表首结点，要求返回逆置后的首结点；空链表允许作为输入。

思路与关键步骤

将链表分为已逆置前缀和未处理后缀。prev 是前缀头，p 是后缀头；先保存后继 next，再反转当前结点的链接，最后同时推进两个部分。每一步均保留尚未处理的后缀。

```
反转(first):
    prev ← null
    p ← first
    当 p ≠ null:
        next ← p.next
        p.next ← prev
        prev ← p
        p ← next
    返回 prev
```

对 1→2→3→null：初始 prev=null，p=1→2→3；处理 1 后 prev=1→null，p=2→3；处理 2 后 prev=2→1→null，p=3；处理 3 后 prev=3→2→1→null，p=null。返回 prev。

正确性与边界

不变量：prev 指向已处理前缀的逆序链，p 指向剩余未处理结点；两部分不重叠且保留全部原结点。每轮恰好移动一个结点，结束时后缀为空，因此 prev 为完整逆序链。空链返回 null；单结点的 next 仍为 null。本方法假设输入为有限无环链表，不能先覆写 p.next 再读取原后继。

复杂度

n 个结点均处理一次，时间 O(n)；仅用固定数量引用，辅助空间 O(1)，复用原结点。递归解法通常使用 O(n) 调用栈，不能与迭代版空间混淆。参见单链表正文。

LeetCode 88：合并两个有序数组

来源：力扣 88 合并两个有序数组。题目给出非递减数组 nums1 和 nums2 的有效长度 m、n，nums1 总长度为 m+n，末尾有 n 个可用槽位；结果写回 nums1。题目不要求稳定区分相等元素的来源。

思路与关键步骤

从两个有效区间的末尾开始比较，较大元素放到 nums1 的最后一个未填位置。因为写入从后向前进行，尚未读出的 nums1 有效元素不会被覆盖。若 nums2 先用完，nums1 的剩余前缀已在正确位置；若 nums1 先用完，继续复制 nums2 的剩余前缀。

```
合并(nums1, m, nums2, n):
    i ← m-1; j ← n-1; k ← m+n-1
    当 j ≥ 0:
        若 i ≥ 0 且 nums1[i] > nums2[j]:
            nums1[k] ← nums1[i]; i ← i-1
        否则:
            nums1[k] ← nums2[j]; j ← j-1
        k ← k-1
```

例：`nums1=[1,2,3,_,_,_]`、`nums2=[2,5,6]`。先写 6，再写 5，再写 3；比较两个 2 时写入 nums2 的 2，此时 nums2 已用完，nums1 剩余的 `[1,2]` 保留原位。结果为 `[1,2,2,3,5,6]`。

写入后	已确定的尾部	i	j
6	6	2	1
5	5, 6	2	0
3	3, 5, 6	1	0
nums2 的 2	2, 3, 5, 6	1	-1

此时 nums2 已用完，nums1 前缀 `[1,2]` 不用移动。最终为 `[1,2,2,3,5,6]`。

正确性与边界

不变量：`k+1` 之后已经是合并结果中最大的元素，且按非递减顺序排列；未处理的元素仍在两个前缀中。每次取两前缀末尾较大者填入 k，维持该性质。`j<0` 时其余 nums1 前缀已经在位；`m=0` 时全部从 nums2 拷贝；`n=0` 时循环不执行。必须先判断 `i≥0` 再访问 `nums1[i]`，也不能把末尾空槽视作有效的 0。

复杂度

每次迭代使 i 或 j 减一，最多执行 m+n 次，因此时间 O(m+n)，辅助空间 O(1)，不计题设已有的 nums1 容量。参见顺序表正文。

外部排序自编练习：缓冲预算与 I/O

本题根据原 Day10 外部排序情景重新设计，明确标为**自编题**；不代表原题已核实为历年真题。

题设

有 1000 条记录，每块恰好 100 条，原文件正好 10 块。初始段生成阶段的排序工作区容纳 100 条；归并阶段另有至少 500 条记录容量，恰好可提供 4 个输入整块缓冲和 1 个输出整块缓冲；控制信息占用另行预算。采用四路平衡归并，各阶段结果完整写盘，无缓存命中、元数据 I/O、部分块和压缩。问初始段数、归并趟数，以及按读块和写块分别计数的总 I/O。

解题过程

每次读 100 条排成一段并写盘，得到 10 个初始段。第一趟把 10 段分为 4、4、2 路各归并一次，得到 3 段；第二趟归并这 3 段得到最终 1 段，故共 2 趟。四路归并有 4 个输入块和 1 个输出块，阶段二预算满足模型；不能把阶段一的 100 条工作区误当成归并阶段总缓冲。

初始生成读 10 块、写 10 块；每一归并趟各读 10 块、写 10 块；因此读 30 块、写 30 块，总 60 次块 I/O，其中归并阶段 40 次。若最终结果不写盘或缓冲、文件块条件变化，答案必须相应重算。正文与完整模型见外部排序。

综合案例解析

本页题目为明确标注的自编练习；源自教材中的算法模型，非历年真题。复杂度均说明输入规模和空间口径。

1. 顺序表：移除指定元素

输入数组 A 和值 x，原地保留不等于 x 的元素并保持原次序，返回有效长度。采用读写双指针：`write=0`；依次读取 A 中每个 v，若 v≠x 则写 `A[write]=v` 并令 write 加 1。返回 write。循环不变量：写区恰为已扫描前缀中应保留元素。空数组返回 0，全为 x 返回 0；n 元素时间 O(n)、辅助空间 O(1)。若要求保留原数组需另分配空间。参见顺序表。

2. 栈：括号匹配

输入仅含三类括号的串，判断匹配。依次扫描：左括号入栈；右括号遇空栈或栈顶类型不符返回 false，否则弹栈；最后仅栈空返回 true。例 `([{}])` 的栈依次为 (、([、([{、([、(、空。空串按约定有效；非法字符须按接口另行定义。n 为长度，时间 O(n)，最坏栈空间 O(n)。参见栈与递归。

3. 二叉树：检查完全性

将根（允许为空）入队，维护 seenGap=false。循环出队：若为空则置 seenGap；若非空且 seenGap 已真则返回 false，否则按左、右顺序入队孩子；队空返回 true。例根 1、左孩子 2、2 的右孩子 3：出队顺序 1、2、空、空、3，遇 3 时判 false。空树返回 true；空值不展开。n 个结点引入至多 2n+1 个引用，时间和队列空间 O(n+1)。详见树图综合 K109。

4. 图：拓扑排序与判环

给定邻接表有向图，先数每个顶点入度，将所有零入度顶点入队；反复取出 u、输出 u，并对每条 u→v 减小 v 入度，降为 0 时入队。输出数小于 V 则有环，否则得到拓扑序。例 0→1、1→2、2→1：初始入度 `[0,2,1]`，只输出 0，故有环。重边须逐条计数，自环不能消去；空图得到空序。时间 O(V+E)，辅助空间 O(V)，输出另计 O(V)。详见树图综合 K110。

5. 散列：线性探测与删除

长度 7、h(k)=k mod 7，插入 6、13、20、7：依次占槽 6、0、1、2，探测次数为 1、2、3、3。查找从 h(k) 起逐槽比较，命中返回；遇从未使用槽或走满一圈才失败。删除 13 后槽 0 标记墓碑，查找 20 仍须继续到槽 1；直接置成从未使用会错误地提前停止。四个键等概率成功且只计访问槽，ASL=(1+2+3+3)/4=9/4。最坏单次探测 O(m)，表空间 O(m)；墓碑累积可能需重建。详见散列表。

6. 外部排序：分阶段 I/O

一千条记录、每块 100 条；生成段阶段内存容纳 100 条，生成 10 个整块有序段；归并阶段另有 4 个整块输入缓冲和 1 个输出缓冲。每一趟把所有数据完整读写一次、最终结果写盘、忽略缓存和元数据。4 路归并段数 `10→3→1`，共 2 趟；初始生成 20 次块 I/O，归并 40 次，总计 60。时间成本需另按比较次数分析，内存至少 5 块且控制信息另计。部分块、直接流出或复用缓存时不能套用 60。完整题设见独立题解。

原有习题修订与条件补全

本页为原 Day01—Day10 章末题的**修订版**，不是对原始文件的修改。每题链接原题及原解析的只读摘录；修订题设与答案以本页为准。除非获得可靠证据，本页 20 题均标为**来源待核实**，不得据题型相似认定为历年真题。记号 Θ 表示渐近紧确界；算法复杂度默认不计输入本身，数值运算按明确约定处理。

Day01-1：最紧复杂度：倍增循环

题源：来源待核实。 保留的原题与原解析；原始资料 Day01.md。

修订题设：设 n 为正整数，数学整数运算不会溢出、每次运算耗时 O(1)。x=2，执行 `while x<n/2: x=2*x`。从 Θ(log n)、Θ(n)、Θ(n log n)、Θ(n²) 中选择最紧渐近量级。

答案与解析：每次循环 x 翻倍，第 k 次后为 2^(k+1)；退出时达到 n/2，循环 Θ(log n)，故选第一项。n≤4 时不执行循环，不影响渐近结论。辅助空间 O(1)。原题仅问 O 上界时，多个选项都成立；本版改为 Θ。

Day01-2：最紧复杂度：平方界循环

题源：来源待核实。 保留的原题与原解析；原始资料 Day01.md。

修订题设：设 n 为非负整数，运算采用不会溢出的数学整数、每次运算 O(1)。x=0，执行 `while n≥(x+1)^2: x=x+1`；求时间的最紧 Θ 量级。

答案与解析：循环结束时 x=floor(sqrt(n))，故时间 Θ(sqrt n)，辅助空间 O(1)。不能将可能溢出的 C int 平方直接视为任意规模数学整数；原选项中的较宽 O 上界也成立。

Day01-3：最紧复杂度：递归阶乘

题源：来源待核实。 保留的原题与原解析；原始资料 Day01.md。

修订题设：设 n≥0，递归 fact(n) 在 n≤1 时返回 1，否则返回 n×fact(n−1)；按单位成本的无溢出整数模型求最紧时间和辅助空间。

答案与解析：n≥1 时经历 n,n−1,...,1 共 n 次调用，n=0 为一次调用；时间 Θ(n+1)，栈空间 Θ(n+1)，渐近写 Θ(n)。实际固定宽度 int 的阶乘可能溢出，此处仅分析规定模型。

Day02-1：线性表选择题：平均移动的前提

题源：来源待核实。 保留的原题与原解析；原始资料 Day02.md。

修订题设：考虑顺序表和单链表。假设插入的 n+1 个合法位置等概率、删除的 n 个已有位置等概率（n≥1）。判断：A 两种结构都按序号 O(1) 随机访问；B 每个元素都有前驱后继；C 一种逻辑结构只能有一种存储结构；D 顺序表连续存储，插删平均移动约半数元素。

答案与解析：D 正确：插入平均 n/2，删除平均 (n−1)/2；链表通常不能按序号随机访问，首尾不都有邻接，一种逻辑结构可有多种存储结构。概率前提不可省略。

Day02-3：删除单链表数据前驱

题源：来源待核实。 保留的原题与原解析；原始资料 Day02.md。

修订题设：带头结点单链表中，p 指向一个数据结点，其直接前驱也是数据结点，p 还有后继。设计删除 p 的直接前驱的方法，不删除头结点。

答案与解析：从 head 出发找 q，使 q.next.next=p；令 r=q.next，执行 q.next=p 后释放 r。定位最坏 O(n)、辅助 O(1)；要检查合法链及指针。原题若 p 是首元则前驱为头结点，原操作不能删除数据前驱。

Day02-5：等长有序数组的平均中位数

题源：来源待核实。 保留的原题与原解析；原始资料 Day02.md。

修订题设：两个非递减数组 A、B 均长 n≥1，允许重复，合并后共 2n 项；中位数定义为第 n 与第 n+1 小之和除以 2，结果允许非整数。不得修改输入；比较与算术按不会溢出的模型计算。

答案与解析：分别对两数组使用按秩淘汰法找第 n 小与第 n+1 小，再平均：每次比较最多淘汰约半个当前秩，最坏 O(log n) 时间、O(1) 辅助空间；也可归并扫描前 n+1 项用 O(n) 时间。例 [1,3]、[2,4] 得 (2+3)/2=2.5。具体实现求和应使用足够宽类型；此题不同于只找第 n 小。

Day03-4：同一容量下的共享双栈

题源：来源待核实。 保留的原题与原解析；原始资料 Day03.md。

修订题设：两个栈共用 M 个数组槽；top0 从 −1 向右，top1 从 M 向左，非空时两者均指向各自栈顶。写判空、判满、入出栈规则，并与总容量同为 M 的固定分区双栈比较。

答案与解析：空分别 top0=−1、top1=M；满为 top0+1=top1。左栈入栈 `A[++top0]=x`，右栈入栈 `A[--top1]=x`；出栈先各自判空，再取 `A[top0--]` 或 `A[top1++]`。每次 O(1)，数组空间 O(M)。共享容量灵活，固定分区可能一侧满而另一侧有空槽；不能与总计 2M 槽的方案直接比较容量利用率。

Day04-5：重复子串和可重叠匹配

题源：来源待核实。 保留的原题与原解析；原始资料 Day04.md。

修订题设：给定字符序列 s、t；允许 s 为空，要求 t 非空。判断 s 是否为非空子串重复至少两次，若是返回最短重复单元与次数；并统计 t 在 s 中可重叠出现次数和首次 0 基位置，未命中返回 −1。

答案与解析：s 为空时重复判断为否。否则构造 π，p=n−π[n−1]；仅 p<n 且 n mod p=0 时重复，次数 n/p。对 t 构造 π 并扫 s，匹配到一次后记录位置，令 j=π[m−1] 继续，故不遗漏重叠。例 s=aaaa、t=aa，单元 a 重复 4 次，匹配位置 0、1、2。时间 O(n+m)，辅助 O(n+m)（分别保存 π）；空模式不在本题输入范围。

Day05-1：完全二叉树高度

题源：来源待核实。 保留的原题与原解析；原始资料 Day05.md。

修订题设：一棵完全二叉树有 1000 个结点，高度定义为从根开始的层数，求高度。

答案与解析：2^9−1=511 <1000≤1023=2^10−1，所以高度 10；等价于 floor(log2 1000)+1。若按根到最深叶的边数定义高度则为 9，不能混用口径。

Day07-1：无向图性质选择题

题源：来源待核实。 保留的原题与原解析；原始资料 Day07.md。

修订题设：允许无向多重边，不考虑自环。判断：A 每个顶点度均为 2 的图一定是简单图；B 度数和等于边数；C 可有恰好 3 个奇度顶点；D 有 n 个顶点的连通图的生成树含 n−1 条边。

答案与解析：D 正确。两顶点之间有两条平行边时各点度为 2 但不是简单图，A 错；握手定理为度和 2|E|，奇度点个数为偶数，B、C 错。修订 A 的前提可实现，避免原命题前提矛盾造成形式逻辑歧义。

Day07-2：邻接矩阵平方计数

题源：来源待核实。 保留的原题与原解析；原始资料 Day07.md。

修订题设：有向无权图的 0/1 邻接矩阵为 A，允许游走重复顶点。$A^2[i,j]$ 在普通整数矩阵乘法下计数什么？

答案与解析：$A^2[i,j]=\Sigma_k A[i,k]A[k,j]$，逐一数中间顶点 k，故计 i 到 j 长度恰为 2 的有向游走数。例 i→j、j→i 有长度 2 的回到 i 的游走，但不意味着存在自环，也不等同于简单路径。

Day07-5：有向图连通、拓扑与最短路

题源：来源待核实。 保留的原题与原解析；原始资料 Day07.md。

修订题设：顶点 0..6，弧为 0→1→2→3→4→5→0 和 5→6。求入出度、强连通性和拓扑序；再讨论单源最短路的边权前提。

答案与解析：出度 [1,1,1,1,1,2,0]，入度均为 1；0..5 是一个强连通分量、6 单独成分量。存在 0→…→5→0，故无拓扑序。有向环不妨碍非负权 Dijkstra；存在负边时一般不能保证 Dijkstra。目标不可达则不存在路径；源可达的负环还需能够到达目标，才令该点对的最短距离无下界。

Day08-4：二分查找的成功与失败 ASL

题源：来源待核实。 保留的原题与原解析；原始资料 Day08.md。

修订题设：有序表为 5,10,…,60，共 12 项，使用闭区间 low/high 与 mid=floor((low+high)/2)。假设 12 个成功键条件等概率、13 个失败间隙条件等概率，仅计与真实键比较。求两个 ASL 和判定树高度。

答案与解析：每层结点数 1、2、4、5，成功比较和 1+4+12+20=37，成功 ASL=37/12。失败间隙中 3 个需 3 次、10 个需 4 次，失败 ASL=49/13；高度 4。概率分布改变时须按条件概率加权；比较成本不是实际运行时间倍数。

Day08-5：线性探测及 ASL

题源：来源待核实。 保留的原题与原解析；原始资料 Day08.md。

修订题设：长 16 的散列表，H(k)=k mod 13，依次插入 19,14,23,1,68,20,84,27,55,11,10,79，线性探测可回绕。成功键等概率；失败时 H 的 0..12 等概率。计访问槽次数，含最后空槽。

答案与解析：表地址 0..15 为 [空,14,1,68,27,55,19,20,84,79,23,11,10,空,空,空]。成功访问次数总和 30，ASL=30/12；失败各起点次数 [1,13,12,11,10,9,8,7,6,5,4,3,2]，总 91，ASL=91/13=7。若只计真实键比较，失败最后空槽不计，均值为 78/13=6。删除可设墓碑或按正确算法重组探测簇，不能随意置为从未使用；聚集使期望复杂度依散列模型与负载而异。

Day09-1：排序某一趟的辨析

题源：来源待核实。 保留的原题与原解析；原始资料 Day09.md。

修订题设：初始 49,38,65,97,76,13,27,49。直接插入与标准从左向右冒泡按常规趟定义；快排限定首元素为枢轴并做一次正确分区；选择排序每趟选剩余最小值与首位交换。某趟结果 13,38,65,97,76,49,27,49，问符合哪一算法的首趟。

答案与解析：简单选择首趟取 13 与首位 49 交换即为目标，选简单选择。限定首枢轴 49 时，目标中任何 49 左侧仍含大于 49 的数，不能作为已正确分区的枢轴。若允许先把 13 选作枢轴，则快排首趟也可得到该状态，原题不能唯一选择。

Day09-4：首枢轴挖坑划分

题源：来源待核实。 保留的原题与原解析；原始资料 Day09.md。

修订题设：输入 49,38,65,97,76,13,27,49；枢轴取首项。右扫描跳过 ≥pivot，左扫描跳过 ≤pivot，且始终检查 i<j；填坑后移动指针。写首次划分并讨论复杂度。

答案与解析：右侧跳过末尾相等的 49，以 27 填左坑；65 填右坑；13 填左坑；97 填右坑；最终在下标 3 放 49，得 [27,38,13,49,76,97,65,49]。单次 O(n)、辅助 O(1)；递归快排平衡栈 O(log n)，最坏时间 O(n^2)、栈 O(n)。随机枢轴和三数取中改善某些输入，但不保证消除最坏；堆排序或带堆回退的方案才可提供相应最坏界。

Day09-5：建堆、调整与稳定性

题源：来源待核实。 保留的原题与原解析；原始资料 Day09.md。

修订题设：对 [49,38,65,97,76,13,27,49] 自底向上建大根堆；相等孩子优先选左。给出初堆、两次取顶后的数组，并证明建堆 O(n)。

答案与解析：初堆 [97,76,65,49,49,13,27,38]；第一次调整 [76,49,65,38,49,13,27 |97]；第二次 [65,49,27,38,49,13 |76,97]。高度至少 k 的完全二叉树结点至多 floor(n/2^k)，所有结点的最大下沉层数之和 ≤Σ(k≥1)floor(n/2^k)<n，因此建堆 O(n)。2a,2b,1c 可排为 1c,2b,2a，证明不稳定。

Day10-1：明确缓冲预算的四路外排

题源：来源待核实。 保留的原题与原解析；原始资料 Day10.md。

修订题设：10000 条记录，每块 100 条；初始段生成时工作区 100 条，生成 100 个整块段。归并阶段另提供 4 个输入整块和 1 个输出整块缓冲及额外控制内存；每趟所有记录读写一次，最终结果写盘。求归并趟数与包括生成阶段的记录级读写次数。

答案与解析：段数依次 100→25→7→2→1，共 4 趟；生成和 4 趟各读写 10000，合计 2×10000×5=100000 次记录访问。单独 100 条工作区无法同时承担该模型的五块归并缓冲；块 I/O 与记录访问口径须分开。

Day10-2：败者树比较的明确口径

题源：来源待核实。 保留的原题与原解析；原始资料 Day10.md。

修订题设：8 路均衡败者树的初始胜者已经确定；输出第一个胜者后，直到产生第 n 个输出恰有 n−1 次调整（n≥1）。每次调整固定比较完整根路径三层，空路标记比较也计一次；不计初始建树和最后一次输出后的调整。求比较次数。

答案与解析：8=2^3，每次路径三层，n−1 次共 3(n−1)。若只数真实关键字比较、末次后也调整或空路被跳过，则次数不同；n=1 在本约定下为 0。

Day10-3：三路最佳归并树

题源：来源待核实。 保留的原题与原解析；原始资料 Day10.md。

修订题设：8 个段长 9,30,12,18,6,24,15,21；允许任意选择至多三段合并，不要求跨段稳定性；目标只最小化归并阶段记录级读写总数，每次参与归并的记录完整读写一次。可补零权虚段使各内部结点恰有三孩子。

答案与解析：补 1 个零权段：合并 (0,6,9)→15，(12,15,15)→42，(18,21,24)→63，(30,42,63)→135；WPL=15+42+63+135=255，归并记录级读写 510。第一次的零权段也可解释为合并两条真实段，并非两孩子内部结点不合法。逐趟等深平衡归并、要求相邻段稳定合并或不同块 I/O 模型需另算。

习题索引

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

题目总索引

按教材实际分级练习自动整理；自编题按原文位置区分，LeetCode 按题号合并。同一题的各章引用均保留。下列“提示”表示文字中可见答案或方向，不能等同完整题解；来源及答案状态如未注明，均需逐题核验。平台题的完整条件以官方题面为准。

六类自编综合题解 | 原题来源待核实

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-001	自编：学生记录中的学号是数据项还是数据元素？一条学生记录是什么？提示：按字段与整体的组成关系判断。；自编	基础	算法基础与复杂度.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-002	自编：代码顺序执行一个 n 次循环与一个 2n 次循环，求时间和常数变量的辅助空间。提示：相加得 Θ(n)，辅助空间 O(1)。；自编	基础	算法基础与复杂度.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-003	自编：无序表 n 个元素等概率成功命中，顺序扫描平均比较多少次？提示：(n+1)/2；写明成功和等概率条件。；自编	基础	算法基础与复杂度.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-509	LeetCode 509 斐波那契数；LeetCode 509	进阶	算法基础与复杂度.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-53	LeetCode 53 最大子数组和；LeetCode 53	进阶	算法基础与复杂度.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-006	自编：对 `[-3,-1,-4]` 分别按“非空”和“允许空区间”计算最大和。提示：分别为 -1 和 0；两者是不同问题。；自编	进阶	算法基础与复杂度.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-007	自编：某日报系统输入一次 100 万条记录，也可能逐条接收数据。比较二重枚举与线性递推，讨论输入存储、内存、吞吐和高分位延迟；给出至少两种实测指标。提示：线性算法的辅助空间 O(1；自编	综合	算法基础与复杂度.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-008	自编：对空表依次头插 1、2、3，删除首元后是什么？提示：2→1。；自编	基础	单链表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-009	自编：带头结点的空表满足什么条件？不带头结点的空表呢？提示：分别为 `head.next=null` 和 `head=null`。；自编	基础	单链表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-010	自编：已知前驱删除其后继时，若后继为 null，应该如何处理？提示：报告不存在，不解引用空值。；自编	基础	单链表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
LC-206	LeetCode 206 反转链表；LeetCode 206	进阶	单链表.md 进阶	有答案/提示	完整题解	题面待逐条复验
LC-141	LeetCode 141 环形链表；LeetCode 141	进阶	单链表.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-013	自编：删除 1→2→2→3→2 中所有 2，并说明连续匹配时前驱为何不前移。提示：结果 1→3，时间 O(n)、辅助空间 O(1)。；自编	进阶	单链表.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-707	LeetCode 707 设计链表；LeetCode 707	综合、进阶	单链表.md 综合、双向循环与静态链表.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-015	自编：循环单链表只有数据结点 A 时，head.next 与 A.next 分别是什么？提示：A 和 head。；自编	基础	双向循环与静态链表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-016	自编：循环双链表只含 x，删除后 head 的两个链接指向哪里？提示：都指向 head。；自编	基础	双向循环与静态链表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-017	自编：静态链表 next[0]=2，next[2]=-1、head=0 的逻辑槽位顺序是什么？提示：0、2；下标不要求连续。；自编	基础	双向循环与静态链表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-018	自编：循环单链表已知 tail，为何尾插 O(1) 但删除尾结点一般仍为 O(n)？提示：删除还要定位前驱。；自编	进阶	双向循环与静态链表.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-019	自编：原静态链案例中插入 C 后删除槽位 1 并回收；写出两条链。提示：数据链 0→2→-1，空闲链 1→3→-1。；自编	进阶	双向循环与静态链表.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-146	LeetCode 146 LRU 缓存；LeetCode 146	综合	双向循环与静态链表.md 综合	有答案/提示	未建立	题面待逐条复验
自编-021	自编：长度 6 的表在下标 2 插入、删除该位置原元素，分别移动几个已有元素？提示：4 个、3 个。；自编	基础	顺序表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-022	自编：容量 5、长度 0 的空表，插入位置 0 是否有效？删除位置 0 呢？提示：插入有效，删除无效。；自编	基础	顺序表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-023	自编：等概率删除长度为 7 的任意元素，平均移动次数是多少？提示：(7-1)/2=3，只统计已有元素移动。；自编	基础	顺序表.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-27	LeetCode 27 移除元素；LeetCode 27	进阶	顺序表.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-88	LeetCode 88 合并两个有序数组；LeetCode 88	进阶	顺序表.md 进阶、归并与基数排序.md 进阶	有答案/提示	完整题解	题面待逐条复验
自编-026	自编：长度 4、容量 4，按翻倍扩容后在头部插入 9；分别统计本例复制和移动已有元素的次数。提示：若扩容分配新数组，复制 4 次、头插移动 4 次；原地扩容时复制次数可不同。；自编	进阶	顺序表.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-027	自编：订单系统需要高频按序扫描、少量中间插入和偶发大批追加。比较动态数组与链表，并说明为何仅凭尾插均摊 O(1) 不能保证单次延迟。提示：考虑定位、缓存局部性、扩容和分配失败。；自编	综合	顺序表.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-028	自编：2×3 行优先数组，A[1,2] 的元素偏移是几？答：5。；自编	基础	数组与矩阵.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-029	自编：4×4 对称矩阵下三角含对角线共几项？答：10。；自编	基础	数组与矩阵.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-030	自编：n=3 的三对角矩阵最多保存几个带内位置？答：7。；自编	基础	数组与矩阵.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-031	自编：按上述对称例计算 A[1,3] 的压缩下标和值。答：交换为 (3,1)，k=3，值 7。；自编	进阶	数组与矩阵.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-867	LeetCode 867 转置矩阵；LeetCode 867	进阶	数组与矩阵.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-033	自编：两条三元组 `(0,2,5),(2,0,7)` 的 3×3 矩阵，A[1,1] 是多少？答：0。；自编	进阶	数组与矩阵.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-034	自编：给定 10,000×10,000 矩阵、仅 1,000 个非零项，分别估算稠密元素槽和三元组数，讨论哪种操作适合何种结构。提示：前者 10^8 槽，后者 1,000 条加元数据；自编	综合	数组与矩阵.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-035	自编：依次 push A、B，pop 后栈顶是什么？答：A。；自编	基础	栈与递归.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-036	自编：容量 8 的顺序栈、top 指向栈顶且空时 -1；何时满？答：top=7。；自编	基础	栈与递归.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-037	自编：([ )] 是否匹配？答：否，遇) 时栈顶为 [。；自编	基础	栈与递归.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-20	LeetCode 20 有效的括号；LeetCode 20	进阶	栈与递归.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-150	LeetCode 150 逆波兰表达式求值；LeetCode 150	进阶	栈与递归.md 进阶	未识别	未建立	题面待逐条复验
自编-040	自编：后缀 `8 3 - 2 *` 求值。答：(8−3)×2=10；时间 O(n)、辅助空间 O(n)。；自编	进阶	栈与递归.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-041	自编：用两个栈建模浏览器后退/前进，说明新页面访问后为什么清空前进栈，并手动追踪 A→B→C、后退一次、访问 D。答：当前 D，后退历史 A、B，前进历史空；模型需为每个标签页维护；自编	综合	栈与递归.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-042	自编：M=6、牺牲一槽，实际容量是多少？答：5。；自编	基础	队列与双端队列.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-043	自编：上述方案 front=4、rear=1，元素有多少？答：(1−4+6) mod 6=3。；自编	基础	队列与双端队列.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-044	自编：带头结点链队列删除最后一个元素后 rear 应指向哪里？答：头结点。；自编	基础	队列与双端队列.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-622	LeetCode 622 设计循环队列；LeetCode 622	进阶	队列与双端队列.md 进阶	未识别	未建立	题面待逐条复验
LC-232	LeetCode 232 用栈实现队列；LeetCode 232	进阶	队列与双端队列.md 进阶	未识别	未建立	题面待逐条复验
自编-047	自编：计数法 M=4，front=rear 且 size=4 时队列空还是满？答：满；模公式单独会误得 0。；自编	进阶	队列与双端队列.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-048	自编：生产者每秒稳定提交 10 个任务，消费者每秒完成 8 个；有限队列长期会怎样？设计满时策略并说明是否会丢任务。提示：不考虑其他因素时积压平均每秒增加 2，有限容量终会用尽；阻；自编	综合	队列与双端队列.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-049	自编：P=aba 的 π 是什么？答：`[0,0,1]`。；自编	基础	BF与KMP.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-050	自编：本文 KMP 中空模式首次匹配位置？答：0；这是 API 约定。；自编	基础	BF与KMP.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-051	自编：P=ababd 的 1 基 next[4] 是多少？答：2，对应 π[2]+1。；自编	基础	BF与KMP.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-28	LeetCode 28 找出字符串中第一个匹配项的下标；LeetCode 28	进阶	BF与KMP.md 进阶、串的表示.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-459	LeetCode 459 重复的子字符串；LeetCode 459	进阶	BF与KMP.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-054	自编：在 aaaa 中找 aa，允许重叠时位置为多少？答：0、1、2；匹配后令 j=π[1]=1。；自编	进阶	BF与KMP.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-055	自编：将文本分成 ababc 与 ababd 两块，逐块执行同一次 KMP 扫描，说明需要跨块保留什么。答：保存模式 π、当前匹配长度 j 和全局偏移；本例在第一块处理完；自编	综合	BF与KMP.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-056	自编：" " 和空串长度是否相同？答：不相同；前者有一个空格字符。；自编	基础	串的表示.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-057	自编：`"abcde"[1,4)` 是什么？答：`bcd`。；自编	基础	串的表示.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-058	自编：`ace` 是 `abcde` 的子串吗？答：不是连续子串，是子序列。；自编	基础	串的表示.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-059	自编：`"aaaa"` 中 `"aa"` 的可重叠出现位置？答：0、1、2。；自编	进阶	串的表示.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-060	自编：复制 `S[2,2)` 的结果是什么？答：空串；按本文约定合法。；自编	进阶	串的表示.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-061	自编：日志由很多短行组成，比较把所有内容连接成一个大串与按块保存的优缺点；讨论跨块匹配、复制成本和下标含义。答案取决于读取、修改和搜索负载，不能只依据一种大 O 结论。；自编	综合	串的表示.md 综合	未识别	未建立	来源按正文标注；答案待逐题核验
自编-062	自编：依次插入 2、1、3，中序序列？答：1、2、3。；自编	基础	BST与AVL.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-063	自编：在上述约定下，空树查找结果？答：null。；自编	基础	BST与AVL.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-064	自编：左子树高 3、右子树高 1 时 BF？答：2，当前结点失衡。；自编	基础	BST与AVL.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-98	LeetCode 98 验证二叉搜索树；LeetCode 98	进阶	BST与AVL.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-110	LeetCode 110 平衡二叉树；LeetCode 110	进阶	BST与AVL.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-067	自编：依次插入 10、30、20，第一个失衡结点的类型？答：RL，先对 30 右旋、再对 10 左旋。；自编	进阶	BST与AVL.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-068	自编：给出只含 1..n 的插入顺序，使普通 BST 高度为 n；再讨论需要有序动态更新时 AVL 与排序数组的查找、插入和范围遍历权衡。答：升序插入会右斜；数组查找 O(log ；自编	综合	BST与AVL.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-069	自编：频次 2、3、4，先合并哪两个？答：2、3，得到 5。；自编	基础	哈夫曼与并查集.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-070	自编：`parent=[0,0,1]` 中 2 的根是谁？答：0，不是直接父结点 1。；自编	基础	哈夫曼与并查集.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-071	自编：前缀码 `0,10,11` 是否存在一个编码是另一个编码的前缀？答：不存在。；自编	基础	哈夫曼与并查集.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-072	自编：权 2、3、4 的 WPL 是多少？答：先 2+3=5，再 4+5=9，WPL=5+9=14。；自编	进阶	哈夫曼与并查集.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-547	LeetCode 547 省份数量；LeetCode 547	进阶	哈夫曼与并查集.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-074	自编：按秩合并时两根秩相同，为什么只增加新根的秩？答：合并后最高路径可能增加一层，另一根已不再是根。；自编	进阶	哈夫曼与并查集.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-075	自编：对包含相同权值的字符集，构造两种左右 0/1 分配，比较 WPL、前缀性和逐位解码；说明码表本身及消息分布可能如何影响实际压缩收益。；自编	综合	哈夫曼与并查集.md 综合	未识别	未建立	来源按正文标注；答案待逐题核验
自编-076	自编：只有根的一棵树高度、边数和叶子数各是多少？答：1、0、1。；自编	基础	树与二叉树基础.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-077	自编：非空二叉树 $n_2=4$ 时 n_0 是多少？答：5。；自编	基础	树与二叉树基础.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-078	自编：完全二叉树 n=10，编号 4 的左右孩子？答：8、9。；自编	基础	树与二叉树基础.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-104	LeetCode 104 二叉树的最大深度；LeetCode 104	进阶	树与二叉树基础.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-080	自编：判断编号 1、2、3、5 的形态是否为完全二叉树。答：否，编号 4 有空洞而 5 存在。；自编	进阶	树与二叉树基础.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-081	自编：高度为 4 的二叉树最多多少结点？答：15；至少 4 个结点。；自编	进阶	树与二叉树基础.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-082	自编：比较用连续数组和链式结点保存 100 个结点的完全树与极度倾斜树，讨论空间、找孩子和插入位置成本；说明必须先明确编号和操作条件。；自编	综合	树与二叉树基础.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-083	自编：上例 B 的第一个孩子和下一个兄弟分别是谁？答：D、C。；自编	基础	树与森林.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-084	自编：双亲数组中的根需要什么约定？答：用 -1 等不与合法下标冲突的标记。；自编	基础	树与森林.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-085	自编：孩子兄弟表示中，原树叶子一定没有二叉树右孩子吗？答：不一定，右链接可能指向它的兄弟。；自编	基础	树与森林.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-086	自编：树 R 的孩子依次为 X、Y、Z，列出 firstChild 和 nextSibling。答：R.firstChild=X，X.nextSibling=Y，Y.nextSib；自编	进阶	树与森林.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-087	自编：仅保存双亲数组，列出某结点所有孩子最坏要多久？答：扫描 n 个结点为 O(n)，若另建索引则另论。；自编	进阶	树与森林.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-088	自编：森林 A(B,C)、D(E) 的森林先序是什么？答：A B C D E。；自编	进阶	树与森林.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-089	自编：设计文档目录的表示，要求频繁列出全部子项与偶尔回溯上级；比较孩子列表和额外双亲指针的空间、维护及访问成本，并说明出现快捷链接时为什么需要图模型。；自编	综合	树与森林.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-090	自编：上图中序是什么？答：D B E A C。；自编	基础	遍历与线索.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-091	自编：空树的层序输出是什么？答：空序列。；自编	基础	遍历与线索.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-94	LeetCode 94 二叉树的中序遍历；LeetCode 94	基础	遍历与线索.md 基础	未识别	未建立	题面待逐条复验
LC-102	LeetCode 102 二叉树的层序遍历；LeetCode 102	进阶	遍历与线索.md 进阶	未识别	未建立	题面待逐条复验
LC-105	LeetCode 105 从前序与中序遍历序列构造二叉树；LeetCode 105	进阶	遍历与线索.md 进阶	未识别	未建立	题面待逐条复验
自编-095	自编：上图 E 的中序前驱、后继是谁？答：B、A；说明如何区分线索和孩子。；自编	进阶	遍历与线索.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-236	LeetCode 236 二叉树的最近公共祖先；LeetCode 236	综合	遍历与线索.md 综合	有答案/提示	未建立	题面待逐条复验
自编-097	自编：有向边 A→B 对 A 的出度和 B 的入度各贡献多少？答：各 1。；自编	基础	图的表示.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-098	自编：本案例中 B 的度和图的边数？答：3 和 6。；自编	基础	图的表示.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-099	自编：权值 0 能否一律当作无边？答：不能，可能是合法零权边。；自编	基础	图的表示.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-1971	LeetCode 1971 寻找图中是否存在路径；LeetCode 1971	进阶	图的表示.md 进阶、图的遍历.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-101	自编：图由三角形和孤立点组成，为什么不是树？答：既有环又不连通，虽然 E=V−1。；自编	进阶	图的表示.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-102	自编：有向图若需要频繁列出某个顶点的入边，普通出边邻接表与十字链表有什么差异？提示：前者可能全图扫描，后者直接沿入弧链走。；自编	进阶	图的表示.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-103	自编：设计一个包含单行路、双向路和旅行时间的站点网络。比较邻接矩阵、邻接表和边集数组对查询邻居、查询指定边、按费用排序的成本；说明如何区分零权边和无边。；自编	综合	图的表示.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-104	自编：本例 A 的 BFS 第 1 层有哪些点？答：B、C。；自编	基础	图的遍历.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-105	自编：BFS 为什么在入队时标记？答：避免不同父顶点重复将同一顶点入队。；自编	基础	图的遍历.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-106	自编：从 A 在本例递归 DFS 的第三个访问顶点是什么？答：D。；自编	基础	图的遍历.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-200	LeetCode 200 岛屿数量；LeetCode 200	进阶	图的遍历.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-108	自编：给本例加孤立点 F，从 A 开始 BFS 会访问 F 吗？答：不会；整图遍历外层需再从 F 启动。；自编	进阶	图的遍历.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-109	自编：地图只允许经过双向道路，如何用 BFS 输出 A 到 E 的一条最少边数路线？提示：首次入队时记录前驱；本例按字母顺序得到 A→B→E，距离 2。若道路有不同费用，解释为何要；自编	综合	图的遍历.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-110	自编：A→B 时拓扑序能否将 B 放在 A 前？答：不能。；自编	基础	拓扑排序与关键路径.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-111	自编：有向环是否存在拓扑序？答：不存在。；自编	基础	拓扑排序与关键路径.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-112	自编：本例工程的最早完成时间？答：8。；自编	基础	拓扑排序与关键路径.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-207	LeetCode 207 课程表；LeetCode 207	进阶	拓扑排序与关键路径.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-210	LeetCode 210 课程表 II；LeetCode 210	进阶	拓扑排序与关键路径.md 进阶	未识别	未建立	题面待逐条复验
自编-115	自编：本 AOE 案例活动 1→3 的最早开始、最迟开始及总时差？答：3、4、1。；自编	进阶	拓扑排序与关键路径.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-2050	LeetCode 2050 并行课程 III；LeetCode 2050	综合	拓扑排序与关键路径.md 综合	有答案/提示	未建立	题面待逐条复验
自编-117	自编：5 个顶点的生成树有几条边？答：4。；自编	基础	最小生成树.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-118	自编：非连通无向图是否有覆盖全部顶点的生成树？答：没有。；自编	基础	最小生成树.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-119	自编：边权可否为负？答：最小生成树可以使用负权边。；自编	基础	最小生成树.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-1584	LeetCode 1584 连接所有点的最小费用；LeetCode 1584	进阶	最小生成树.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-121	自编：本例 Prim 从 A 首先选哪条边？答：AB，权 2。；自编	进阶	最小生成树.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-122	自编：Kruskal 已选 BC、CD、AB、DE 后再看 BD，应否加入？答：否，两端已连通。；自编	进阶	最小生成树.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-123	自编：将上例 AC 的权值改为 2，重新计算两种算法；说明边权相同时最小树可能不唯一，但不能仅凭存在相同边权就断言一定不唯一。比较铺设总费用和两点最短路的不同目标。提示：选 BC、；自编	综合	最小生成树.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-124	自编：无权图求最少边数通常可用什么算法？答：BFS。；自编	基础	最短路径.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-125	自编：Dijkstra 的不可达顶点距离和前驱如何表示？答：∞ 和 —。；自编	基础	最短路径.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-126	自编：Floyd 是否允许负权边？答：允许，但相关负权环会使部分点对不存在有限最短距离。；自编	基础	最短路径.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-743	LeetCode 743 网络延迟时间；LeetCode 743	进阶	最短路径.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-1091	LeetCode 1091 二进制矩阵中的最短路径；LeetCode 1091	进阶	最短路径.md 进阶	未识别	未建立	题面待逐条复验
自编-129	自编：本页 Dijkstra 案例中，B 被确定后 dist[C] 和 pred[C] 是什么？答：3 和 B。；自编	进阶	最短路径.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-130	自编：给出有向边 A→B=2、A→C=5、C→B=−10；说明标准 Dijkstra 为何不能可靠确定 B，并比较使用 Bellman–Ford 的条件。提示：B 可能先被确定为 ；自编	综合	最短路径.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-131	自编：本页四阶 B 树一个节点最多几个键？答：3。；自编	基础	B树与B加树.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-132	自编：本页 B+ 树查找命中内部 30 后是否可以直接返回记录？答：不能，进入对应右子树叶节点。；自编	基础	B树与B加树.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-133	自编：本页非根 B+ 叶节点最少几条记录？答：2。；自编	基础	B树与B加树.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-134	自编：本页 B 树初始 [20|40] 的右叶 [50,60] 查 60，节点访问几次？答：2 次；键比较数取决于节点内搜索方法。；自编	进阶	B树与B加树.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-135	自编：本页 B+ 树插入 40 后，为什么 30 同时出现在父节点和右叶？答：父节点复制分隔键，叶中保留真实记录。；自编	进阶	B树与B加树.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-136	自编：上述 B+ 删除 50 后为什么父分隔键从 40 改为 35？答：借位后右叶最小键为 35。；自编	进阶	B树与B加树.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-137	自编：订单表有大量等值查询和编号范围扫描，比较散列表与本页 B+ 树；讨论页缓存、叶链、索引维护和写入分裂。不要声称散列绝不可以通过额外排序支持范围查询，也不要假定数据库必然选择某；自编	综合	B树与B加树.md 综合	未识别	未建立	来源按正文标注；答案待逐题核验
自编-138	自编：m=7，h(k)=k mod 7，解释 10 与 17 为何冲突，并计算 5 条记录的装填因子。；自编	基础	散列表.md 基础	未识别	未建立	来源按正文标注；答案待逐题核验
自编-139	自编：为什么开放定址删除必须区分从未使用和已删除？用上面的 17、24 说明。；自编	基础	散列表.md 基础	未识别	未建立	来源按正文标注；答案待逐题核验
自编-140	自编：比较拉链法与线性探测在存储位置、删除处理和 α 上的区别。；自编	基础	散列表.md 基础	未识别	未建立	来源按正文标注；答案待逐题核验
自编-141	自编：依次插入 6、13、20、7 到长度 7 的线性探测表，写出各次探测和最终表；假设等概率查找已插入键，计算成功 ASL 并说明比较口径。；自编	进阶	散列表.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
LC-706	LeetCode 706 设计哈希映射；LeetCode 706	进阶	散列表.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-1	LeetCode 1 两数之和；LeetCode 1	进阶	散列表.md 进阶	有答案/提示	未建立	题面待逐条复验
自编-144	自编：为一个缓存设计键到值的查询和容量限制方案。比较开放定址、拉链法，并说明扩容、删除、键分布及需要按顺序淘汰时的额外结构；分别分析期望成本和碰撞严重时的最坏成本。；自编	综合	散列表.md 综合	未识别	未建立	来源按正文标注；答案待逐题核验
自编-145	自编：无序表 5 个不同键等概率成功查找，顺序扫描成功 ASL？答：3 次真实键比较。；自编	基础	顺序折半与分块查找.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-146	自编：空数组闭区间折半查找的初始 low、high？答：0、-1，循环不进入。；自编	基础	顺序折半与分块查找.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-147	自编：哨兵查找失败，n=4、计入哨兵比较为几次？答：5 次。；自编	基础	顺序折半与分块查找.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-704	LeetCode 704 二分查找；LeetCode 704	进阶	顺序折半与分块查找.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-35	LeetCode 35 搜索插入位置；LeetCode 35	进阶	顺序折半与分块查找.md 进阶	未识别	未建立	题面待逐条复验
自编-150	自编：本页分块例查找 5，索引比较和块内比较各几次？答：2 次、3 次。；自编	进阶	顺序折半与分块查找.md 进阶	有答案/提示	未建立	来源按正文标注；答案待逐题核验
LC-34	LeetCode 34 在排序数组中查找元素的第一个和最后一个位置；LeetCode 34	综合	顺序折半与分块查找.md 综合	有答案/提示	未建立	题面待逐条复验
自编-152	自编：为什么严格大于才交换的冒泡稳定？答：相等记录不会互换。；自编	基础	交换与选择排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-153	自编：数组 0 基下，节点 2 的两个孩子下标？答：5、6，存在与否取决于长度。；自编	基础	交换与选择排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-154	自编：对 5 个元素做简单选择，比较次数？答：10。；自编	基础	交换与选择排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-155	自编：按本页末元素枢轴重做共同数组的三次分区，列出区间。；自编	进阶	交换与选择排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-156	自编：对 [2甲,2乙,1] 模拟堆排序，说明不稳定。；自编	进阶	交换与选择排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
LC-75	LeetCode 75 颜色分类；LeetCode 75	进阶	交换与选择排序.md 进阶	有答案/提示	未建立	题面待逐条复验
LC-912	LeetCode 912 排序数组；LeetCode 912	综合	交换与选择排序.md 综合	有答案/提示	未建立	题面待逐条复验
自编-159	自编：10 个初始段采用 4 路平衡归并，需几趟？答：2 趟。；自编	基础	外部排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-160	自编：按一块一输入缓冲及一块输出缓冲的模型，4 路最少几块缓冲？答：5 块，控制信息另计。；自编	基础	外部排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-161	自编：当前段输出 5，补入 3，应怎样处理？答：冻结至下一段。；自编	基础	外部排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-162	自编：复算本文 1000 条案例，分别写出生成段、每一归并趟及总块 I/O。；自编	进阶	外部排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-163	自编：复算四路败者树输出 1、补入 7 后的两次比赛。；自编	进阶	外部排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验

身份	题目与来源	难度（教材分级）	出现位置	提示	独立题解	核验状态
自编-164	自编：计算段长 2、3、4、5 的 WPL 与记录级读写量，并说明是否包含初始段生成。；自编	进阶	外部排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-165	自编：在给定输入缓冲、输出缓冲和内存预算下设计多路归并，并讨论负载不均、记录稳定性、末尾部分块与统计口径。提供一个明确预算的练习。；自编	综合	外部排序.md 综合	未识别	示例题解	来源按正文标注；答案待逐题核验
自编-166	自编：合并两个有序段时相等先取哪一段才能稳定？答：原顺序在前的左段。；自编	基础	归并与基数排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-167	自编：LSD 为什么每一位都要稳定？答：保留已处理低位的顺序。；自编	基础	归并与基数排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-168	自编：长度为零的归并排序如何终止？答：空区间直接返回。；自编	基础	归并与基数排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-169	自编：按本文两位数案例分别演示 LSD 和 MSD，并比较其分组方向。；自编	进阶	归并与基数排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-170	自编：r 个长度为 n 的预分配桶与复用链域的 O(r) 额外桶头有何区别？；自编	进阶	归并与基数排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
LC-23	LeetCode 23 合并 K 个升序链表；LeetCode 23	综合	归并与基数排序.md 综合	有答案/提示	未建立	题面待逐条复验
自编-172	自编：[2甲,1,2乙] 稳定排序后两条 2 的顺序如何？答：甲在乙前。；自编	基础	排序基础与插入排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-173	自编：直接插入已升序的 n 个键，关键字比较多少次？答：n−1（n≥1）。；自编	基础	排序基础与插入排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-174	自编：折半插入要把新记录放在相等键之前还是之后才能稳定？答：之后。；自编	基础	排序基础与插入排序.md 基础	有答案/提示	未建立	来源按正文标注；答案待逐题核验
自编-175	自编：对共同输入手算 gap=2 和 gap=1 两趟希尔排序。；自编	进阶	排序基础与插入排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-176	自编：解释为什么折半定位不把最坏移动次数降至 O(n log n)。；自编	进阶	排序基础与插入排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-177	自编：用 [2甲,2乙,1] 验证希尔排序的不稳定性。；自编	进阶	排序基础与插入排序.md 进阶	未识别	未建立	来源按正文标注；答案待逐题核验
自编-178	自编：比较几乎有序且关键字比较昂贵的列表采用直接插入、折半插入或归并的成本；说明额外空间、稳定性和移动次数。；自编	综合	排序基础与插入排序.md 综合	有答案/提示	未建立	来源按正文标注；答案待逐题核验

综合应用

线性结构综合 K101—K108 与树图综合 K106—K110为原 Day10 案例迁移，题源待核实；这些案例不计入上表的分级题数量。结构选型与工程实践提供三个自编工程案例。

原有习题修订版

原题及原解析保留在各条目中的只读摘录链接；下列题目为**修订或补充条件后的教学版本**，不冒充历年真题，来源待核实。

原题编号	教材版本	状态
Day01-1	修订题设与解析（文内 Day01-1）	已实施、已检查、待审核；来源待核实
Day01-2	修订题设与解析（文内 Day01-2）	已实施、已检查、待审核；来源待核实
Day01-3	修订题设与解析（文内 Day01-3）	已实施、已检查、待审核；来源待核实
Day02-1	修订题设与解析（文内 Day02-1）	已实施、已检查、待审核；来源待核实
Day02-3	修订题设与解析（文内 Day02-3）	已实施、已检查、待审核；来源待核实
Day02-5	修订题设与解析（文内 Day02-5）	已实施、已检查、待审核；来源待核实
Day03-4	修订题设与解析（文内 Day03-4）	已实施、已检查、待审核；来源待核实
Day04-5	修订题设与解析（文内 Day04-5）	已实施、已检查、待审核；来源待核实
Day05-1	修订题设与解析（文内 Day05-1）	已实施、已检查、待审核；来源待核实
Day07-1	修订题设与解析（文内 Day07-1）	已实施、已检查、待审核；来源待核实
Day07-2	修订题设与解析（文内 Day07-2）	已实施、已检查、待审核；来源待核实
Day07-5	修订题设与解析（文内 Day07-5）	已实施、已检查、待审核；来源待核实
Day08-4	修订题设与解析（文内 Day08-4）	已实施、已检查、待审核；来源待核实
Day08-5	修订题设与解析（文内 Day08-5）	已实施、已检查、待审核；来源待核实
Day09-1	修订题设与解析（文内 Day09-1）	已实施、已检查、待审核；来源待核实
Day09-4	修订题设与解析（文内 Day09-4）	已实施、已检查、待审核；来源待核实
Day09-5	修订题设与解析（文内 Day09-5）	已实施、已检查、待审核；来源待核实

原题编号	教材版本	状态
Day10-1	修订题设与解析（文内 Day10-1）	已实施、已检查、待审核；来源待核实
Day10-2	修订题设与解析（文内 Day10-2）	已实施、已检查、待审核；来源待核实
Day10-3	修订题设与解析（文内 Day10-3）	已实施、已检查、待审核；来源待核实

待核实题目

原资料的 50 道章末题需逐题检查来源、题干、条件、答案、推导、时间与空间复杂度及题设差异。来源不明不代表题目错误或自编；保留时须明确标注。

逐题任务与原文位置见第二阶段设计目录中的《原有50道习题核验台账.md》。

公式与术语附录

> 本资料以知识理解、案例、习题与工程应用为主。2027 年官方 408 大纲正文尚未逐项核验；题源待核实的习题不可认定为历年真题。

公式与术语速查

本附录索引正文中的结论和适用条件，不代替正文推导、案例和边界讨论。复杂度的 O 是上界，Θ 表示紧确数量级；时间默认固定宽度关键字比较为 O(1)，辅助空间不含输入和输出，其他口径在对应条目另行说明。各表中“常见错误”提示不能忽略的前提。不同章节下标约定可能不同，使用前应先检查。

1. 线性表与数组

公式或结论	符号与适用条件	常见错误	正文
地址 base+i·w	连续等宽槽、0 基下标 i 合法；base 是首地址，w 是槽宽字节数	把链表或不连续行对象套入此式	顺序表
平均移动：插入 n/2；删除 (n-1)/2	表长 n；插入的 n+1 个位置等概率、删除的 n 个现有位置等概率，删除要求 n≥1；仅计已有元素移动	当作任意访问分布成立，或把扩容复制算入同一公式	顺序表
行优先 base+(i·c+j)w；列优先 base+(j·r+i)w	r 行 c 列、0 基、整个矩阵连续且每项宽 w；i、j 是行列下标	将行数 r 与列数 c 对调	数组与矩阵
对称/下三角下标 k=i(i-1)/2+j-1；三对角 k=2i+j-3	矩阵 i、j 为 1 基，压缩数组 k 为 0 基；前式存下三角且 i≥j；后式只对合法的 `	i-j	≤1` 位置有效

2. 栈、队列与串

公式或结论	符号与适用条件	常见错误	正文
栈为后进先出；容量足够时交错出入栈的不同序列数 C_n=binom(2n,n)/(n+1)	n 个互异元素按固定顺序各入出一次，n≥0	误将任意排列都当合法出栈序列	栈与递归
循环队列牺牲一槽：空 front=rear，满 (rear+1) mod M=front，长度 (rear-front+M) mod M，容量 M−1	M≥2、front 指队头、rear 指队尾下一槽	把这一模长度公式直接用于满容量计数法或标志法	队列与双端队列
串是字符的有限序列；连接复制新串成本 O(n+m)	n、m 是输入串长；单字符复制 O(1)，结果需要新分配	将共享视图或字符编码下的字节偏移一概当作该模型	串的表示

3. 二叉树与 AVL

公式或结论	符号与适用条件	常见错误	正文
$n_0=n_2+1$	非空二叉树；n_0 为叶数、n_2 为度为 2 的结点数	将公式无条件套给空树	树与二叉树基础
第 i 层至多 2^(i-1) 结点；h 层至多 2^h-1	根为第 1 层，i≥1；h 为按层数计的高度	混用按边数计的高度	树与二叉树基础

公式或结论	符号与适用条件	常见错误	正文
非空完全二叉树高度 $\lfloor \log_2 n \rfloor + 1$，叶数 $\lceil n/2 \rceil$	$n \geq 1$；高度按层数；完全树按层从左至右无空洞	对任意形态的二叉树套用完全树公式	树与二叉树基础
AVL 平衡因子 BF=左子树高−右子树高，每结点 `	BF	≤1`	本教材空树高 0、按层数计高度；BF 对每个结点检查

4. 哈夫曼树 WPL

公式或结论	符号与适用条件	常见错误	正文
$WPL=\Sigma w_i l_i$	w_i 为第 i 个叶结点权值，l_i 为根到该叶的边数；哈夫曼树按给定非负权构造	把层数直接当深度；以给定分布的总成本最优推断每条消息一定最短	哈夫曼与并查集

5. 并查集

公式或结论	符号与适用条件	常见错误	正文
`find(x)` 找代表元，`union(a,b)` 合并两个集合；根满足 `parent[root]=root`	parent 为双亲数组；这是正文采用的根指向自身约定	把直接父结点当作根；认为它能直接恢复路径或拆分集合	哈夫曼与并查集
初始化 O(n)，随后 m 次操作总成本 $O((n+m)\alpha(n))$	n 为元素数；同时使用按秩合并与路径压缩；α 为反阿克曼函数	将操作序列均摊界误认为单次操作的最坏 O(1)	哈夫曼与并查集

6. 图遍历、最小生成树与最短路径

公式或结论	符号与适用条件	常见错误	正文
整图 DFS/BFS：邻接表 $O(V+E)$，邻接矩阵 $O(V^2)$	V 为顶点数、E 为边数；初始化与扫描整图，辅助空间 O(V)，不含图本身	未说明存储结构或把单源可达部分代价当作整图代价	图的遍历
连通无向图生成树有 $V-1$ 条边	$V \geq 1$；非连通图无覆盖全图的生成树，可求生成森林	仅由 $E=V-1$ 就断言原图必为树	图的表示、最小生成树
Dijkstra：单源、边权非负；Floyd：$D[i,j] \leftarrow \min(D[i,j], D[i,k]+D[k,j])$	i、j、k 为顶点；Floyd 的 k 为最外层并判断 ∞，允许负边；可达负权环会令相关点对无有限最短距离	将 Dijkstra 用作有负权边的一般保证；把最小生成树当作最短路径树	最短路径

7. 拓扑排序与关键路径

公式或结论	符号与适用条件	常见错误	正文
有向图存在拓扑序当且仅当无有向环	顶点代表 AOV 活动，边代表先后约束；邻接表 Kahn 实现 O(V+E)	把拓扑序直接当作最短工期	拓扑排序与关键路径
`ve[v]=max(ve[u]+w(u,v))`；`vl[u]=min(vl[v]-w(u,v))`；活动 `e=ve[u]`、`l=vl[v]-w`、时差 `l-e`	单源单汇 DAG、活动时长 w 非负，所有事件在源汇路径上；资源充足且允许并行；源 ve=0，汇 vl=ve[汇]	顶点事件与边活动混淆；认为多个活动可同时各自消耗全部共享时差	拓扑排序与关键路径

8. 查找与 ASL

公式或结论	符号与适用条件	常见错误	正文
ASL成功=$\Sigma p_i C_i$；失败同理按失败条件概率加权	p_i 是已知成功条件下查找第 i 个键的概率，$\Sigma p_i=1$；C_i 是指定口径的比较次数	未声明等概率或把成功、失败条件期望直接平均	顺序折半与分块查找
无序表顺序查找成功 ASL `(n+1)/2`	n≥1，n 个不同键等概率成功命中，只数真实键比较	直接套到非等概率、失败或含哨兵的另一计数口径	顺序折半与分块查找

9. 散列表

公式或结论	符号与适用条件	常见错误	正文
装填因子 α=n/m	n 为有效记录数；m 为开放定址槽数或拉链法桶数，m>0	认为拉链法 α 必然不大于 1；将期望 O(1) 当作每次最坏 O(1)	散列表
线性探测槽 `(h(key)+i) mod m`，i=0..m−1	h 为合法初始槽；查找至未使用槽、命中或扫描一圈；墓碑要继续查找	删除后将墓碑直接变成从未使用的空槽，截断后续查找	散列表

10. B 树与 B+ 树

公式或结论	符号与适用条件	常见错误	正文
本页引用的四阶 B 树：最多 4 个孩子、3 个键；有 c 个孩子的内部节点有 c−1 个键；非根至少 1 键	m=4 为最大分支数，c 是内部节点孩子数；非叶根至少 2 个孩子，空树单独处理，真实叶同层	将临时溢出状态当作最终合法状态；把其他教材“阶”的约定直接套入	B树与B加树
本教材四阶 B+ 变体：内部 c 孩子、c−1 个分隔键；非根叶 2–3 条记录；真实记录在有序叶链，分隔键取右子树最小键	内部最多 4 个孩子，非根内部至少 2 个；根特殊处理；相等时向右进入对应子树	把其他教材的 B+ 变体键数当作本页规则；声称 B 树不支持范围查询	B树与B加树

11. 排序复杂度与稳定性

下表对应正文的具体实现；n≥2，平均列以互异键均匀随机排列为假设，LSD 的 d、r 为键位数和基数。空间包含递归栈，通常不含输入及最终输出；若实现将暂存输出数组计入工作空间，则按条目注明。稳定性指相等关键字的原始相对次序保持不变。

算法	最好 / 平均 / 最坏时间；辅助空间	稳定性与适用条件	常见错误	正文
直接插入	Θ(n) / Θ(n²) / Θ(n²)；O(1)	稳定：严格大于才移动	将等键也向右移动后仍断言稳定	排序基础与插入排序
折半插入	Θ(n log n) / Θ(n²) / Θ(n²)；O(1)	本实现插在已有相等键后，稳定	忽略移动，误称总最坏 O(n log n)	排序基础与插入排序
希尔	随增量序列及输入而异；O(1)	本实现不稳定，末增量须为 1	将固定时间界套给所有增量序列	排序基础与插入排序
带提前终止冒泡	Θ(n) / Θ(n²) / Θ(n²)；O(1)	严格大于才交换，稳定	未设置无交换终止就套用最好 Θ(n)	交换与选择排序
末键枢轴快排	平衡 Θ(n log n) / Θ(n log n) / Θ(n²)；平衡栈 O(log n)，最坏 O(n)	不稳定；平均按互异键等概率排列	忽略递归栈或把平均界当最坏界	交换与选择排序
简单选择	Θ(n²) / Θ(n²) / Θ(n²)；O(1)	本实现不稳定	认为每趟仅一次交换就必然稳定	交换与选择排序

算法	最好 / 平均 / 最坏时间；辅助空间	稳定性与适用条件	常见错误	正文
堆排序	Θ(n)（全相等键）/ Θ(n log n) / Θ(n log n)；O(1)	本实现不稳定，平均按互异键随机排列	忽略本版全相等时的线性最好情况	交换与选择排序
数组二路归并	Θ(n log n) / Θ(n log n) / Θ(n log n)；O(n)	相等取左段，稳定	不计临时数组和递归栈	归并与基数排序
LSD 基数	Θ(d(n+r))；计数分配/输出 O(n+r)	d 位、r 个桶，每轮稳定分配并扫描桶	用比较排序的平均模型直接解释；忽略桶空间	归并与基数排序

基数排序**空间口径**：计稳定输出数组与 r 个桶头/计数器为 O(n+r)；复用已有结点链域、仅额外计桶头/尾指针为 O(r)；若预先分配 r 个各长 n 的桶槽则为 O(rn)。这三种是不同实现，不能用一个结论覆盖全部。

12. 外部排序与 I/O

公式或结论	符号与适用条件	常见错误	正文
`ceil(log_k R)` 趟	R>1 个初始段，每趟最多 k 路分组、直到一段，k≥2；R≤1 为 0 趟	将段数减少直接推成必少一趟；忽视缓冲预算	外部排序
最佳归并树 $WPL=\Sigma w_i l_i$，若每轮完整读写记录一次，归并阶段记录级总读写 2×WPL	w_i 是第 i 个初始段的记录数，l_i 是其参与归并次数；不含初始段生成，含每次归并的读和写	把 WPL 直接当读写总量，或把记录数直接等同磁盘块 I/O	外部排序
k 路整块缓冲的示例预算：k 个输入块加 1 个输出块	该模型每路一个整块缓冲，控制信息另计；块 I/O 还依赖段长、块填充和实际读写	把生成初始段的内存工作区容量直接当作归并阶段总预算	外部排序

KMP 符号与术语索引

术语或结论	符号与适用条件	常见错误	正文
`π[i]`：`P[0..i]` 最长相等真前后缀长度	P 为模式串，i 为 0 基，真前后缀不能等于整个串	把 π[i] 当作 1 基待比较字符位置	BF与KMP
`next[1]=0`；`next[j]=π[j-2]+1`（2≤j≤m）	本教材另列的 1 基教材约定，m 为模式长度；next 是失配后位置，0 为哨兵	混用 0 基 π 与 1 基 next；直接套别的教材 next 公式	BF与KMP
`nextval[1]=0`；令 k=next[j]，若 k>0 且 P[j]=P[k]，则 `nextval[j]=nextval[k]`，否则为 k	1 基 P、j≥2；跳过已知会重复失配的同一字符	把 nextval 当成前缀长度或不检查 k=0 就访问 P[k]	BF与KMP
非空串最小正周期 `p=n-π[n-1]`；完整重复需 `n mod p=0`	n 是串长；若要求至少重复两次还须 p<n	将周期与完整重复混同；将 `n-next[n]` 当通用公式	BF与KMP